

MASTER THESIS
Leonard Bardtke

Multi-Agent Deep Reinforcement Learning for Efficient Aerial Wildfire Fighting

Faculty of Computer Science and Digital Society

Leonard Bardtke

Multi-Agent Deep Reinforcement Learning for Efficient Aerial Wildfire Fighting

Master thesis submitted for examination in Master's degree
in the study course *Master of Science Informatik*
at the Faculty of Computer Science and Digital Society
at University of Applied Science Hamburg

Supervisor: Prof. Dr. Thomas Clemen

Supervisor: Prof. Dr. Jan Sudeikat

Submitted on: 10 June 2026

Leonard Bardtke

Thema der Arbeit

Multi-Agent Deep Reinforcement Learning for Efficient Aerial Wildfire Fighting

Stichwörter

Reinforcement Learning, Multi-Agent Reinforcement Learning, Multi-Agent Deep Reinforcement Learning, Agent-based Simulation, Wildfire fighting, Explainable Reinforcement Learning, Reward Decomposition, Observation Decomposition

Kurzzusammenfassung

Luftgestützte Waldbrandbekämpfung erfordert koordinierte, adaptive Entscheidungsfindung unter sich schnell verändernden Bedingungen. Diese Arbeit untersucht die Anwendung von Multi-Agent Deep Reinforcement Learning (MARL) in einer operativ fundierten agentenbasierten Waldbrandsimulation des SoSID Toolkits, mit Fokus auf systematische Integration und Interpretierbarkeit gelernter Strategien. Dazu werden Komplexitätsstufen definiert, die eine schrittweise Integration und Validierung ermöglichen, sowie das Dual Decomposition Framework (DDF) entwickelt, das Belohnungs- und Beobachtungskomponenten modularisiert und deren individuellen Einfluss auf die Policy analysiert.

Eine Ablationsstudie mit 20 Experimenten identifiziert die kritischen Beobachtungs- und Belohnungskomponenten und zeigt deren Einfluss auf die gelernte Policy. Curriculum Learning, bei dem die Feuerintensität schrittweise erhöht wird, verbessert zudem die Policy-Performance. Darüber hinaus deckte die RL-Integration zwei Implementierungsfehler auf, die anschließend behoben wurden.

Die gelernte Policy wird gegen fünf regelbasierte heuristische Taktiken des SoSID Toolkits evaluiert und übertrifft vier davon konsistent hinsichtlich der verbrannten Fläche. Die fünfte Taktik übertrifft die RL-Policy ausschließlich bei sehr hohen Brandausbreitungsraten, da ihr struktureller Vorteil in der indirekten Unterdrückungsstrategie liegt. Die Ergebnisse belegen die Wirksamkeit von MARL für die luftgestützte Waldbrandbekämpfung und etablieren eine systematische, interpretierbare Methodik für die Policy-Gestaltung.

Leonard Bardtke

Title of Thesis

Multi-Agent Deep Reinforcement Learning for Efficient Aerial Wildfire Fighting

Keywords

Reinforcement Learning, Multi-Agent Reinforcement Learning, Multi-Agent Deep Reinforcement Learning, Agent-based Simulation, Wildfire fighting, Explainable Reinforcement Learning, Reward Decomposition, Observation Decomposition

Abstract

Aerial wildfire suppression requires coordinated, adaptive decision-making under rapidly evolving conditions. This thesis investigates the application of Multi-Agent Deep Reinforcement Learning (MARL) to an operationally grounded agent-based wildfire simulation of the SoSID Toolkit, with a focus on systematic integration and interpretability of learned strategies. A staged complexity framework is introduced to enable step-by-step integration and validation, and the Dual Decomposition Framework (DDF) is developed to modularize reward and observation components and analyze their individual contributions to the learned policy.

An ablation study comprising 20 experiments identifies the critical observation and reward components and their influence on the learned policy. Curriculum learning, in which fire intensity is progressively increased during training, further improves policy performance. Additionally, the RL integration exposed two implementation errors, which were subsequently corrected.

The learned policy is evaluated against five rule-based heuristic tactics of the SoSID Toolkit, outperforming four of them consistently in terms of burned area. The fifth tactic outperforms the RL policy only at very high fire spread rates, as its structural advantage lies in the indirect suppression strategy. The results demonstrate the effectiveness of MARL for aerial wildfire suppression and establish a systematic, interpretable methodology for policy design.

Acknowledgements

This thesis was conducted in collaboration with the German Aerospace Center (DLR), and I would like to express my sincere gratitude to everyone who supported me throughout this journey.

At the Hamburg University of Applied Sciences, I would like to thank Professor Dr. Thomas Clemen for supervising this thesis and for the valuable guidance provided throughout the research process. Beyond his academic supervision, I am particularly grateful for his effort in establishing the connection to DLR, which made this collaboration possible in the first place and opened the door to a truly enriching research experience. His trust in me to take on this project and his constructive feedback were a constant source of motivation and greatly contributed to the success of this work. I would also like to acknowledge Professor Dr. Jan Sudeikat for kindly taking on the role of second examiner. Finally, I would like to thank the MARS team for bringing students together. The exchange with fellow students facilitated through this network provided a valuable source of motivation and perspective throughout my studies.

At DLR, I am especially grateful to my supervisor Nabih Naeem for the continuous support and guidance. His open ear and willingness to engage with questions and challenges at any time made a significant difference throughout the entire process. I would also like to thank my colleague Nikolaos Kalliatakis for generously sharing his expertise and providing valuable insights that greatly enriched my understanding of the subject matter. My gratitude also goes to Prajwal Prakasha for the leadership, the trust in my work, and for providing the direction and vision that shaped the scope of this research. I would also like to express my gratitude to DLR for providing me with the opportunity to present this research at the EASN Conference in Madrid. Being able to share this work with the international research community was an incredibly enriching experience and a highlight of this thesis journey. Furthermore, I wish to express my appreciation to the entire Panda Works team for their warm welcome and for creating an inclusive and motivating working environment from day one.

The research presented in this work has been performed in the framework of the COLOSSUS project (Collaborative System of Systems Exploration of Aviation Products, Services and Business Models) and has received funding from the European Union Horizon Europe program under grant agreement No. 101097120. The Swiss participation in the Colossus project is supported by the Swiss State Secretariat for Education, Research and Innovation (SERI) under contract number 22.00609.

Contents

List of Figures	ix
List of Tables	1
1 Introduction	2
1.1 Motivation	2
1.2 Contributions and Relevance	2
1.3 Project Context	4
1.4 Thesis Structure	4
2 Background	6
2.1 Reinforcement Learning	6
2.1.1 Markov Decision Process	8
2.1.2 Policy and Value Functions	8
2.1.3 Solution Methods	9
2.1.4 Deep Reinforcement Learning	9
2.1.5 Multi-Agent Reinforcement Learning	10
2.1.6 Proximal Policy Optimization	11
2.1.7 Curriculum Learning	12
2.2 SoSID Toolkit	12
2.2.1 Scenario	14
2.2.2 Wildfire Model	15
2.2.3 Suppressant Drop Model	16
2.2.4 Correction Coefficient	17
2.2.5 Baseline Simulation and Heuristic Benchmarks	18
2.2.6 Aircraft types	19
3 State of the Art	21
3.1 Aerial Wildfire Monitoring and Suppression	21

3.2	Multi-Agent Coordination Strategies	22
3.3	Explainability in Reinforcement Learning	24
3.4	Research Gap	25
4	Design	26
4.1	Multi-Agent Control Strategies	26
4.2	Policy Architecture	27
4.3	Progressive Complexity Levels	27
4.3.1	Complexity Level 1	28
4.3.2	Complexity Level 2	28
4.3.3	Complexity Level 3	28
4.3.4	Complexity Level 4	28
4.4	Dual Decomposition Framework	30
4.5	Action Space	32
4.5.1	Alternative Action Space Approaches	32
4.6	Observation Space	33
4.7	Reward Function	34
4.7.1	Reward Normalization	35
5	Implementation	36
5.1	RL Implementation	36
5.1.1	Tactic Integration	38
5.1.2	Discrete Action Movement Mapping	38
5.1.3	Reward Component Implementation	40
5.2	Refueling and Resupply Abstraction	48
5.3	Agent Position Randomization	48
5.4	Wildfire State	50
6	Experiments and Results	51
6.1	Experimental Setup	51
6.2	Hardware and Software Environment	55
6.3	Ablation Study	55
6.3.1	Phase 1: Observation Space Baseline	58
6.3.2	Phase 2: Reward Function Ablation	60
6.3.3	Phase 3: Observation Space Extension and Optimization	66
6.4	RL-Heuristic Comparison	74

7	Discussion	77
7.1	RQ1 – Systematic Complexity Structuring	77
7.1.1	RL Integration as a Validation Tool	78
7.2	RQ2 – Modular Decomposition	79
7.2.1	Phase 1: Observation Space Baseline	79
7.2.2	Phase 2: Reward Function Ablation	79
7.2.3	Phase 3: Observation Space Extension	81
7.3	RQ3 – RL vs. Heuristic Comparison	83
7.3.1	Emergent Tactic Analysis	84
7.4	Limitations	87
8	Conclusion	88
8.1	Future Work	89
	Bibliography	91
A	Appendix	97
A.1	Training input file	97
A.2	Tools Used	99
	Declaration of Authorship	100

List of Figures

2.1	The agent–environment interaction in reinforcement learning	7
2.2	Overview of the Sintra Scenario. The figure shows the broader Sintra region with the vertiports marked, from which suppression aircraft are deployed at the start of each mission and to which they return for refueling.	14
2.3	Active Area of the Sintra Scenario. The figure shows the active area in detail, within which fire propagation and suppression operations take place. The <i>Active Area</i> boundary delimits the operational region. <i>Vegetation</i> cells are flammable and represent the primary fuel source. <i>Residential Areas</i> and <i>Nonflammable</i> cells do not propagate fire. <i>Water Sources</i> mark locations where agents can resupply their water tanks. The <i>Ignition Center</i> indicates the point of fire origin. During a simulation, cells transition from vegetation to <i>Burning</i> and subsequently to <i>Burnt</i> , reflecting the progression of the wildfire.	15
2.4	Effect of the Correction Coefficient on Fire Spread. Burned area after 10,000 simulation timesteps for three values of the correction coefficient. Higher <i>cc</i> values result in significantly larger burned areas, illustrating the sensitivity of fire propagation to this parameter.	17
2.5	Visualization of the direct and indirect heuristic suppression tactics in the simulation.	19
2.6	Aircraft types used in the simulation. eVTOL (left) and seaplane (right).	19
4.1	Complexity Levels.	29
4.2	Dual Decomposition Framework. The framework consists of three stages: Problem Setting, RL Framework with Decomposition Module, and Tactic Evaluation. Blue elements correspond to the rule-based approach; green elements correspond to the RL approach.	31
4.3	The action space variations	32

5.1	Component diagram of the RL training architecture. Illustrates the integration of the RL framework into the SoSID Toolkit. Components introduced as part of this work are highlighted in green.	37
5.2	Liang–Barsky destination clipping. The black cross marks the originally computed destination outside the active area. The red cross shows the result of naive coordinate-wise clipping, which distorts the heading. The blue cross shows the clipped destination produced by the Liang–Barsky algorithm, which preserves the original direction.	39
5.3	Computation Flow of the Proximity Reward.	41
5.4	Computation Flow of the Inactivity Penalty.	43
5.5	Computation Flow of the Suppression Reward.	45
5.6	Computation Flow of the Connected Suppression Reward.	46
5.7	Effect of the Sparseness Parameter on Starting Position Distribution. Higher values reduce the density of candidate positions, concentrating agent initialization to fewer, more spread-out locations within the active area.	49
5.8	The wildfire states.	50
6.1	Observation Space Baseline Evaluation. Performance evaluation of experiments Exp.1 (red), Exp.2 (blue), Exp.3 (green), Exp.4 (yellow), Exp.5 (purple), Exp.6 (brown) and Exp.7 (pink) using a cc value of 0.10. .	59
6.2	Evaluation of Timestep- and Inactivity-Penalty. Performance evaluation of Exp.7 (red) used as a baseline without any penalty components, Exp.8 (blue) using the Timestep Penalty, Exp.9 (green) using the Inactivity Penalty, and Exp.10 (yellow) including both penalty components. The evaluation was performed across increasing fire-spread complexity ($cc = 0.10$ to $cc = 0.26$).	62
6.3	Suppression Reward vs. Connected Suppression Reward. Performance of Exp.7 (Suppression Reward, no penalties), Exp.10 (Suppression Reward, with penalties), Exp.11 (Connected Suppression Reward, no penalties), and Exp.12 (Connected Suppression Reward, with penalties). The evaluation was performed across increasing fire-spread complexity ($cc = 0.10$ to $cc = 0.28$)	64

6.4	Behavioral Difference Between Suppression Reward Variants. Left: Exp.10 (Suppression Reward) leaves isolated burning cells unextinguished. Right: Exp.12 (Connected Suppression Reward) establishes a continuous suppression line.	65
6.5	Evaluation of Observation Components. Performance of Exp.13 (Last-Suppression), Exp.14 (Last-Suppression, Cardinal-Directions), Exp.15 (Last-Suppression, Ignition-Center), and Exp.16 (all three components), evaluated across increasing fire-spread complexity ($cc = 0.10$ to $cc = 0.30$).	68
6.6	Evaluation of Connected-Suppression Reward Variants. Performance comparison of the three Connected-Suppression reward calculation methods described in Section 5.1.3: the Region Size Method , Graph Diameter Method and the Wedge Point Distance Method . The evaluation was performed across increasing fire-spread complexity ($cc = 0.10$ to $cc = 0.32$).	71
6.7	Evaluation of the Optimization Experiments. Performance comparison of the two optimization experiments Exp.19 and Exp.20 compared with the best performing policy without any optimization (Exp.17). The evaluation was performed across increasing fire-spread complexity ($cc = 0.10$ to $cc = 0.34$).	73
6.8	RL-Heuristic Comparison. Performance comparison of the best performing rl policy (Exp.20) with the heuristic tactics. The evaluation was performed across increasing fire-spread complexity ($cc = 0.10$ to $cc = 0.36$).	76
7.1	Frequency distribution of suppressed cells per action for the RL agent and the Heuristic approach.	85
7.2	Snapshot of the emergent suppression tactic. The active fire front (red) is concentrated on the eastern perimeter, where limited initial suppression has slowed the eastward spread. The suppression line (white) is being built along the northern side while agents return to the water source (left) for resupply.	86

List of Tables

2.1	Heuristic tactics of the SoSID Toolkit	18
6.1	General training and scenario configuration parameters.	52
6.2	Curriculum learning configuration parameters.	53
6.3	PPO hyperparameters used during training.	54
6.4	Phase 1: Baseline Identification	58
6.5	Phase 2: Reward Function Ablation	60
6.6	Phase 3: Observation Space Extension and Optimization	66
A.1	Tools and Resources Used	99

1 Introduction

1.1 Motivation

Wildfire activity in Europe has intensified markedly in recent years. According to the European Forest Fire Information System [EFFIS, 2025], more than 1,025,224 hectares burned across the EU in 2025 alone, nearly three times the area recorded during the same period in 2024, and well above the 19-year average of 323,135 hectares (2006–2024). Beyond the direct ecological damage, forest fires are a significant source of greenhouse gas emissions, releasing 41.83 Mt CO₂ in 2025 alone. These trends highlight the urgency of developing more effective and scalable wildfire suppression strategies.

Reinforcement learning (RL) has demonstrated strong performance across complex sequential decision-making tasks, ranging from robotics to strategic planning [Shakya et al., 2023]. Applied to wildfire management, Multi-Agent Reinforcement Learning (MARL) offers a principled framework for coordinating fleets of autonomous aerial vehicles to suppress fires collaboratively [Zhao et al., 2024, Blais and Akhloufi, 2024]. A key challenge, however, is the black-box nature of learned policies, as the internal decision logic is difficult to inspect. This limitation is particularly problematic when the goal extends beyond performance to the discovery of novel suppression tactics. Explainable Reinforcement Learning (XRL) addresses this by making learned behavior more transparent. Reward decomposition, which breaks the reward signal into semantically distinct components, is one established XRL technique that allows the contribution of each incentive to be analyzed separately [Juozapaitis et al., 2019].

1.2 Contributions and Relevance

This thesis establishes a foundation for the integration of Reinforcement Learning (RL) into a complex agent-based wildfire simulation. It demonstrates a structured approach

for embedding RL-based decision-making into an existing large-scale simulation environment, thereby enabling adaptive suppression strategies within a System-of-Systems context. By allowing agents to learn through interaction with the environment, the framework enables the emergence of potentially novel or non-intuitive strategies. Such emergent behaviors may provide insights that complement established wildfire-fighting practices and inform future operational concepts.

To support a stable and systematic integration process, progressive complexity levels were designed. These levels iteratively increase the environmental and operational complexity of the simulation, allowing controlled development and validation of the RL integration.

In addition, this work introduces the Dual Decomposition Framework (DDF), which modularizes reward functions and observation spaces into separable components. This structured decomposition facilitates systematic experimentation and enhances the interpretability of learned agent behavior by enabling controlled analysis of individual design elements.

Beyond the primary research objectives, the RL integration and subsequent behavioral analysis exposed two implementation errors. An inconsistency in the simulation’s cell state transition logic was identified and corrected. Additionally, a targeting deficiency in the heuristic implementation was uncovered through the emergent tactic analysis and likewise corrected, though a re-evaluation against the corrected heuristic was not conducted within the scope of this work. These findings illustrate that RL-based integration and behavioral analysis can serve as a validation mechanism that can be used to detect errors in complex simulation environments.

Part of this work, including the Dual Decomposition Framework and preliminary results, was presented at the EASN Conference [Bardtke et al., 2026].

Overall, this thesis addresses the following research questions:

- How can the complexity of Reinforcement Learning integration into an agent-based wildfire simulation be systematically structured to enable effective learning of suppression strategies?
- How does modular decomposition of reward and observation space components affect the behavior and interpretability of learned suppression policies?
- Can multi-agent RL learn suppression strategies that match or exceed the performance of established heuristics in an operationally grounded wildfire simulation?

1.3 Project Context

This thesis was conducted as a collaborative effort between the Hamburg University of Applied Sciences (HAW) and the German Aerospace Center (DLR). The work is embedded within the broader research activities of the COLOSSUS project, which investigates methodologies for System-of-Systems Engineering (SoSE).

The wildfire simulation environment used in this thesis was developed within the COLOSSUS project as part of an Agent-Based Simulation (ABS) framework operating in a System-of-Systems (SoS) context. In this setting, individual constituent systems, such as aerial firefighting agents, are modelled as autonomous agents interacting within a shared simulation environment. The overall SoS behaviour emerges from the coordinated operation of these interacting agents.

This thesis extends the existing simulation framework by integrating Reinforcement Learning (RL) into the ABS environment. The objective is to introduce autonomous decision-making capabilities, enabling individual agents to autonomously select actions in response to the evolving wildfire scenario. Through this integration, the work contributes to expanding the analytical and exploratory capabilities of the COLOSSUS simulation framework, particularly with respect to investigating novel coordination strategies and emergent behaviours within the wildfire-fighting use case.

1.4 Thesis Structure

The remainder of this thesis is structured as follows. Chapter 2 covers the theoretical background, including the relevant concepts from reinforcement learning, deep RL, and multi-agent RL, as well as the System of Systems Inverse Design (SoSID) toolkit, which includes the ABS environment on which the implementation of this thesis is based. Chapter 3 reviews related work on autonomous aerial wildfire management, multi-agent coordination strategies, and explainable RL, and identifies the research gap addressed by this thesis. Chapter 4 defines the action space, observation space, and reward function, and introduces the Dual Decomposition Framework. Chapter 5 describes the technical integration of RL into the SoSID Toolkit, covering the tactic implementation, action mapping, and training-specific adaptations such as refueling abstraction and agent position randomization. Chapter 6 presents the experimental setup, training configuration,

and ablation study results, including quantitative comparisons against heuristic baselines. Chapter 7 interprets these results, discusses the contribution of individual design choices, and analyzes the emergent behavior of the learned policy. Chapter 8 summarizes the findings and outlines directions for future work.

2 Background

This section provides the theoretical foundation for the reinforcement learning methods employed in this thesis. Section 2.1 introduces the core concepts of reinforcement learning, including the Markov decision process formalism and value-based and policy-based solution methods. Section 2.1.4 discusses the extension to deep reinforcement learning through function approximation. Section 2.1.5 presents multi-agent reinforcement learning, describing the interaction paradigms and the challenges in a multi-agent setting. Section 2.1.6 details Proximal Policy Optimization, the algorithm used for training in this work. Section 2.1.7 introduces curriculum learning as a strategy for improving policy generalization. Section 2.2 focuses on the simulation platform that provides the environment in which the proposed MARL approach is deployed and evaluated.

2.1 Reinforcement Learning

Reinforcement Learning (RL) constitutes, alongside supervised learning and unsupervised learning, one of the three principal paradigms in machine learning. In supervised learning, a labeled dataset is available in which each input is associated with a corresponding output, enabling the model to learn a direct mapping between inputs and targets. In contrast, unsupervised learning operates without labeled outputs and aims to identify underlying patterns or structures within the data. [Sutton and Barto, 2018]

Reinforcement Learning differs fundamentally from both paradigms. It is inspired by natural learning processes observed in humans and animals, where behavior is shaped through trial-and-error interaction with the environment and evaluative feedback. In RL, an agent performs actions within an environment and receives feedback in the form of a reward signal. This reward evaluates the quality of the executed action or sequence of actions. The objective of the agent is to learn a strategy that maximizes cumulative reward over time. Formally, the agent interacts with an environment by observing its

current state and selecting an action based on that observation. The environment then transitions to a new state and provides a reward signal, which may be immediate or delayed over multiple time steps. This interaction pattern is shown in Figure 2.1. While the reward function evaluates the immediate quality of an action, the value function estimates the expected long-term return associated with a state or action in pursuit of the overall objective. Consequently, a state may yield only a small immediate reward yet possess a high value if it is likely to lead to states associated with substantially higher future rewards. [Sutton and Barto, 2018]

The learned value estimates are used to derive or improve a policy. A policy defines the agent's behavior by mapping observed environmental states to actions. It represents the core component of a reinforcement learning agent, as it ultimately determines how decisions are made within the environment.

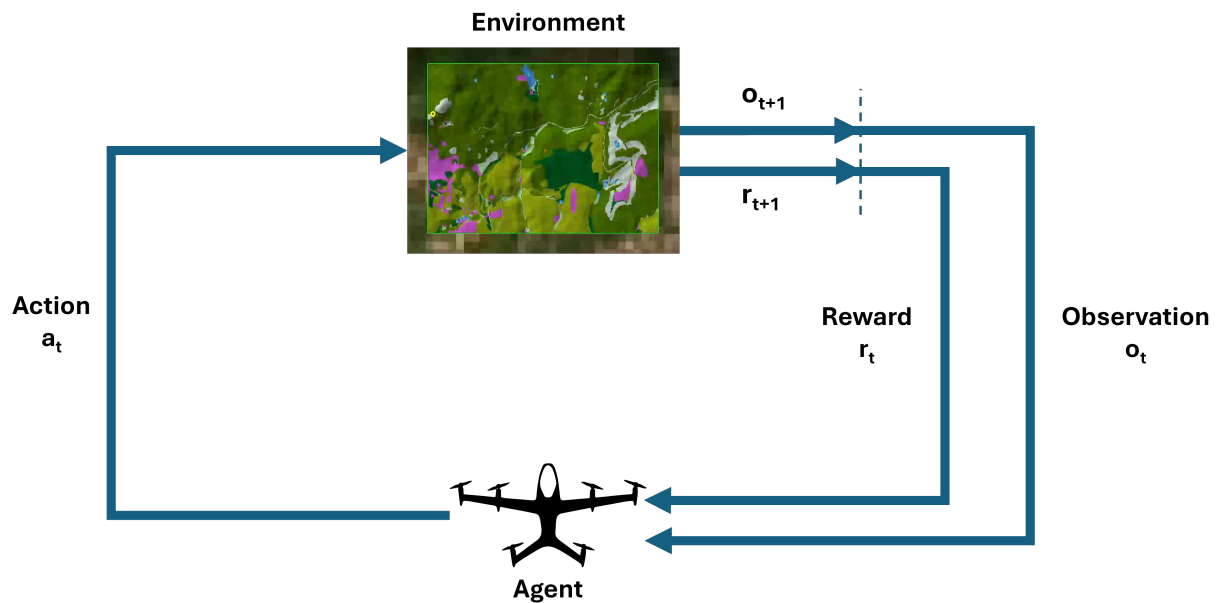


Figure 2.1: The agent–environment interaction in reinforcement learning

2.1.1 Markov Decision Process

The standard mathematical framework for reinforcement learning is the Markov Decision Process (MDP), defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$, where:

- $\mathcal{S}$ is the state space, representing all possible configurations of the environment
- $\mathcal{A}$ is the action space, representing all decisions available to the agent
- $\mathcal{P} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$ is the state transition probability function, where $\mathcal{P}(s'|s, a)$ denotes the probability of transitioning to state s' after taking action a in state s
- $\mathcal{R} : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is the reward function, providing immediate feedback for state-action pairs
- $\gamma \in [0, 1]$ is the discount factor, determining the relative importance of immediate versus future rewards

The Markov property requires that the transition probability depends only on the current state and action, not on the history of prior states:

$$P(s_{t+1}|s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0, a_0) = P(s_{t+1}|s_t, a_t) \quad (2.1)$$

This memoryless assumption simplifies the learning problem by ensuring that the current state contains all information necessary for optimal decision-making. [Sutton and Barto, 2018]

2.1.2 Policy and Value Functions

A policy $\pi : \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$ defines the agent's behaviour by specifying a probability distribution over actions for each state. The goal is to find π^* maximizing the expected cumulative discounted reward:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \quad (2.2)$$

Policy quality is measured by the state-value function $V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s]$ and the action-value function $Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a]$, both satisfying recursive Bellman equations.

2.1.3 Solution Methods

RL algorithms fall into three families. *Value-based methods* learn an approximation of Q^* and derive a policy via greedy action selection; Q-learning [Watkins and Dayan, 1992] is the canonical example, updating estimates via temporal-difference learning. *Policy-based methods* directly parameterize and optimize the policy via gradient ascent on the expected return, using the policy gradient theorem [Sutton et al., 1999]:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a | s) Q^{\pi_{\theta}}(s, a)] \quad (2.3)$$

This enables application to continuous action spaces and stochastic policies. *Actor-critic methods* combine both approaches: a critic estimates the value function to reduce gradient variance, while an actor updates the policy using the advantage function $A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$, balancing the stability of value-based methods with the flexibility of policy-based ones. [Sutton and Barto, 2018]

2.1.4 Deep Reinforcement Learning

Classical reinforcement learning algorithms rely on tabular representations, in which value functions or policies are stored explicitly for every state or state-action pair. While this approach is feasible for small, discrete environments, it scales poorly as the size of the state space increases. In high-dimensional or continuous domains, the number of possible states becomes prohibitively large, making explicit tabular storage and iterative updates computationally impractical. Deep Reinforcement Learning addresses this by replacing tabular representations with neural networks, which approximate value functions $Q(s, a; w) \approx Q^{\pi}(s, a)$ and policies $\pi(a | s; \theta)$ using learned parameters rather than explicit lookup tables. Crucially, neural networks generalize across similar states through their inductive bias, allowing learned representations to transfer to previously unseen configurations. This property is particularly important in spatially extended environments such as the wildfire simulation considered in this work, where the agent must respond to a continuous and highly variable range of fire configurations that cannot be exhaustively represented during training.

2.1.5 Multi-Agent Reinforcement Learning

Multi-Agent Reinforcement Learning (MARL) extends the single-agent framework to settings in which multiple agents interact within a shared environment. Each agent observes the environment, selects actions, and receives rewards, with the joint behaviour of all agents determining state transitions and individual outcomes.

Interaction Paradigms

MARL problems are commonly classified according to the structure of agent objectives:

- **Cooperative:** All agents share a common reward function and optimize a joint objective. Agents must coordinate to achieve the best collective outcome.
- **Competitive:** Agents pursue opposing objectives and maximize individual rewards, typically at the expense of others.
- **Mixed:** Agents have individual reward functions that are neither fully aligned nor fully opposed, requiring them to balance cooperation and competition.

This thesis addresses the cooperative setting, in which wildfire suppression agents work toward the shared goal of minimizing burned area.

Challenges in Multi-Agent Reinforcement Learning

MARL introduces several challenges beyond those encountered in single-agent RL [Liang et al., 2025]:

- **Non-stationarity:** From the perspective of a single agent, the environment is non-stationary because other agents are simultaneously learning and changing their policies. This violates the Markov property and complicates convergence analysis.
- **Scalability:** The joint action space grows exponentially with the number of agents, which may cause problems such as performance degradation or computational resource constraints.

-
- **Credit assignment:** When agents receive a shared team reward, it is difficult to attribute success or failure to individual actions. Determining which agent’s decision contributed to the outcome is essential for effective learning.
 - **Partial observability:** Agents often observe only a local view of the environment, lacking access to the global state or the actions of other agents. This transforms the problem into a Partially Observable Markov Decision Process (POMDP) for each agent.

2.1.6 Proximal Policy Optimization

Proximal Policy Optimization (PPO) [Schulman et al., 2017] is an on-policy actor-critic algorithm that extends the policy gradient approach. Standard policy gradient methods become unstable when multiple gradient updates are performed on the same data batch, as unconstrained updates can move the policy too far from the one used to collect the experience. PPO addresses this by introducing a clipped surrogate objective that limits the size of each policy update.

At the core of PPO is the probability ratio $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$, which measures how much the updated policy deviates from the behavior policy that collected the data. The clipped surrogate objective is then defined as

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad (2.4)$$

where $\hat{A}_t$ is an estimate of the advantage function and ϵ is a small hyperparameter, typically set to 0.2. By taking the minimum of the clipped and unclipped terms, the objective prevents the policy from benefiting from large deviations while still providing a useful gradient signal when the ratio remains within the trust region. Advantage estimates are commonly computed using Generalized Advantage Estimation (GAE) [Schulman et al., 2015], which introduces a trace-decay parameter λ to balance bias and variance. The full objective additionally includes a value function loss and an entropy bonus to encourage exploration.

Despite being an on-policy algorithm, PPO has been shown to achieve competitive or superior performance compared to off-policy methods in cooperative MARL settings when used with a shared-policy architecture [Yu et al., 2022], which is the primary reason for its selection in this work.

2.1.7 Curriculum Learning

Curriculum learning dates back to [Elman, 1993] and is inspired by the way humans learn, beginning with simple tasks and progressively moving to more complex ones, allowing algorithms to steadily improve their overall performance [Li et al., 2025].

In the context of RL, a curriculum can be defined over task complexity, initial state distributions, or environmental parameters. For instance, agents may first be trained in simplified environments and progressively exposed to more challenging scenarios as their competence improves. Curriculum learning is particularly valuable in domains where the optimal policy is difficult to discover from scratch. By providing intermediate training stages, curricula enable agents to acquire foundational skills before tackling the full problem. This mirrors human learning, in which complex skills are often developed incrementally.

For the wildfire suppression problem addressed in this thesis, curriculum learning can be applied by gradually increasing the fire spread rate during training, allowing agents to first learn suppression strategies in slow-spreading fires before generalizing to more aggressive spread dynamics.

2.2 SoSID Toolkit

The SoSID toolkit [Kilkis et al., 2021, Prakasha et al., 2025] is an agent-based Python simulation environment developed by the German Aerospace Center (DLR) to support System-of-Systems (SoS) analysis in domains such as aerial wildfire suppression and urban air transportation. Within the COLOSSUS project, SoSID provides operational analysis capabilities for evaluating aircraft design parameters and their effects on overall SoS behavior. In particular, it enables the investigation of emergent phenomena arising from the interaction of multiple constituent systems.

In the wildfire suppression use case (EVE), the Agent-Based Modeling (ABM) of the SoSID toolkit is used to represent each aircraft as an autonomous agent equipped with predefined rules, tasks, and decision logic. The interactions among these agents collectively determine system-level outcomes, enabling assessment of how local tactical decisions influence global wildfire dynamics. The wildfire environment is modeled using a grid-based cellular automaton in which cells transition through discrete states, including

suppressed, nonflammable, and progressive burning stages. Terrain reconstruction enables geographically grounded and realistic wildfire simulations, including the recreation of historical wildfire events for evaluation purposes.

ABM simplifies the modeling and adaptation of heterogeneous constituent systems by encapsulating behavior within individual agents. Tactical logic such as direct attack (dropping suppressant directly onto burning cells) and indirect attack (establishing a containment perimeter) can be implemented and varied independently. This modularity allows systematic exploration of how variations in agent behavior, design characteristics, or operational parameters affect wildfire propagation and overall suppression performance. Fire propagation and suppression dynamics are confined to a designated *active area*, which defines the operational boundary of the simulation domain. The active area is illustrated in Figure 2.2 as the green rectangle. Infrastructure elements, including airports, may reside outside this region. Each episode initializes agents at assigned airport positions. Following takeoff, agents transit to the active area to commence suppression operations. Agents may temporarily exit the active area to perform refueling maneuvers, which are restricted to airport locations. Episode termination occurs under two conditions: successful suppression before the fire reaches the active area boundary (recorded as success), or fire escape beyond the boundary prior to full suppression (recorded as failure).

Agent navigation follows a goal-based control architecture in which each agent is assigned a specific destination, such as a selected fire front cell, a water refill location, or a vertiport waypoint. Rather than directly controlling low-level motion commands, higher-level decision modules determine these target locations based on the selected tactic. Once a goal has been defined, the agent executes a predefined path planning logic to reach the assigned goal.

2.2.1 Scenario

The simulation environment is based on a wildfire scenario representing the Sintra region in Portugal. The scenario is implemented as a grid-based environment, where each cell is assigned one of several terrain and fire-state types. Fire propagation is additionally influenced by meteorological factors including wind direction, wind speed, air humidity, and temperature, as well as the correction coefficient described in Section 2.2.4.

Figure 2.2 shows an overview of the Sintra region, including the vertiports from which the suppression aircraft are deployed. Figure 2.3 zooms into the active area, which defines the operational boundary within which fire propagation and fire suppression take place.

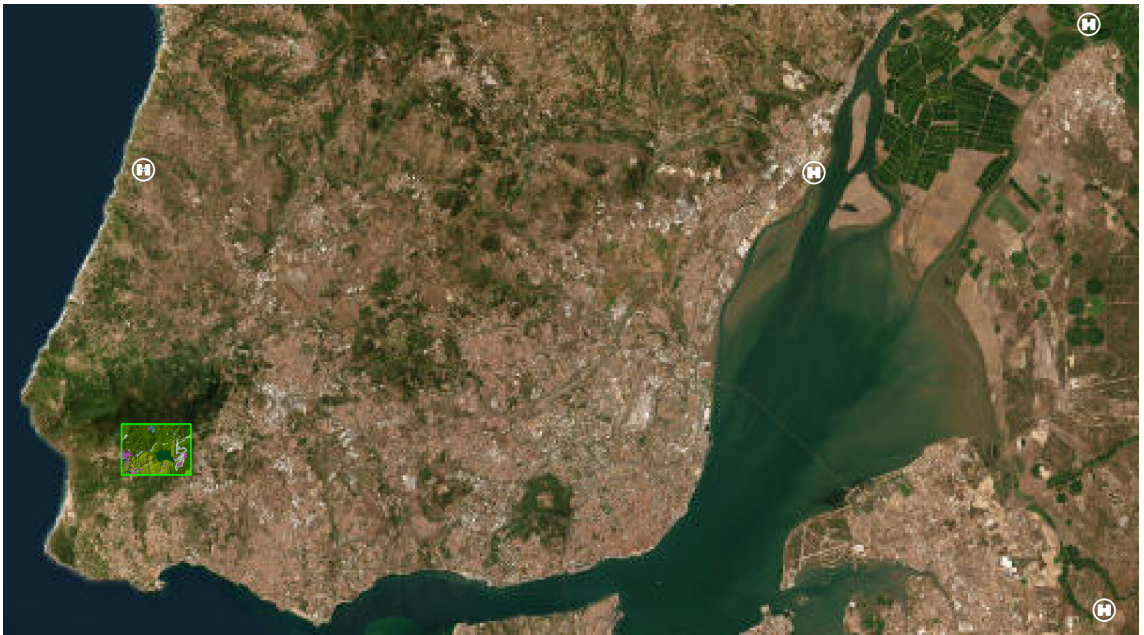


Figure 2.2: **Overview of the Sintra Scenario.** The figure shows the broader Sintra region with the vertiports marked, from which suppression aircraft are deployed at the start of each mission and to which they return for refueling.



Figure 2.3: **Active Area of the Sintra Scenario.** The figure shows the active area in detail, within which fire propagation and suppression operations take place. The *Active Area* boundary delimits the operational region. *Vegetation* cells are flammable and represent the primary fuel source. *Residential Areas* and *Nonflammable* cells do not propagate fire. *Water Sources* mark locations where agents can resupply their water tanks. The *Ignition Center* indicates the point of fire origin. During a simulation, cells transition from vegetation to *Burning* and subsequently to *Burnt*, reflecting the progression of the wildfire.

2.2.2 Wildfire Model

The wildfire model underlying the SoSID Toolkit is based on a Cellular Automata (CA) approach adopted from [Rui et al., 2018], in which the landscape is represented as a grid of discrete cells. Each cell can occupy one of six states: non-flammable, combustible,

early burning, full burning, extinguishing, or burnt. At every simulation timestep, burning cells evaluate their Moore neighborhood and attempt to ignite adjacent combustible cells according to a locally computed spread rate R . Crucially, the transition probability between states is not fixed but is governed by a rich set of environmental and topographic inputs, making the fire behavior highly context-dependent. The spread rate at any given cell is determined as a function of vegetation combustibility, wind speed, wind direction, terrain slope, elevation, relative humidity, and ambient temperature [Cigal et al., 2026]. As demonstrated in the environmental sensitivity study of Cigal et al. [2026], these parameters interact non-linearly. Wind speed is among the most influential factors: beyond a threshold of approximately 6 m/s the burnt area grows exponentially, as high wind both accelerates direct fire propagation and drives the fire toward mountainous terrain where slope effects compound the spread rate further. Terrain slope itself acts as an amplifier, where steeper gradients accelerate fire spread, with a threshold elevation effect becoming significant above roughly 175 m in the studied scenario. Wind direction interacts with topography indirectly: if aligned toward high-slope terrain, its effect on spread rate is amplified considerably, whereas head-on winds relative to the fire front slow the advance and favor direct suppression tactics. Ambient temperature is the most continuously sensitive parameter, with fire spread increasing substantially at temperatures above 25 °C and accelerating further beyond 30 °C. Relative humidity exerts an inverse influence: drier conditions reduce fuel moisture and intensify fire growth, though its overall impact is smoother and somewhat less pronounced than that of temperature or wind. A key model parameter that mediates the overall aggressiveness of fire spread is the correction coefficient (cc), introduced to calibrate the CA model to realistic fire behavior (see Section 2.2.4). Higher values of cc result in markedly larger burnt areas and faster spread dynamics, making it the primary lever for defining scenario difficulty in experiments.

2.2.3 Suppressant Drop Model

The effectiveness of an aerial suppression drop is determined by the suppressant drop model introduced in Prakasha et al. [2021]. Each drop is modeled as an elliptical suppression patch, whose dimensions are derived from two aircraft-specific parameters: the suppressant payload capacity and the flow rate. The total suppression area scales linearly with payload volume, while the patch width follows a power-law relationship with the suppressant flow rate, both relationships fitted to empirical water drop test data from 20 different aircraft types [Prakasha et al., 2021]. Since the CA fire model operates on a

discrete raster grid, a probabilistic rounding mechanism is applied at the grid boundary of the patch: cells only partially covered by the ellipse are suppressed with a probability proportional to their coverage fraction, ensuring that marginal increases in payload translate proportionally into suppression effectiveness over many drops. This means that the suppression capability of each agent in the simulation is therefore directly tied to the physical aircraft design parameters, coupling aircraft-level performance to SoS-level mission outcomes.

2.2.4 Correction Coefficient

In the wildfire model, the correction-coefficient (cc) regulates the sensitivity of the fire to its surroundings. Lower values result in slower fire propagation, whereas higher values produce more rapidly expanding wildfire behavior. As such, the correction coefficient serves as the primary parameter for modulating scenario difficulty across training and evaluation conditions.

Figure 2.4 illustrates fire propagation at three cc values after 10,000 simulation timesteps. At $cc = 0.1$ the burned area reaches 56,600,m², rising to 166,800,m² at $cc = 0.2$ and 381,300,m² at $cc = 0.3$, demonstrating the sensitivity of fire spread to the correction coefficient.

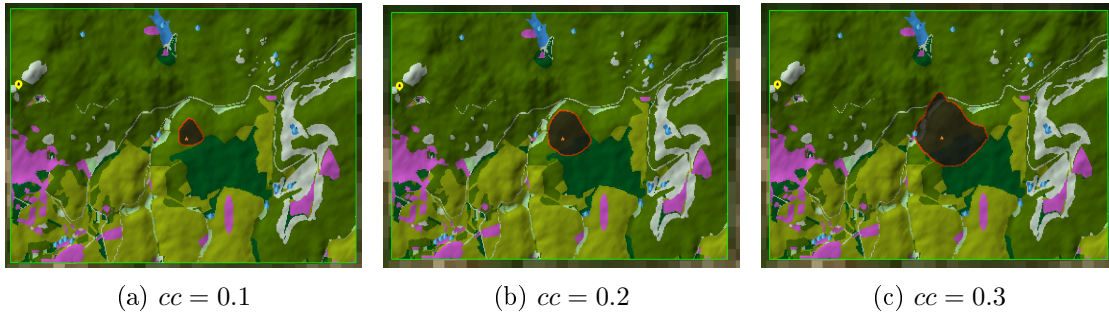


Figure 2.4: **Effect of the Correction Coefficient on Fire Spread.** Burned area after 10,000 simulation timesteps for three values of the correction coefficient. Higher cc values result in significantly larger burned areas, illustrating the sensitivity of fire propagation to this parameter.

2.2.5 Baseline Simulation and Heuristic Benchmarks

The SoSID Toolkit provides five heuristic suppression tactics that serve as baselines for evaluating the performance of trained RL policies. These heuristics represent deterministic, rule-based strategies whose logic is transparent and easily interpretable and are based on real-life suppression tactics.

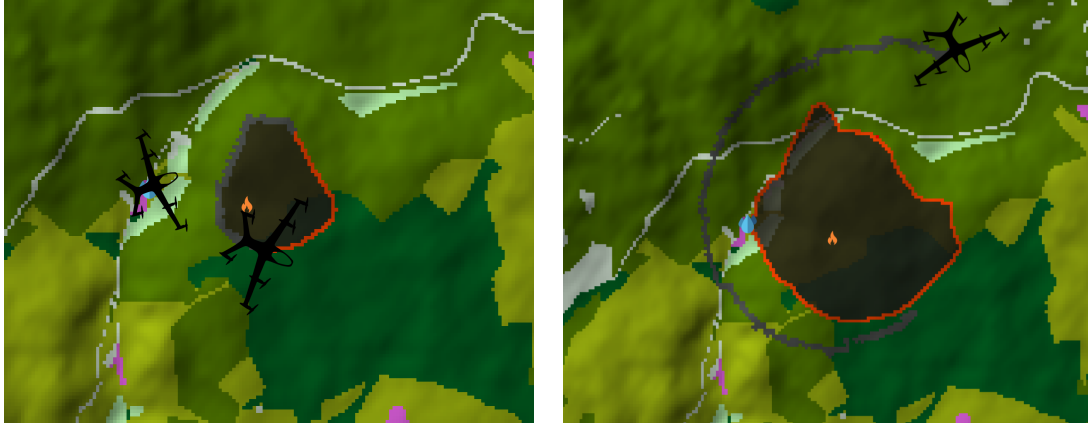
Four of the heuristics implement a direct attack strategy (VIP, Water, Vegetation, and Topography), actively selecting and suppressing segments of the fire front. The fifth heuristic implements an indirect attack strategy, aiming to contain the wildfire by establishing a circular suppression perimeter. Both approaches are illustrated in Figure 2.5.

Each heuristic is organized into three successive stages. First, *select_poi* determines the Point of Interest (POI), i.e. the region of the fire front targeted for suppression. The POI type varies by tactic: for example, the vegetation heuristic identifies the burning segment based on the combustibility of the vegetation, while the Indirect heuristic derives its POI from a computed containment perimeter. Second, *track_poi* maintains focus on the precise fire-front segment associated with the selected POI, updating continuously as the fire spreads or suppression efforts take effect. Third, *suppress* translates the tracked segment into a cell-level action, determining which individual grid cell is suppressed at each timestep.

Direct heuristics select a target point on the fire front according to a weighted scoring function and continuously pursue this target as the fire evolves until suppression is achieved. The scoring function of each tactic is described in Table 2.1.

Table 2.1: Heuristic tactics of the SoSID Toolkit

Tactic Name	Functionality
Direct (water)	Selects fire front cells based on a weighted scoring function combining distance to the aircraft, proximity to user-defined very-important (VIP) points, and proximity to urban or residential areas.
Direct (vip)	Selects fire front cells using a weighted combination of distance to the aircraft and proximity to user-defined VIP points.
Direct (vegetation)	Targets fire front cells based on distance to the aircraft and vegetation-dependent combustibility, prioritizing highly flammable areas.
Direct (topography)	Targets fire front cells based on distance to the aircraft and terrain slope, prioritizing regions with elevated spread potential.
Indirect	Constructs a circular containment line around the wildfire by suppressing adjacent cells, aiming to restrict outward fire spread before switching to direct suppression.



(a) Direct attack: agents engage actively burning cells at the fire front. (b) Indirect attack: agents form a containment circle around the wildfire.

Figure 2.5: Visualization of the direct and indirect heuristic suppression tactics in the simulation.

2.2.6 Aircraft types

In the simulation, two types of suppression aircraft are defined: electric vertical take-off and landing (eVTOL) aircraft and seaplanes. The two types differ in their flight characteristics and resupply mechanics, which has direct implications for the design of the action space and operational constraints.

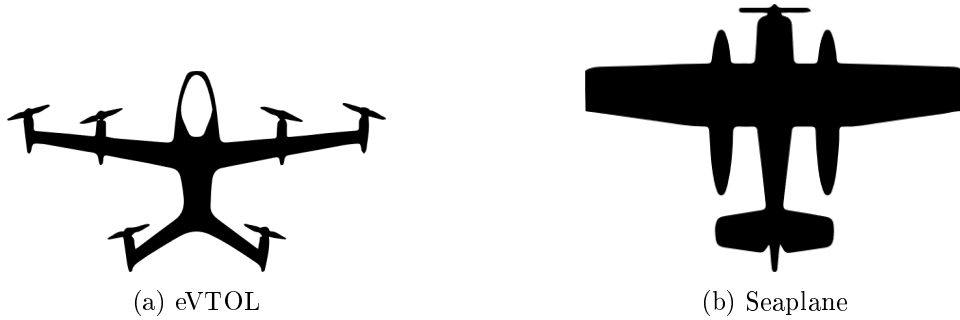


Figure 2.6: **Aircraft types used in the simulation.** eVTOL (left) and seaplane (right).

eVTOLs can take off and land vertically and hover in place, allowing them to maneuver independently of forward velocity. This makes direction-based action spaces a natural

fit, as the aircraft can move in any direction at any time without needing to maintain airspeed. Resupply is performed at fixed ground-based water depots.

Seaplanes, by contrast, require a continuous water surface for takeoff and landing, which they use to refill their water tanks during low-altitude scooping maneuvers. This requires that open water is available in the operational area and necessitates orientation-based control, as the aircraft must maintain a forward movement for maneuvering.

In the Sintra scenario used in this work, no sufficiently large open water surfaces are available in the operational area to support seaplane resupply operations. For this reason, only eVTOLs are used as suppression agents throughout this thesis.

3 State of the Art

This chapter reviews the literature relevant to the three principal research domains addressed in this thesis: autonomous aerial wildfire management using UAVs, multi-agent coordination strategies in reinforcement learning and explainable RL. The chapter concludes by identifying the research gap that motivates the contributions of this work.

3.1 Aerial Wildfire Monitoring and Suppression

The integration of unmanned aerial vehicles into wildfire management has attracted sustained research attention, motivated by the operational limitations of conventional crewed aircraft and ground-based sensor networks. Boroujeni et al. [2024] provide a systematic survey of AI-enabled UAV systems across the full wildfire management lifecycle, covering pre-fire fuel monitoring, active suppression, and post-fire damage assessment. The survey underscores that while deep learning has matured substantially for fire detection and segmentation tasks, the application of reinforcement learning to active suppression remains considerably less developed, with most existing work focusing on monitoring rather than intervention. Early work on autonomous UAV-based suppression was established by Haksar and Schwager [2018], who modelled forest fire fighting as a factored Markov decision process and derived a distributed deep RL policy enabling a network of UAVs to contain fire spread. Operating under local information constraints and communication limits, their approach demonstrated that the learned policy outperforms a hand-tuned heuristic and scales gracefully to varying forest sizes and agent counts. Experiments were validated both in Monte Carlo simulation and on physical multi-robot testbeds, providing an early proof-of-concept for RL-driven aerial firefighting. The environment is an abstract lattice-based grid with uniform fire spread parameters; the authors explicitly identify wind and terrain effects as limitations of their model and directions for future work. Tanha et al. [2023] propose a MARL framework using an actor-critic architecture in which learning is centralized while execution is decentralized. Their environment is

modelled as a uniform grid in which all cells contain equal fuel and fire spread follows parameterized discrete-time Markov models without terrain or wind effects. The system defines two such models of increasing fidelity and reports improved suppression performance relative to prior deep Q-network baselines. Blais and Akhloufi [2024] extend the swarm robotics perspective, proposing a system combining an overwatch UAV with a fleet of payload-carrying suppression drones. The overwatch agent provides situational awareness that informs decentralized drop decisions, and preliminary results indicate that this hierarchical structure improves coverage efficiency relative to flat swarm coordination. The scheduling dimension of multi-UAV suppression has been addressed by Chen et al. [2024a], who introduce a proactive scheduling system incorporating spatio-temporal confidence-aware assessment to direct UAVs towards locations of high operational value. Validated on both real testbeds and large-scale simulations, their system reduces fire expansion ratio by 59 % and improves coverage ratio by 190 % relative to prior scheduling methods. While highly effective, such scheduling-centric approaches assume relatively static role assignments and do not explore the emergence of suppression tactics through end-to-end learning. On the monitoring side, Julian and Kochenderfer [2019] developed deep RL methods for fixed-wing aircraft to maximise sensor coverage of an active fire, demonstrating the applicability of RL to the broader wildfire observation problem. UAV-based detection systems using computer vision, such as those employing YOLOv8 variants for real-time smoke and flame detection, have reached a high level of maturity and now complement suppression research by providing accurate fire-state observations that can serve as inputs to RL-based suppression agents [Zhao et al., 2024].

3.2 Multi-Agent Coordination Strategies

Cooperative multi-agent reinforcement learning has converged on Centralized Training with Decentralized Execution (CTDE) as its dominant paradigm [Amato, 2024]. This framework allows agents to exploit global state information during training through a centralized critic or value function, while restricting execution to locally observable information. This separation is critical in practical deployment scenarios where agents cannot communicate arbitrary state information in real time. CTDE addresses the non-stationarity inherent in multi-agent settings by conditioning training on the joint state, while the decentralized execution constraint ensures scalability and robustness under communication-limited operation. Within the CTDE framework, two families of approaches are predominant. Value decomposition methods, including VDN [Sunehag

et al., 2018] and QMIX [Rashid et al., 2020], factorise the joint action-value function into per-agent utilities that satisfy the Individual-Global-Max (IGM) condition, ensuring that decentralised greedy action selection remains globally optimal. Policy gradient methods extend the actor-critic paradigm to the multi-agent setting, with Multi-Agent PPO (MAPPO) emerging as a particularly effective instantiation. Yu et al. [2022] demonstrate that a shared-policy MAPPO implementation achieves competitive or superior performance relative to value decomposition methods across a range of cooperative tasks, an observation that directly motivates the algorithmic choice in the present work. Importantly, their shared-policy variant does not employ a centralized critic, and coordination emerges from parameter sharing alone. Recent research has identified limitations in the standard CTDE formulation. Zhou et al. [2023] argue that conventional CTDE methods do not fully exploit global information during training because agent policies still operate on independent local observations during the training forward pass, restricting the expressiveness of the learned joint policy. Their Centralized Advising and Decentralized Pruning (CADP) framework introduces explicit inter-agent message exchange during training, followed by smooth communication pruning to recover decentralised execution policies. This line of work underscores a broader tension between the richness of centralized training information and the practical constraints of decentralised deployment.

In the wildfire domain specifically, the choice between centralized and decentralized control has direct operational implications. A centralized controller observing the full simulation state can in principle compute globally optimal suppression assignments, but this approach does not scale to large agent counts and requires full state observability at execution time. Decentralized approaches, where each agent acts on its local observation, are more realistic but must overcome coordination failures arising from partial observability and redundant coverage. The shared-policy architecture adopted in this thesis occupies a pragmatic middle ground: a single policy network is trained across all agent experiences, preserving scalability and sample efficiency while enabling behavioural consistency across the fleet. Curriculum learning has been applied to multi-agent settings to address the difficulty of discovering effective joint policies from scratch in complex environments. Li et al. [2025] demonstrate that progressive task complexity scheduling accelerates convergence and improves policy robustness in scalable MARL, consistent with earlier findings in the single-agent setting [Elman, 1993].

3.3 Explainability in Reinforcement Learning

The interpretability of learned policies is a central challenge for the deployment of RL in high-stakes domains. The field of Explainable Reinforcement Learning (XRL) has emerged to address this challenge through a range of techniques targeting different components of the RL pipeline. Qing et al. [2025] provide a survey of XRL approaches organised around four axes of explainability: agent model, reward, state, and task. Within the reward-explaining category, they distinguish between reward shaping methods, which modify the reward signal to guide learning, and reward decomposition methods, which retain the original reward structure but decompose it into semantically meaningful components for post-hoc analysis.

Juozapaitis et al. [2019] establish reward decomposition as a principled approach to explaining agent preferences. By decomposing the reward function into a sum of semantically meaningful sub-rewards, their method allows an agent’s action choices to be understood in terms of trade-offs between distinct objectives. They introduce the concept of *minimum sufficient explanations*, which identify the smallest subset of reward components that justify a given action preference. Their approach further demonstrates that decomposed action-value functions can be learned consistently using an off-policy variant of Q-learning, ensuring that the decomposed values remain accurate representations of the agent’s decision-making process.

Extensions of reward decomposition to the multi-agent setting have begun to appear in the recent literature. Septon et al. [2023] build on this foundation by pairing reward decomposition with global policy summaries, demonstrating that combining local action-level explanations with a global view of policy behaviour yields more complete interpretability and improves user understanding of learned RL systems. Chen et al. [2024b] propose a spatial-temporal return decomposition method for cooperative MARL that attributes credit along both the temporal and spatial axes involving multiple agents. This work generalises scalar decomposition to the multi-agent credit assignment problem, where distinguishing individual contributions from a shared team reward is particularly challenging.

Despite this progress, the joint application of reward decomposition and observation-space decomposition within a unified framework has not been systematically explored. Existing work typically treats these two dimensions independently, either decomposing

the reward while holding the observation space fixed, or performing ablations over observation components without a modular reward structure. This gap limits the ability to characterise how perceptual information and reward incentives interact to shape learned behaviour—which is particularly important in complex multi-agent environments where emergent tactics depend on the interplay between what agents observe and what they are incentivised to do.

A gap that the existing XRL literature has not addressed is the simultaneous decomposition of both the reward function *and* the observation space. The Dual Decomposition Framework introduced in this thesis addresses this gap by applying modular decomposition to both axes in parallel, enabling a more complete account of the factors that govern agent decision-making in the aerial suppression domain.

3.4 Research Gap

Prior RL-based aerial wildfire suppression research has been conducted in abstract grid environments that omit terrain effects and real geographic grounding. Haksar and Schwager [2018] explicitly acknowledge this limitation, noting that wind and terrain effects will need to be addressed for real-world applicability. Tanha et al. [2023] similarly operate on uniform grids with parameterized fire models that do not capture spatially varying terrain or fuel distributions. No existing work has evaluated RL suppression policies within a simulation that reconstructs actual terrain topology, includes operational constraints such as fuel capacity and fixed refueling infrastructure, and enables direct comparison with established rule-based tactics under identical conditions. Furthermore, while reward decomposition has been proposed as a tool for policy interpretability, it has not been applied to wildfire suppression, nor has it been combined with systematic observation space ablation in a unified experimental framework. Finally, existing work does not characterize the specific spatial suppression strategies that emerge from training: prior studies report aggregate performance metrics without characterizing the specific behavioral strategies that agents develop. This thesis addresses all three gaps within the SoSID simulation environment.

4 Design

4.1 Multi-Agent Control Strategies

Several paradigms exist for controlling a team of agents in a multi-agent reinforcement learning setting, each offering a different trade-off between coordination capability and scalability.

In **Centralized RL**, a single controller observes the full global state of the environment and selects actions for all agents simultaneously. This enables globally optimal coordination in principle, but the joint action space grows exponentially with the number of agents, making the approach computationally intractable at scale. It additionally requires full state observability at execution time, which is rarely available in practice.

In **Decentralized RL**, each agent maintains its own policy and acts solely on its local observation. This approach scales naturally and does not require global state access, but makes coordination harder to achieve as agents have no explicit mechanism to align their behaviors.

A practical middle ground is *parameter sharing*, in which agents are decentralized at execution time but share a single policy network during training. All agents contribute experience to a common training stream, and the shared weights reduce the effective parameter space and improve sample efficiency. Yu et al. [2022] demonstrate that PPO-based methods with parameter sharing, including the fully decentralized variant IPPO, achieve competitive performance with more complex off-policy multi-agent methods, while retaining the scalability and simplicity of decentralized execution. This thesis adopts the parameter-sharing paradigm; the policy architecture is described in detail in Section 4.2.

4.2 Policy Architecture

A shared-policy architecture is employed, in which all agents are controlled by a single policy network. This approach corresponds to the parameter-sharing variant of PPO described by Yu et al. [2022]: agents observe only local information and act independently both during training and execution, without a centralized critic or access to the global state. During training, agents interact with the environment in parallel and generate individual trajectories, which are aggregated to update the shared policy parameters. This design promotes consistent behavior across agents while reducing the complexity of the learning setup.

Using a single shared policy substantially decreases the number of trainable parameters, contributing to more stable learning dynamics and improved sample efficiency. Moreover, the shared-policy paradigm enhances scalability, as learned behaviors can be deployed across an arbitrary number of agents without requiring additional policy training.

4.3 Progressive Complexity Levels

The integration of RL into the SoSID Toolkit was structured across four progressively increasing complexity levels. This staged approach served as a proof of concept and facilitated a systematic and controlled incorporation of RL into the simulation environment. By dividing the integration into discrete steps, each level can be independently validated before the next is introduced, making it possible to verify that the expected behavior is achieved at each stage and to isolate the source of any issues to the most recently added component. Beyond managing complexity, the staged structure also aids in identifying the source of reward hacking: when unexpected agent behavior emerges after introducing a new component, the incremental design makes it possible to trace the cause to the most recently added element. The first three levels form the Simplified Design, in which resource constraints are removed, such that fuel availability and suppressant tank capacity are assumed to be unlimited. Level 4 represents the goal design in which all operational constraints are reinstated.

4.3.1 Complexity Level 1

The first level uses a single-agent environment in which the sole objective is to locate the active wildfire. The action space is restricted to the four cardinal directions (North, South, East, West). The observation space consists of a direction vector pointing toward the ignition center and a local grid encoding the fire states of surrounding cells, as described in Section 4.6. Refueling is not modeled at this level and the agent operates with unlimited fuel capacity. The reward function combines the agent’s change in distance to the nearest burning cell with the number of burning cells within its local neighborhood.

4.3.2 Complexity Level 2

The second level extends the first by adding suppression capability to the action space alongside an 8-directional movement set. The observation space is unchanged. The reward function is revised accordingly: instead of rewarding proximity to the fire, agents receive a reward proportional to the number of cells successfully suppressed. An additional mission success reward is granted when the wildfire is fully extinguished. Both fuel and suppressant remain unlimited at this level, and no resupply operations are required.

4.3.3 Complexity Level 3

The third level transitions from a single agent to a multi-agent setting, with four agents operating concurrently under a shared policy. The action and observation spaces are unchanged from Level 2. The reward function is modified by replacing the mission success reward with a Timestep Penalty. This change shifts the incentive from completion alone toward efficient suppression, since strategies that extinguish the fire in fewer steps accumulate a smaller penalty.

4.3.4 Complexity Level 4

Level 4 represents the goal design and reintroduces the resource constraints removed in earlier levels. Agents now operate with finite fuel and suppressant capacity. A distinction is made between refueling and replenishing extinguishing agents during the training phase and the deployment phase, which is described in more detail in Section 5.2. In

addition, the movement space is further expanded to 16 evenly distributed directions, enabling finer spatial control. With the goal design finalized, subsequent work can focus on hyperparameter tuning, reward shaping, and expanding the observation space. The ablation study described in Section 6.3 investigates the effect of individual reward and observation space components.

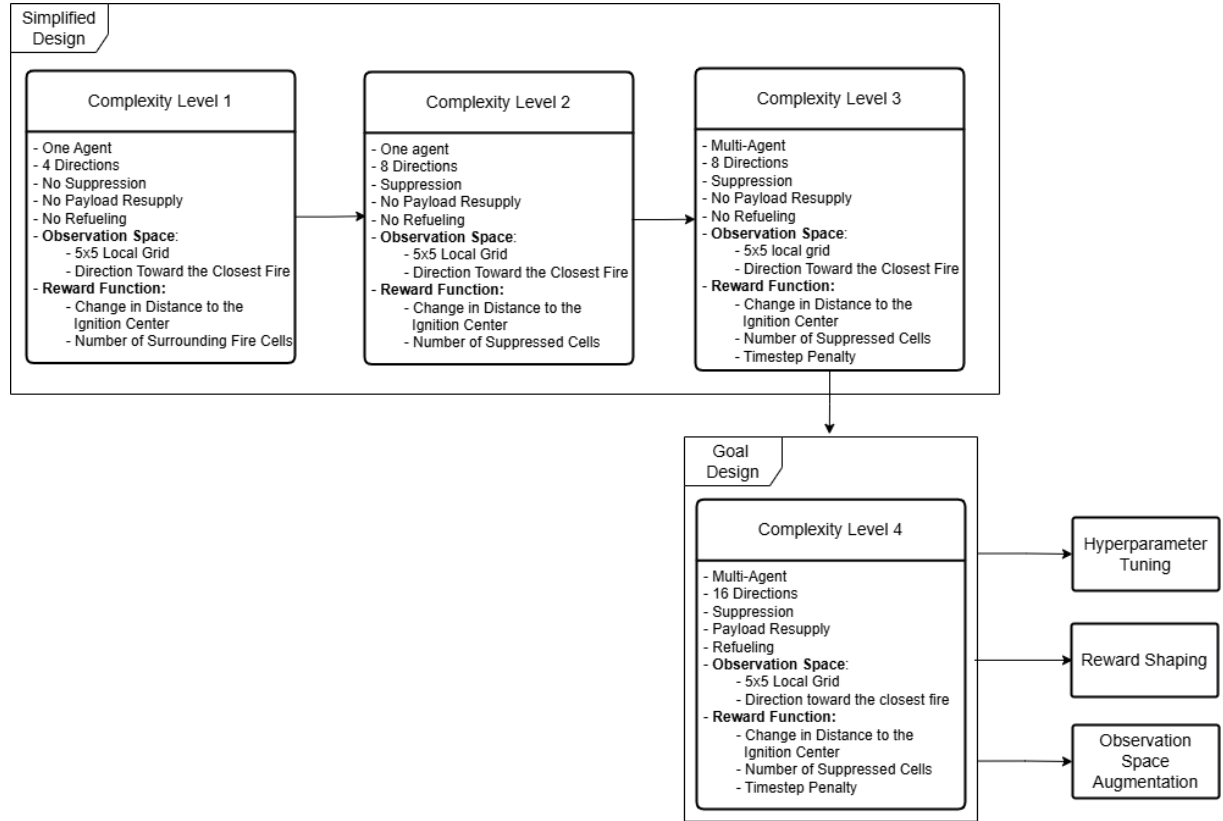


Figure 4.1: Complexity Levels.

4.4 Dual Decomposition Framework

The central contribution of this thesis is the Dual Decomposition Framework (DDF), a structured methodology for building and analyzing interpretable reinforcement learning policies. Reward decomposition has previously been shown to improve policy transparency by revealing how distinct reward components influence agent decision-making [Juozapaitis et al., 2019, Septon et al., 2023]. Building on this foundation, the DDF introduces a parallel decomposition of the observation space alongside the reward function, enabling a joint analysis of how environmental perception and reward structure interact to shape emergent agent behavior. The modular architecture further facilitates the integration of new components and supports systematic investigation of their individual contributions to the learned policy.

As illustrated in Figure 4.2, the DDF consists of three interconnected stages, with the two parallel approaches distinguished by color: the rule-based approach is shown in blue and the RL approach in green.

The first stage, *Problem Setting*, formalizes the operational scenario by defining the scenario configuration, environment parameters, and fleet composition. Environmental parameters include, for example, atmospheric conditions, terrain features, vegetation, and infrastructure. These inputs are translated into operational requirements and concept of operations (ConOps), which in turn inform the SoS Architecture in terms of composition, configuration, and control. This stage establishes a shared evaluation context that applies equally to both the RL and heuristic approaches, ensuring consistent comparability.

The second stage, the *RL Framework*, contains the *Decomposition Module* and the simulation environment. The Decomposition Module organizes both the observation space and the reward function into modular, independently configurable components. Multiple configurations can be assembled by selecting and combining individual components, enabling controlled ablation studies without modifying the underlying training architecture. Within the environment, the SoSID Toolkit executes the wildfire simulation and provides the interface for the standard RL interaction loop: the agent receives observations and rewards from the environment, selects actions according to the current policy, and the resulting experience is used to update the policy through training.

Decomposing the reward and observation space independently provides only partial insight. If a reward component improves performance, it remains unclear whether this is

due to the incentive itself or because the agent lacked the perceptual input needed to act on a different reward. Conversely, adding an observation component may appear ineffective if the reward function does not incentivize using it. Only by varying both dimensions in a controlled, coordinated manner can their interaction be properly analyzed. The DDF makes this joint analysis tractable by modularizing both dimensions simultaneously.

The third stage, *Tactic Evaluation*, compares trained RL policies against heuristic baselines using total burned area as the primary performance metric. Since both approaches operate under the same problem settings defined in Stage 1, the comparison is made under consistent conditions.

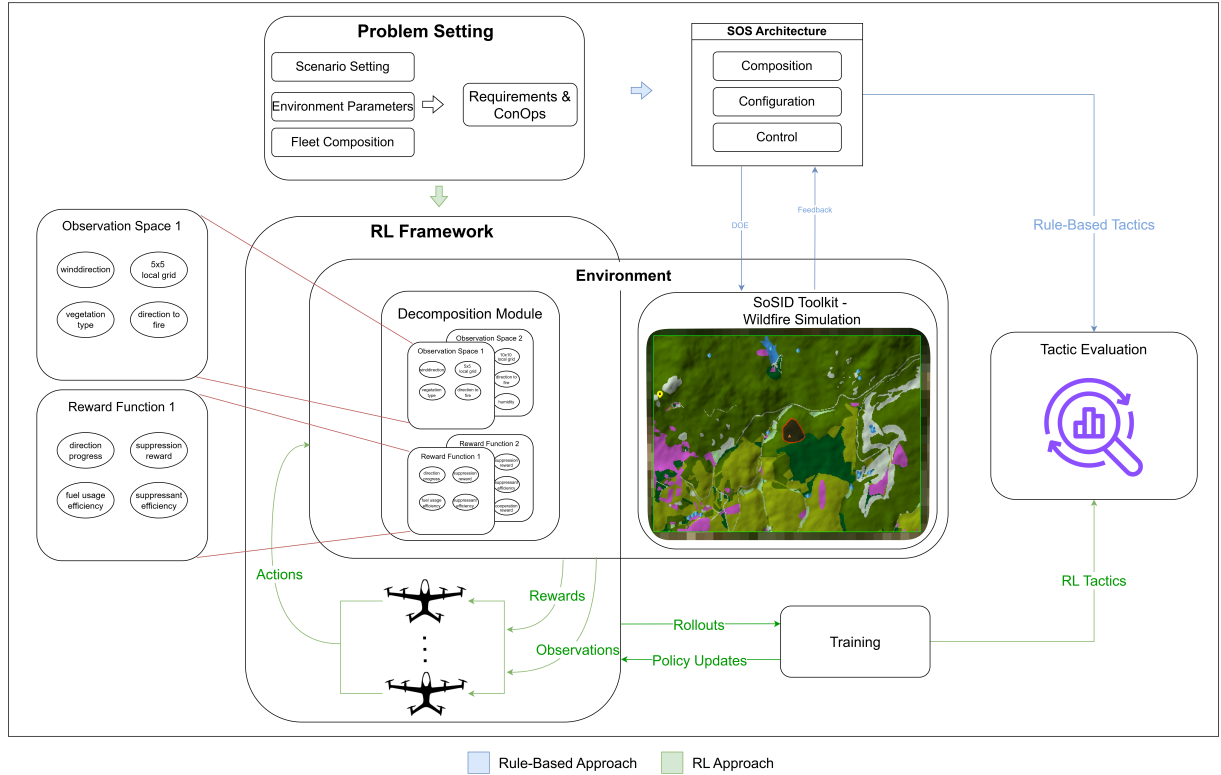


Figure 4.2: **Dual Decomposition Framework.** The framework consists of three stages: Problem Setting, RL Framework with Decomposition Module, and Tactic Evaluation. Blue elements correspond to the rule-based approach; green elements correspond to the RL approach.

4.5 Action Space

The action space consists of discrete movement and suppression actions. Movement actions correspond to directional transitions across the grid, whereas the suppression action extinguishes burning cells within the agent’s current location. Initial experiments employed a four-directional movement set aligned with the cardinal directions (North, South, East, West) to limit action-space dimensionality and improve training stability during early development. The final configuration extends the discrete movement set to 16 evenly distributed directions, thereby enabling finer-grained spatial control and improved maneuverability during suppression operations. As illustrated in Figure 4.3, the action space is progressively expanded throughout the complexity levels, beginning with four cardinal directions, followed by eight directions, and ending in sixteen directional headings in the final configuration.

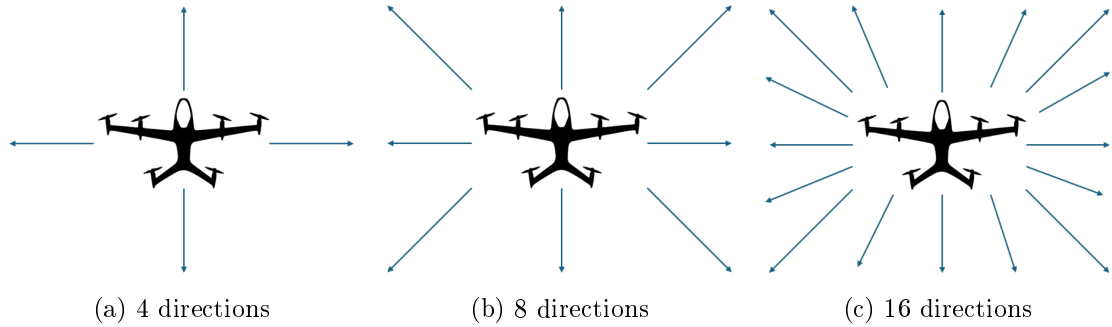


Figure 4.3: The action space variations

4.5.1 Alternative Action Space Approaches

Orientation-based actions, in which the agent turns by a fixed angle followed by forward movement, allow smooth trajectories but increase the effective planning horizon, as the agent must chain multiple turn actions to reach a suppression target. This formulation is better suited to fixed-wing platforms such as seaplanes, which require continuous forward motion for maneuvering, and was therefore not adopted here.

Burning-cell-based actions, in which the agent directly selects a burning cell as its target, remove the need to learn exploration and fire localization. However, the action space varies dynamically with the number of active burning cells, growing as the fire

spreads and collapsing as cells are suppressed, introducing non-stationarity that complicates policy learning. Additionally, restricting actions to currently burning cells prevents the emergence of anticipatory suppression strategies, in which agents position themselves ahead of the fire front rather than reacting to it.

Direction-based actions avoid both of these limitations and constitute a natural abstraction for eVTOL platforms, which can maneuver independently of forward velocity. This work therefore adopts a discrete direction-based action space, as described above.

4.6 Observation Space

Following the decomposition principle of the DDF, the observation space is organized as a set of modular perceptual components that can be independently activated or deactivated per experiment. This design enables controlled variation of the information available to each agent and supports systematic analysis of how individual observation modalities affect the learned policy. The components used in this work are described below:

- **Local Grid:** Provides the agent with local situational awareness by encoding the fire state of the cells within a configurable grid centered on the agent’s position. Each cell in the grid corresponds to one of the discrete environmental states, including *flammable*, *non-flammable*, *extinguished* and various fire stages. The grid size is set at the beginning of each experiment and determines how much of the environment is visible to the agent.
- **Closest N Fire Direction:** Direction vectors from the agent’s position toward the N nearest actively burning cells, where N is defined at experiment initialization. Providing a basic navigational signal for locating and approaching the fire.
- **Last Suppression Direction:** A direction vector from the agent’s position to the most recently suppressed cell across the fleet. This gives the agent a persistent spatial reference to the active suppression front, supporting continuity in suppression behavior.
- **Cardinal Directions:** Four direction vectors pointing from the agent toward the northernmost, southernmost, easternmost, and westernmost burning cells on the fire perimeter. Together they convey the spatial extent and distribution of the active fire in a compact form.

- **Ignition Center Direction:** A direction vector from the agent’s position toward the fire ignition point. Since this reference remains fixed as the fire spreads, it provides a stable spatial anchor for strategies that take the fire’s origin into account.

4.7 Reward Function

Following the same decomposition principle, the reward function is divided into individual components that can be independently combined per experiment. Each component is assigned a weight that determines its relative contribution to the total reward signal. The components used in this work are described below:

- **Fire Proximity Reward:** A dense reward signal based on the change in distance between the agent and the nearest burning cell. Moving closer to the fire yields a positive reward, while increasing the distance incurs a penalty. This continuous feedback guides agents toward the fire front throughout training, reducing the exploration burden compared to sparse reward formulations.
- **Last Suppression Proximity Reward:** A proximity-based dense reward that is calculated relative to the most recently suppressed cell by any agent. Before the first suppression occurs, it falls back to the Fire Proximity Reward and uses the distance to the nearest burning cell instead. This directs agents toward the active suppression front, promoting coordinated and continuous fire containment.
- **Suppression Reward:** Encourages active fire extinguishment by rewarding agents proportionally to the number of cells successfully suppressed. Attempting to suppress a cell that is not actively burning results in a penalty, discouraging wasteful suppression actions.
- **Connected Suppression Reward:** Rewards suppression actions based on whether the newly extinguished cells are spatially connected to an existing suppressed region. This promotes the formation of a continuous containment line rather than isolated suppressions scattered across the fire perimeter. As with the Suppression Reward, attempting to suppress a cell that is not burning results in a penalty.
- **Inactivity Penalty:** A negative reward applied when an agent remains within proximity of the fire front without executing a suppression action. This discourages passive behavior and promotes decisive engagement near active burning cells.

-
- **Timestep Penalty:** A small negative reward applied at every simulation step. Since the cumulative penalty grows with episode length, agents that suppress the fire more efficiently accumulate a smaller total penalty, aligning the optimization objective with operational efficiency.
 - **Mission Success:** Returns a large positive reward when all burning cells have been suppressed before episode termination.

4.7.1 Reward Normalization

To ensure that component weights reflect intended relative importance rather than differences in raw value magnitude, each reward component is independently normalized to the range $[-1, 1]$ before weighting is applied. The total reward signal is then computed as the weighted sum

$$r = \sum_i w_i \cdot \hat{r}_i, \quad (4.1)$$

where $\hat{r}_i$ denotes the normalized value of component i and $w_i \in [0, 1]$ its associated weight. The weights are constrained to sum to one, ensuring that the total reward signal remains within the normalized range $[-1, 1]$.

The weights were not determined through systematic optimization, but selected empirically based on preliminary experiments conducted during development (see Section 6.3). Suppression-related components, such as the Suppression Reward and Connected Suppression Reward, were assigned comparatively high weights, as these components provide the most direct learning signal toward the primary objective of fire extinguishment and were observed to accelerate the acquisition of effective suppression behavior. Supporting components, such as proximity rewards and penalty terms, received lower weights to provide auxiliary guidance without overshadowing the core suppression incentive. Within the supporting components, weights were distributed equally to ensure that the individual contribution of each auxiliary signal could be assessed in isolation, as unequal weighting would make it difficult to isolate and evaluate the individual impact of each component on agent behavior.

5 Implementation

5.1 RL Implementation

The RL implementation is built on Ray RLlib [Liang et al., 2018, Wu et al., 2021], a distributed reinforcement learning library within the Ray framework [Moritz et al., 2018]. The RL framework is integrated into the SoSID Toolkit’s ABM simulation through a custom Gymnasium environment that acts as an interface between the learning algorithm and the simulation (Figure 5.1). At each timestep, the Gymnasium environment queries the simulation state and constructs a structured observation vector for each agent. The selected actions, i.e., directional movement or suppression commands, are translated into control commands and executed within the ABM. This wrapper allows the standard RL interaction loop to operate on top of the existing simulation without modifying its internal logic.

A clear separation of concerns is maintained between the ABM and the RL module. The ABM handles fire propagation, terrain modelling, and low-level agent mechanics, while the RL module is concerned only with policy learning. The Dual Decomposition Module, described in Section 4.4, is responsible for composing the observation space and reward function from configurable components. All configurations are defined in a structured JSON file, making experiments reproducible and allowing scenario parameters such as fire size or agent count to be adjusted without changing the source code.

Distributed training is handled by Ray RLlib, which runs multiple simulation instances in parallel across CPU cores. Each instance collects experience independently, which accelerates training and broadens state space coverage. To support this, the ABM was adapted to run in a stepped, synchronous mode so that each RL timestep corresponds to exactly one simulation step.

Training progress was monitored via TensorBoard, which logged episode rewards, suppression statistics, and burnt area over time. This allowed iterative refinement of both the reward function and environment parameters throughout the training process.

During this implementation process, a simulation inconsistency was identified in the cell state transition logic. When agents applied a suppression action to already burnt cells, these cells transitioned from the *Burnt* state to the *Suppressed* state. Since the evaluation metric counts only cells in the *Burnt* state at the end of an episode, this transition artificially reduced the measured burned area, producing an inaccurate assessment of suppression performance. The issue was resolved by restricting suppression state transitions to *Burning* and *Combustible* cells, ensuring that *Burnt* cells are excluded from the suppression logic and that the burned area metric accurately reflects the fire damage.

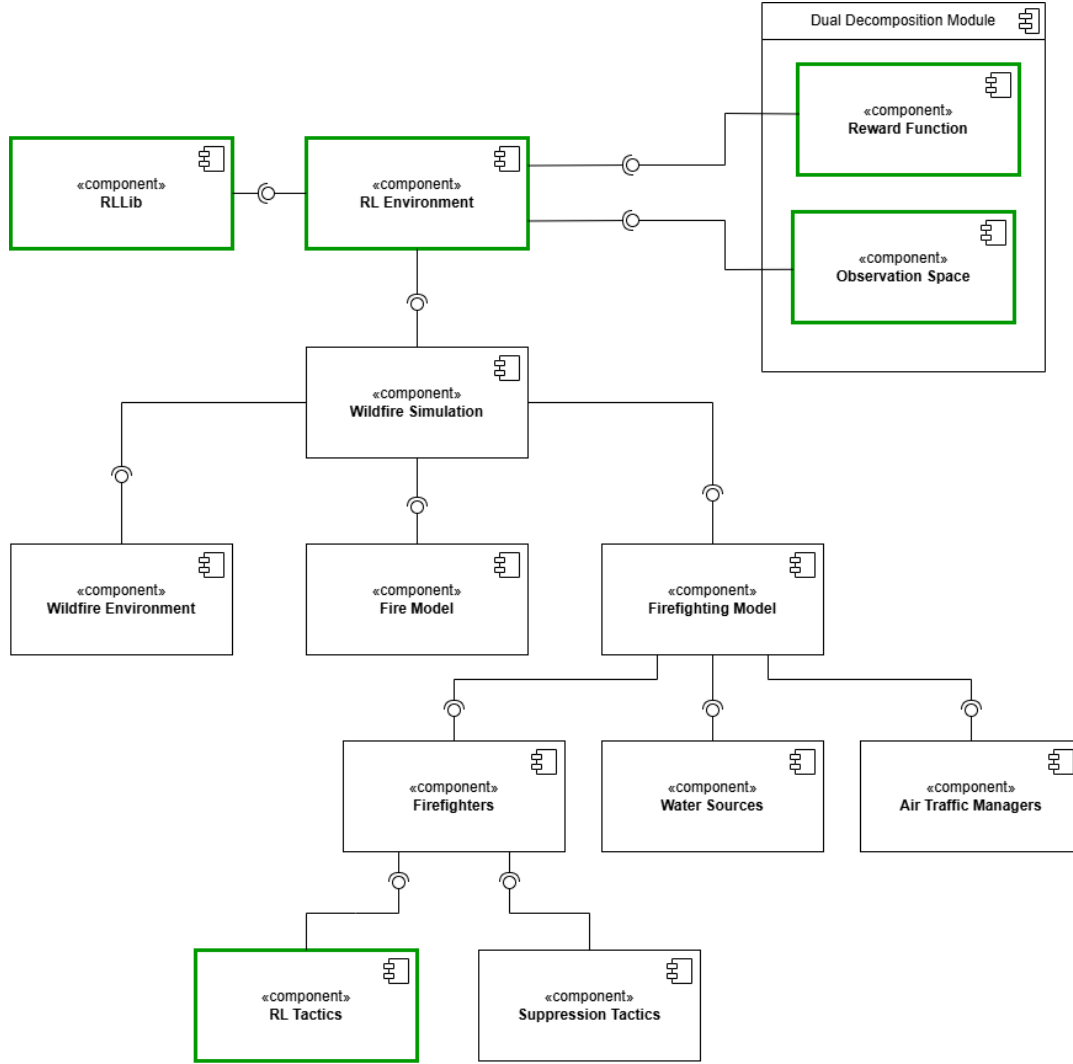


Figure 5.1: **Component diagram of the RL training architecture.** Illustrates the integration of the RL framework into the SoSID Toolkit. Components introduced as part of this work are highlighted in green.

5.1.1 Tactic Integration

SuppressionTactic is a pre-existing, rule-based finite-state machine (FSM) that governs agent behavior through a sequence of operational states. To enable learning-based decision-making, the *RLTactic* class is introduced (Figure 5.1). *RLTactic* inherits from *SuppressionTactic* and selectively overrides specific decision points while preserving the original state-transition structure. In particular, the method *trigger_select_firefront()* is replaced by an RL-driven decision step, denoted as *select_action()*. After takeoff, agents transition to the *select_action()* state, where tactical control is delegated to a learned policy. In this state, the agent receives its current observation and selects an action according to the reinforcement learning policy, which is then executed within the tactical framework.

The action execution differs depending on whether the system operates in training or deployment mode. During training, RLib calls *env.step(actions)* on the Gymnasium environment, which in turn calls the step function of the simulation with the actions for each agent. The simulation then triggers *select_action()* for each agent through the FSM, where the action is executed (e.g., moving in a specific direction or suppressing the current cells). This ensures that all state transitions used for policy optimization are directly attributable to the policy’s decisions. As described in Sections 5.2 and 5.3, the refueling and resupply procedures, as well as the initial positioning of the agent are handled differently during training than in the deployment phase.

During deployment, a trained policy is loaded and queried directly using the current observation. The resulting action is executed via the same *select_action()* interface, without interaction with the training environment. Overall, *RLTactic* functions as a hybrid control architecture that embeds a learned decision module within an otherwise deterministic tactical state machine. This design maintains the original state-transition logic of *SuppressionTactic*, ensuring that RL-based and heuristic approaches operate under identical operational constraints and remain directly comparable.

5.1.2 Discrete Action Movement Mapping

To enable navigation for the RL-controlled agents, the discrete action space is mapped onto the existing goal-based control scheme. As described in Section 2.2, agents are assigned a target destination and navigate toward this goal using a rule-based navigation controller. In the RL-based implementation, high-level actions correspond to directional

commands rather than direct goal assignments. When an action from the discrete action space is selected (e.g., NORTH), an intermediate navigation goal is generated by offsetting the agent’s current position by a fixed number of grid cells in the corresponding direction. The agent then follows the predefined rule-based navigation logic to reach this intermediate goal. This procedure is repeated at every simulation step, allowing the learned policy to influence agent movement while preserving the existing low-level navigation and control mechanisms.

Liang–Barsky Clipping

Depending on the agent’s position and the selected direction, the computed intermediate goal may fall outside the rectangular active area (black cross in Figure 5.2). To keep agents within the operational region and avoid expanding the effective state space, the destination is clipped to the area boundary before being passed to the navigation controller. A naive approach of clipping the destination coordinate-wise to the boundary would distort the trajectory, since each axis is truncated independently (red cross in Figure 5.2). The Liang–Barsky line clipping algorithm [Liang and Barsky, 1984] avoids this by computing the intersection of the directed ray from the agent’s current position to the computed destination with the boundary of the active area. The result is the furthest reachable point along the original direction that lies within bounds, preserving the intended heading (blue cross in Figure 5.2).



Figure 5.2: **Liang–Barsky destination clipping.** The **black cross** marks the originally computed destination outside the active area. The **red cross** shows the result of naive coordinate-wise clipping, which distorts the heading. The **blue cross** shows the clipped destination produced by the Liang–Barsky algorithm, which preserves the original direction.

Adaptive Step Size

In the SoSID Toolkit, each agent maintains a target altitude determined by the terrain below the navigation goal, for example following the slope of a hillside. When the goal is sufficiently far away, the required altitude adjustment is distributed across the horizontal flight path, allowing the agent to ascend or descend gradually during cruise. When the goal is close, however, there is insufficient horizontal distance to complete the altitude change during forward movement, so the agent spends most of the step ascending or descending vertically with only limited horizontal progress. To ensure consistent horizontal movement comparable to heuristic agents, the step size, i.e., the offset distance used to compute the intermediate goal, is set to a fixed value large enough to maintain horizontal flight.

During training, however, this fixed step size caused agents to overshoot burning cells. When approaching burning cells, the intermediate goal was placed beyond individual burning cells, causing agents to fly over them without triggering suppression, or to reach them only after multiple passes. To address this, an adaptive step size was introduced, by reducing the offset distance when the agent is within a defined proximity of the nearest burning cell. This allows agents to navigate more precisely near the fire front and reliably reach individual cells for suppression.

5.1.3 Reward Component Implementation

This section details the implementation logic of the individual reward components introduced in Section 4.7. For each component, a flowchart illustrates the per-agent computation executed at every timestep.

Proximity Reward

The Proximity Reward provides a dense guiding signal that directs agents toward the fire. The computation is illustrated in Figure 5.3.

At each step, the agent’s current distance to the nearest burning cell is compared against the distance recorded in the previous step. If the agent has just performed a suppression action or a resupply was triggered, the reference distance is reset before the comparison. This reset is necessary due to the refueling and resupply abstraction described in Section 5.2. As the agents are repositioned instantaneously during training, the previous

distance would otherwise produce a misleading reward signal.

Once the agent comes within the `max_reward_distance` of a burning cell, it receives the maximum reward regardless of the distance reduction. This allows the agent to maneuver freely near the fire without being penalised for small positional adjustments. Outside this range, the reward depends on the magnitude of the distance reduction: a reduction exceeding the progress threshold yields the maximum reward, a smaller reduction yields a partial reward, and an increase in distance yields the minimum reward. The reference distance is updated at the end of each step.

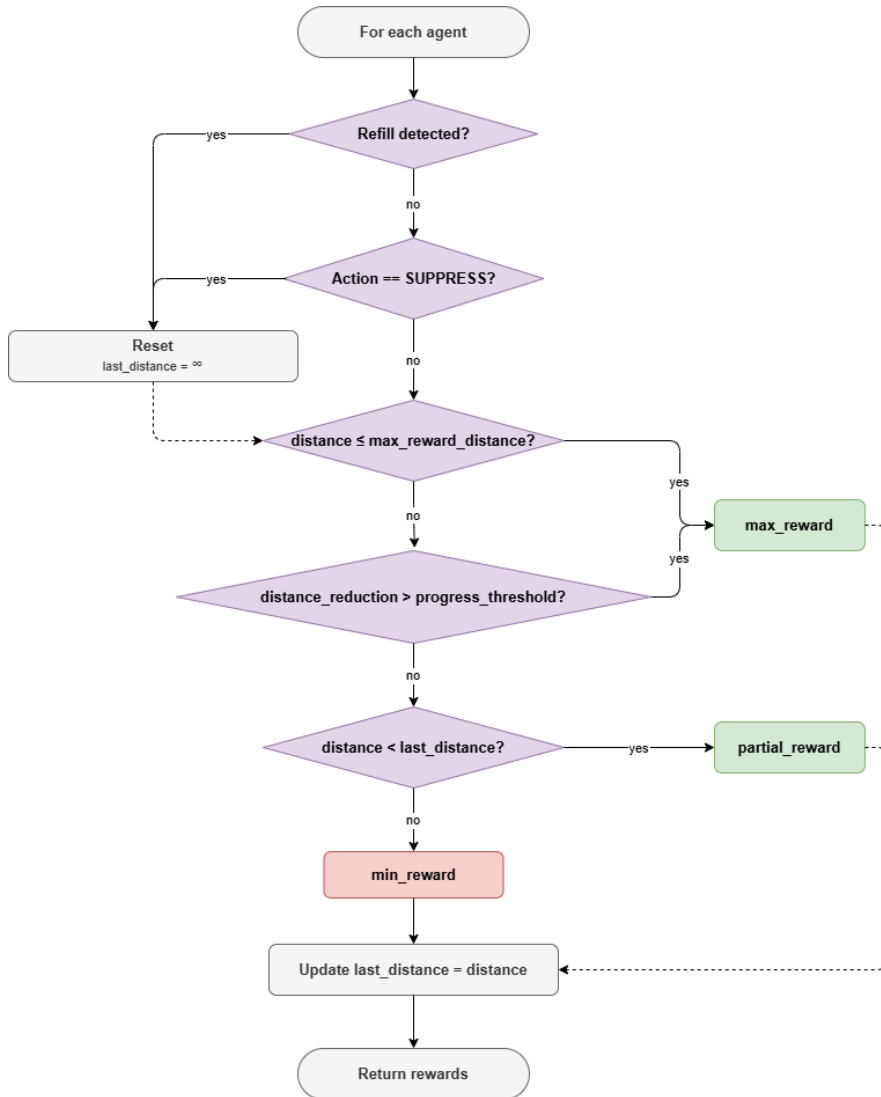


Figure 5.3: **Computation Flow of the Proximity Reward.**

During development, a reward hacking behavior was observed with an early version of this component that applied only a positive reward for approaching the fire, with no penalty for moving away. Agents discovered that they could repeatedly fly toward the fire to collect positive reward, retreat without penalty, and approach again, collecting the proximity reward indefinitely without ever suppressing a burning cell. The penalty for moving away from the fire, described above, was introduced to eliminate this problem.

Inactivity Penalty

The Inactivity Penalty addresses the case where an agent is positioned near the fire but repeatedly chooses not to suppress. The computation is shown in Figure 5.4. An *activation distance* defines the radius around the fire within which inactivity is tracked. At each step, if the agent executes a suppression action, the *inactivity streak counter* is reset to zero. If no suppression occurs and the agent is within the activation distance, the counter is incremented. Once the counter reaches the configured threshold, the minimum reward value is applied as a penalty. Outside the activation distance, no penalty is issued regardless of the action taken.

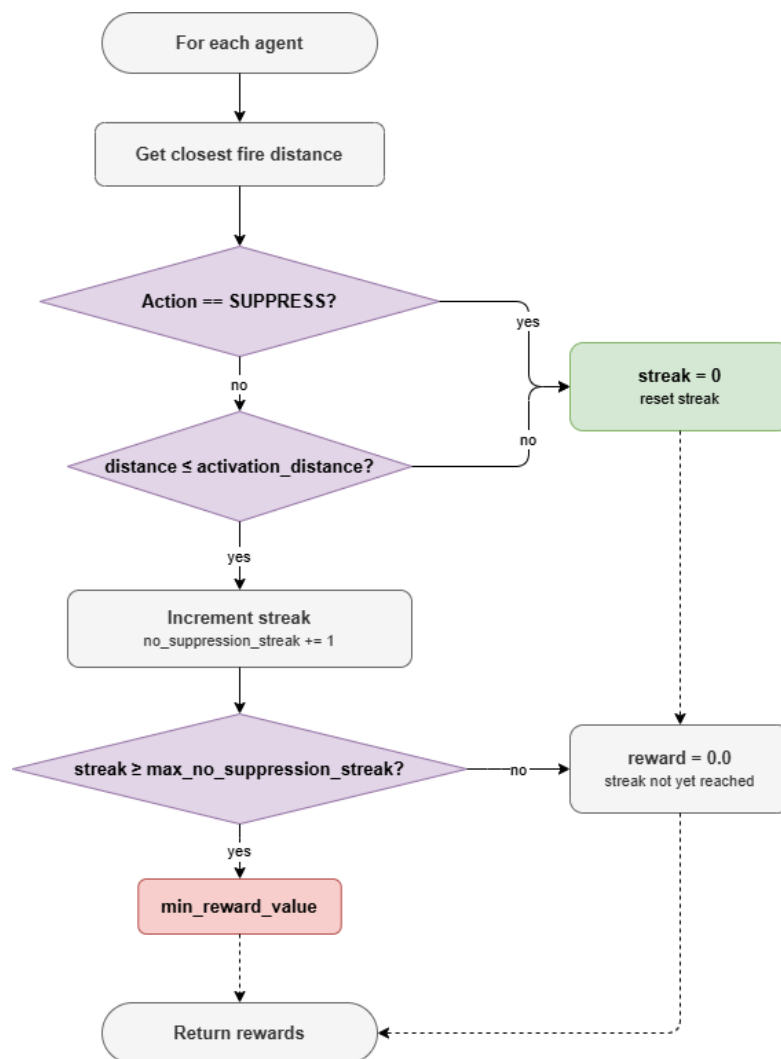


Figure 5.4: Computation Flow of the Inactivity Penalty.

Suppression Reward

The Suppression Reward incentivises agents to actively extinguish burning cells, as illustrated in Figure 5.5. When the agent does not perform a suppression action, no reward is assigned. To determine how many cells an agent extinguished in the current step, two naive approaches fall short. Counting the total number of suppressed cells in the environment introduces an Agent Credit-Assignment Problem, as the contribution of an individual agent cannot be separated from that of others. Counting the suppressed cells visible in the agent’s current local grid observation is equally inadequate, since this includes cells suppressed in previous timesteps and therefore does not reflect the outcome of the current action. Instead, the agent’s local grid observation from the previous timestep is compared with the current one, and only cells that transitioned from burning to suppressed within that single step are attributed to that agent. If no cells were extinguished despite a suppression action, the minimum reward value is applied. Otherwise, the reward scales proportionally with the number of attributed cells, capped at a configurable maximum.

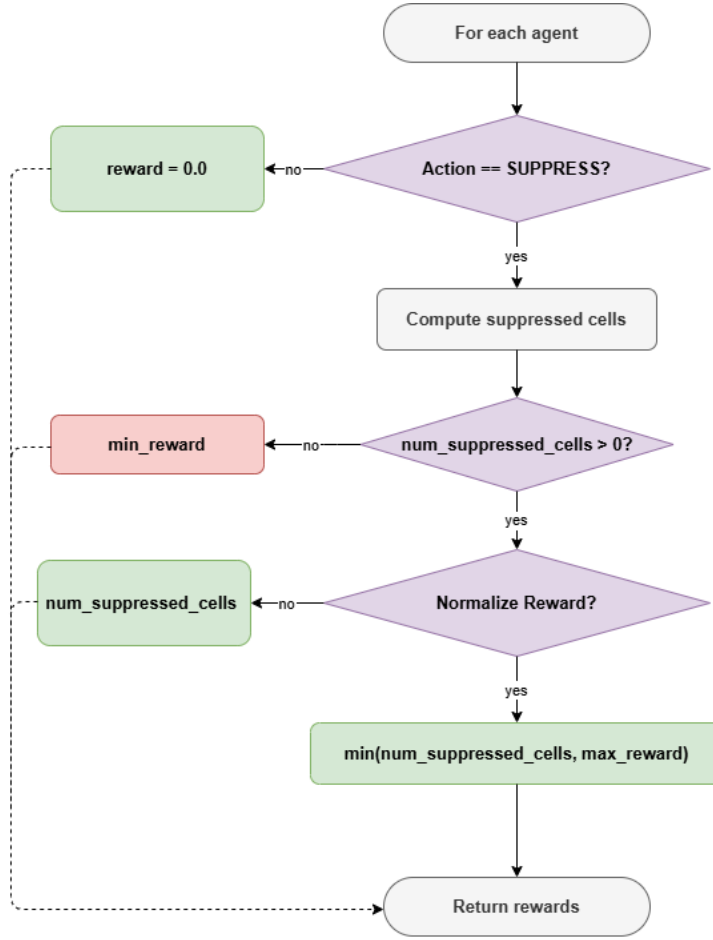


Figure 5.5: **Computation Flow of the Suppression Reward.**

Connected Suppression Reward

The *Connected Suppression Reward* extends the *Suppression Reward* by incorporating spatial context into the reward calculation. Rather than rewarding the raw count of newly extinguished cells, it considers the shape and extent of continuous suppression regions. The same local grid comparison used in the *Suppression Reward* is applied to assign newly suppressed cells to the acting agent, thereby avoiding the problem of credit assignment described above. Once the newly suppressed cells are identified, an eight-directional flood-fill algorithm retrieves the full connected region of suppressed cells. This region is then passed to one of three reward calculation methods, described below. The full computation flow is shown in Figure 5.6.

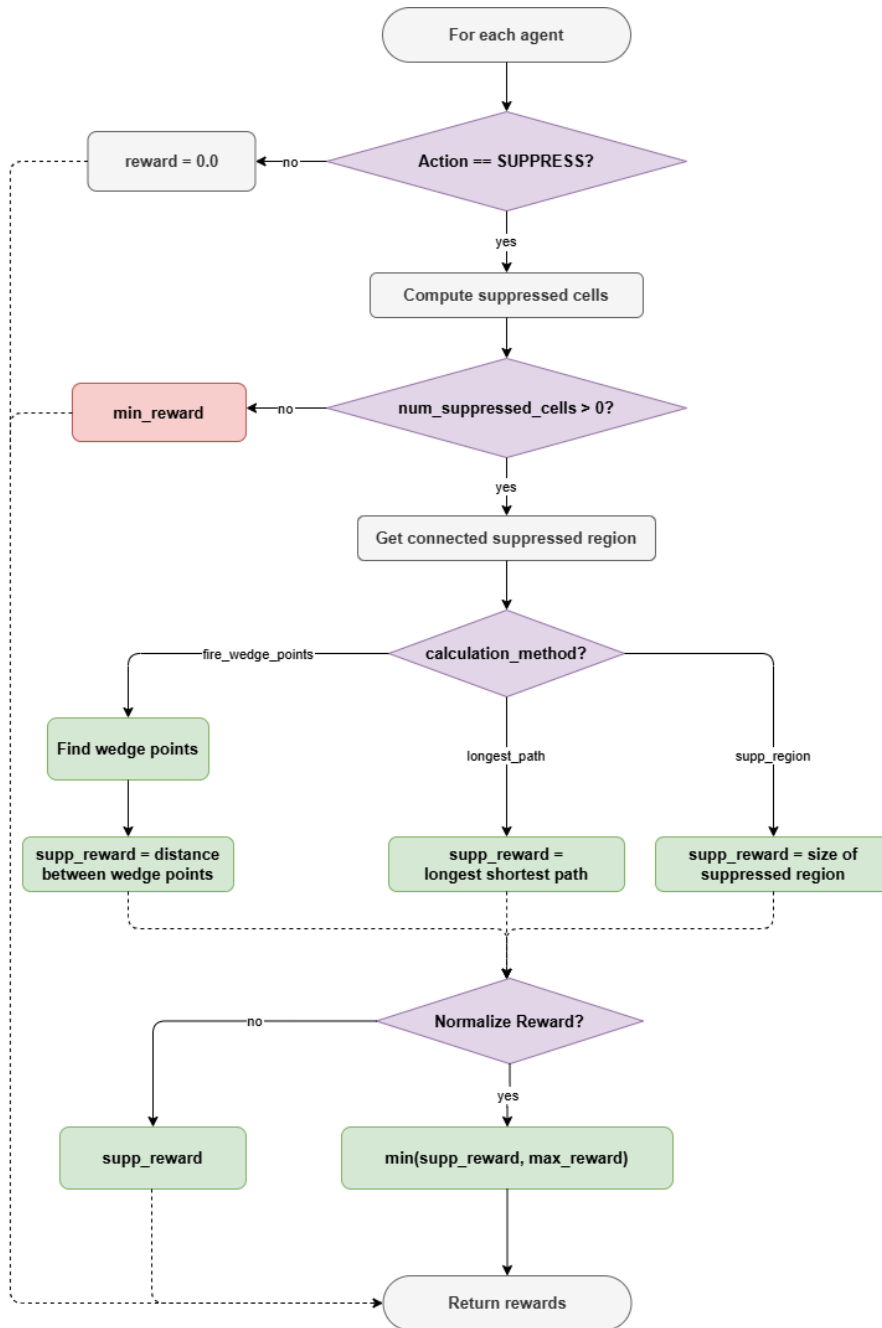


Figure 5.6: Computation Flow of the Connected Suppression Reward.

Region Size Method The simplest of the three methods uses the number of cells in the connected suppressed region directly as the reward. While straightforward to compute, this method does not consider the shape of the region: a compact, blob-shaped suppression area yields the same reward as a linear suppression line of equal size, even though the latter is considerably more effective at containing fire spread.

Graph Diameter Method This method addresses the shape limitation of the Region Size Method by measuring the spatial extent of the suppressed region rather than its size. The connected region is treated as an unweighted graph, where each cell is a node and edges connect eight-directionally adjacent cells. The diameter is approximated using two successive Breadth-First Search traversals: the first BFS starts from the agent’s current position and finds the most distant cell in the region; the second BFS starts from that cell and returns the longest shortest path, which serves as the diameter estimate. The reward is proportional to this value. By rewarding elongated shapes over compact blobs, this method encourages agents to form continuous suppression lines, independently of where the fire originated.

Wedge Point Distance Method The wedge point distance method goes further by also considering the orientation of the suppression region relative to the fire. Each cell in the connected region is converted to a polar angle relative to the ignition center, and the resulting set of angles is sorted. The largest angular gap between consecutive cells is identified, and the two cells on either side of this gap are the wedge points. The reward is the Euclidean distance between these two points. Compared to using the angular span directly, the Euclidean distance approximates the physical width of the remaining gap in the containment perimeter, which scales with how far the suppression region extends from the ignition center. A large distance indicates that the suppression region spans a wide arc around the ignition point, corresponding to effective containment. Unlike the Graph Diameter Method, this approach is ignition-center-relative: the same physical region yields a different reward depending on where the fire started.

Comparison The three methods represent increasing levels of geometric awareness. The Region Size Method rewards coverage without considering shape. The Graph Diameter Method additionally rewards elongated, line-like suppression regions over inefficient blobs. The Wedge Point Distance Method further accounts for the orientation of the suppression region relative to the ignition point, making it the most directly aligned with containment objectives when the ignition location is known.

5.2 Refueling and Resupply Abstraction

The handling of refueling and water resupply differs between training and deployment phases. This distinction is necessary because, during training, the reinforcement learning algorithm continuously outputs control actions for each agent, whereas in the operational simulation an agent may temporarily be unable to execute these actions (e.g., while returning to a base or water source for refueling or resupply). In such situations, the learning algorithm would incorrectly assume that the prescribed actions are being executed, leading to incorrect policy updates and biased gradient estimates.

To avoid this mismatch between action selection and environment dynamics, refueling and resupply are treated differently during training. Under normal deployment, when an agent exhausts its water supply while performing a suppression action, it autonomously selects the nearest water source, follows a predefined trajectory to that location, refills its water tank, and subsequently resumes wildfire suppression.

During training, this process is abstracted to maintain alignment between agent actions and environment transitions. Specifically, when an agent requires resupply, it is instantaneously relocated to the nearest water source, and its water tank is refilled immediately. To preserve physical consistency, the amount of fuel that would have been consumed during the resupply flight is subtracted from the agent’s fuel reserve. From this updated state, the agent continues executing actions provided by the learning algorithm without interruption.

This abstraction ensures that the policy is updated only based on state–action transitions that are fully controlled by the learned policy, thereby improving training stability.

For the refueling instead of being relocated to the airport, the agents are relocated to the position inside the active area which is closest to the airport. The reason is to keep the agent inside the active area to reduce the state space and help with the exploration. From there the fuel tank is being refilled and the agent can continue the training.

5.3 Agent Position Randomization

During training, agents are initialized at a random position within the active area rather than at the vertiports. This removes the passive transit phase, allowing agents to begin interacting with the environment immediately. Randomizing the starting position also improves policy generalization, as the agent is exposed to a wider range of initial conditions during training.

The spatial density of candidate starting positions is controlled by the *sparseness* parameter. During environment initialization, a fixed grid of valid positions is generated within the bounding coordinates of the active area, with spacing determined by the *sparseness* value. At the start of each episode, a position is sampled independently for each agent from this set. Figure 5.7 illustrates how the *sparseness* value affects the distribution of candidate starting positions.

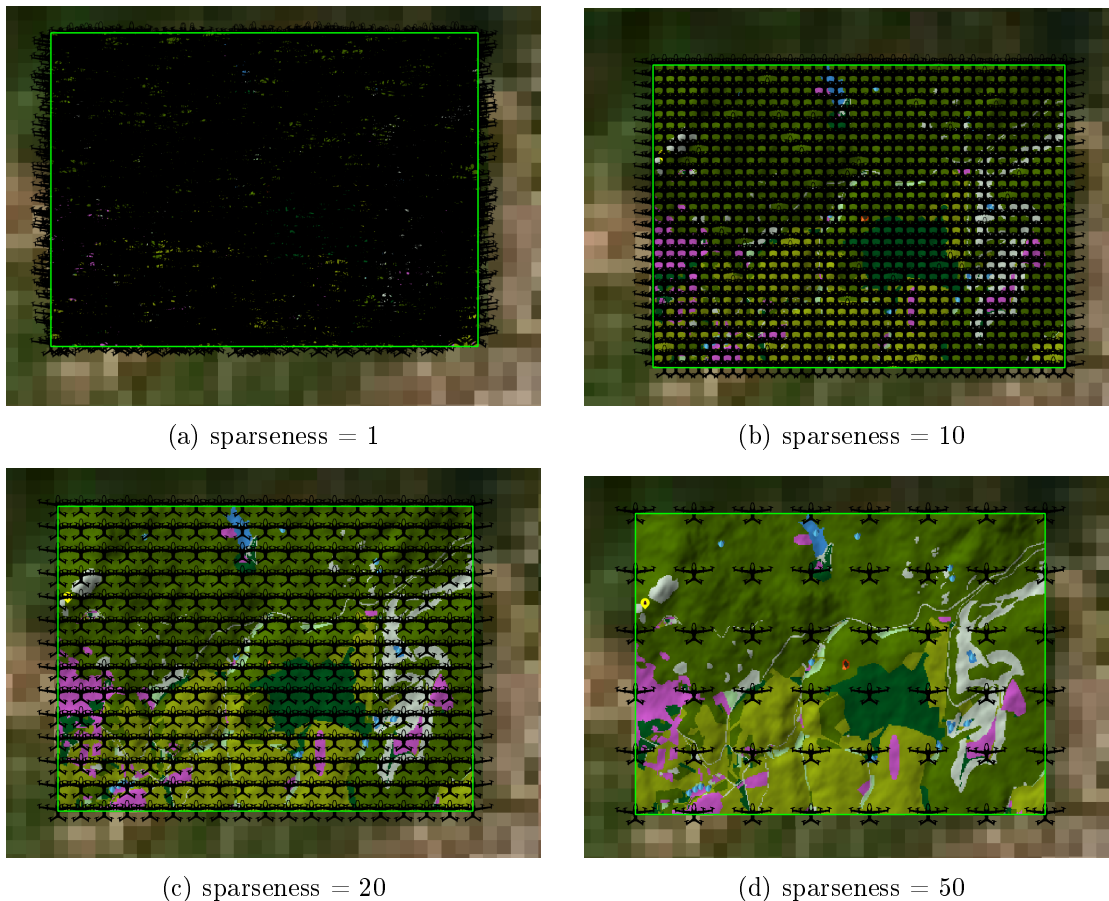


Figure 5.7: **Effect of the Sparseness Parameter on Starting Position Distribution.** Higher values reduce the density of candidate positions, concentrating agent initialization to fewer, more spread-out locations within the active area.

5.4 Wildfire State

As described in Section 5.3, agents are initialized within the active area during training, whereas during deployment they must transit from their respective airports to the wildfire location. This discrepancy introduces a temporal offset: by the time agents reach the fire when starting from the vertiports, the wildfire has typically progressed significantly beyond the initial states. In contrast, during training the fire would still be in its early stages.

To reduce this distribution shift, multiple snapshots representing advanced stages of wildfire progression are captured and incorporated into the training initialization process. Agents are initialized using these pre-evolved fire configurations, ensuring that the policy is exposed to conditions representative of operational deployment. Figure 5.8 shows three such fire states at $cc = 0.2$, with 50, 100, and 150 actively burning cells respectively. This approach additionally improves policy generalization by exposing agents to a diverse range of fire development stages rather than exclusively early ignition scenarios.

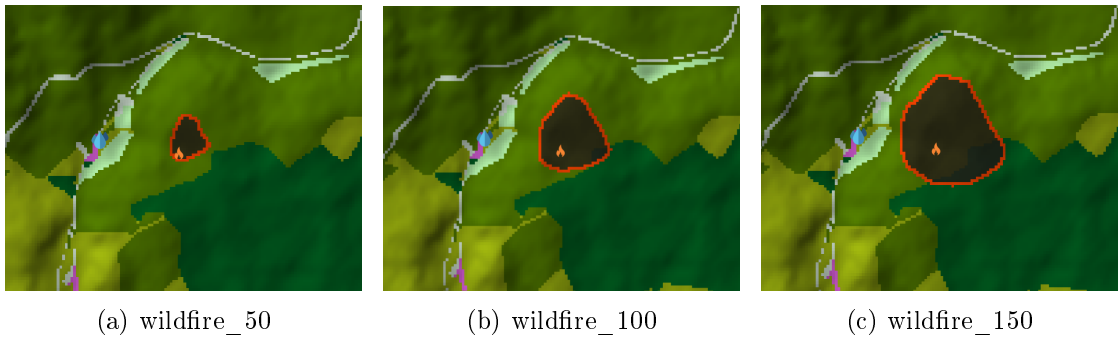


Figure 5.8: The wildfire states.

6 Experiments and Results

6.1 Experimental Setup

All experiments were conducted using a configuration-driven experimentation pipeline. Each experimental run was initialized from a structured JSON configuration file that defined the complete training setup, including training hyperparameters (Table 6.3), scenario and training configuration parameters (Table 6.1), parameters for configuring curriculum learning (Table 6.2) and the observation and reward function components which were described in Section 4.7 and 4.6. An exemplary configuration file illustrating the structure used across all experiments is provided in appendix A.1. This approach ensured reproducibility and enabled systematic modification of experimental conditions without manual changes to the source code. The configuration-based design further supported automated large-scale experimentation. The hyperparameter table also lists the values used for training. These were selected based on the RLlib documentation [Anyscale, Inc., 2024] and preliminary studies. Systematic hyperparameter tuning was not performed due to the time constraints of this thesis. To assess the performance of the trained strategies, an automated evaluation script ran the trained strategies under standardized conditions. The `correction_coefficient` was incrementally increased after each successfully completed mission until reaching a value at which the policy was no longer capable of suppressing the fire, thereby identifying the upper threshold of the policy’s firefighting capability. Performance metrics were logged throughout the evaluation. The primary metric used in this work is the total burnt area, as it directly reflects the wildfire containment objective and provides a consistent measure across both successful and failed suppression missions.

General Settings

Parameter	Type	Description
name	string	Unique identifier for the experiment, used for logging and checkpointing.
type	string	Control architecture: <code>centralized</code> uses a single commander agent; <code>decentralized</code> gives each agent an independent policy.
iterations	int	Total number of training iterations.
max_sim_iterations	int	Maximum simulation steps per episode before forced termination.
max_burning_cells	int	Episode terminates early if simultaneously burning cells exceed this threshold.
num_agents	int	Number of firefighting agents in the environment.
resume_training	bool	If <code>true</code> , resumes from the latest available checkpoint.
norm_rewards	bool	If <code>true</code> , normalizes rewards to the range set by <code>norm_target_range</code> .
norm_target_range	[float, float]	Target normalization range, e.g. <code>[-1, 1]</code> . Active only when <code>norm_rewards</code> is <code>true</code> .
rand_agent_pos	bool	Randomizes agent starting positions each episode. (Section 5.3)
random_sparseness	int	Controls spacing between random start positions. Higher values spread agents further apart. (Section 5.3)
use_16_directions	bool	If <code>true</code> , agents use 16-directional movement; otherwise 8 directions.
load_wildfire	bool	Loads a pre-generated wildfire scenario from file (Section 5.4).
wildfire_filename	string	The <code>.pkl</code> scenario file to load.
tactic	string	Behavioral tactic module: <code>ray_tactic_recharge</code> is used for MARL training with water recharging.
shared_policy	bool	If <code>true</code> , all agents share one policy network, reducing memory and improving sample efficiency.
correction_coefficient	float	Fixed correction coefficient used during training when curriculum learning is disabled.

Table 6.1: General training and scenario configuration parameters.

Curriculum Learning

Parameter	Type	Description
<code>enable</code>	<code>bool</code>	Enables curriculum learning.
<code>adaptive_stop</code>	<code>bool</code>	If <code>true</code> , the episode termination condition is dynamically adjusted relative to the current correction coefficient.
<code>correction_coefficient_start</code>	<code>float</code>	Starting value of the correction coefficient.
<code>correction_coefficient_step</code>	<code>float</code>	Increment applied to the correction coefficient at each curriculum stage.
<code>correction_coefficient_max</code>	<code>float</code>	Upper limit for the correction coefficient.
<code>max_mission_success_streak</code>	<code>int</code>	Number of consecutive successful missions required before advancing to the next curriculum stage.

Table 6.2: Curriculum learning configuration parameters.

Hyperparameters

Parameter	Type	Value	Description
lr	float	5e-4	Learning rate for the optimizer.
gamma	float	0.99	Discount factor for future rewards. Values near 1.0 make the agent more far-sighted.
use_gae	bool	true	Enables Generalized Advantage Estimation (GAE) for more stable training.
lambda_	float	0.95	GAE smoothing factor, trading off bias and variance (0 = high bias, 1 = high variance).
clip_param	float	0.2	PPO clipping parameter ϵ , limiting the size of each policy update.
vf_loss_coeff	float	0.5	Weight of the value function loss in the total training loss.
entropy_coeff	float	0.01	Weight of the entropy bonus, encouraging exploration.
kl_coeff	float	0.2	Coefficient for the KL divergence penalty.
kl_target	float	0.01	Target KL divergence used to adaptively adjust <code>kl_coeff</code> .
vf_share_layers	bool	true	If true, the policy and value function share hidden layers.
fcnet_hiddens	[int]	[512, 256, 128]	Hidden layer sizes of the fully connected network.

Table 6.3: PPO hyperparameters used during training.

6.2 Hardware and Software Environment

All experiments were conducted on a workstation equipped with an AMD Ryzen Threadripper PRO 7975WX CPU (32 cores) and 512 GB of RAM. The system was running Windows 11.

All components were implemented in Python 3.10 and the RL models were developed using PyTorch 2.1 [Paszke et al., 2019]. For the single-agent reinforcement learning development in Complexity Levels 1 and 2, as described in Section 4.3, the Stable-Baselines3 library [Raffin et al., 2021] and Gymnasium [Brockman et al., 2016] were used. For the MARL experiments in Complexity Levels 3 and 4 (Section 4.3), the implementation was migrated to Ray RLlib [Liang et al., 2018, Moritz et al., 2018] to support distributed training and multi-agent coordination. Experiments were executed sequentially. Training a single policy required approximately 24–30 hours, depending on the scenario configuration and environment complexity.

6.3 Ablation Study

To analyze the influence of individual design choices, an ablation study was conducted over the reward function and observation space components described in Sections 4.7 and 4.6. In total, twelve components were considered across both categories. The ablation methodology is enabled by the proposed Dual Decomposition Framework, introduced in Section 4.4, which modularizes both the reward function and the observation space into independent, configurable components. This decomposition allows the systematic activation and deactivation of individual elements without modifying the underlying learning architecture or simulation pipeline, enabling controlled and reproducible experimental comparisons.

Three component groups were treated as mandatory across all configurations, due to their functional necessity and to reduce the number of component combinations:

- **Suppression Reward:** A suppression reward must be present to incentivize agents to actively extinguish burning cells. Either the *Suppression Reward* or the *Connected Suppression Reward* can be applied here.

- **Local-Grid Observation:** Always included, as it provides agents with the minimum spatial context needed to perceive their immediate surroundings and nearby fire states.
- **Mission Efficiency Incentive:** All configurations include either the *Timestep Penalty* or the *Mission Success* reward. During development, a reward hacking behavior was observed in which agents suppressed a small number of burning cells and then waited for the fire to spread further, exploiting the suppression reward by allowing the fire to grow before re-engaging. Since a larger active fire provided more suppression opportunities, agents had no incentive to extinguish the fire quickly. Including either an episode-length penalty or a terminal success reward ensures that efficient suppression is encouraged throughout training.
- **Proximity Reward–Observation Coupling:** Two observation components are only included alongside a corresponding proximity reward. The *N-Closest-Burning* observation is paired with the *Fire Proximity Reward*, and the *Last Suppression* observation is paired with the *Suppression Proximity Reward*. Without a reward signal grounded in the same spatial information, an agent has no incentive to exploit these observations during policy learning, effectively making them redundant. The coupling ensures that each proximity reward and its associated observation reinforce one another: the *Fire Proximity Reward* motivates agents to act on the positions provided by *N-Closest-Burning*, while the *Suppression Proximity Reward* encourages agents to return to locations recorded in the *Last Suppression* observation.

All remaining components were systematically varied during the ablation experiments. Where the *N-Closest-Burning* component is used, it is applied with $N = 1$ throughout all experiments; the effect of larger values of N was not systematically explored due to time constraints. The grid size is set to 15×15 cells, providing the agent with a local view of its immediate surroundings.

The experiments are organized in three sequential phases. A working baseline must first be established before individual components can be meaningfully ablated, since varying a component in a configuration that already fails to achieve the mission objective yields no useful signal. In the first phase, the minimal observation space sufficient for successful wildfire suppression is determined by selectively removing components and recording the outcome. The simplest successful configuration is then held fixed throughout the subsequent phases. In the second phase, the observation space is kept constant while reward

function components are varied in isolation, assessing their individual contributions. In the third phase, three aspects are investigated with the reward function fixed at the best-performing configuration from the second phase: the observation space is extended by progressively introducing additional components, the calculation method of the *Connected Suppression Reward* is analyzed by comparing three variants, and optimization approaches are applied to the best-performing policy. This ordering ensures that each phase rests on validated choices from the preceding one. For this reason, the configuration of each phase and its corresponding results are presented together, allowing the rationale behind each subsequent design decision to be traced directly.

Since evaluating all possible combinations was infeasible within the scope of this work, not all theoretically possible component combinations were evaluated. Instead, experiments were assessed continuously during execution, and only configurations considered meaningful based on intermediate results were pursued. Certain combinations were therefore deliberately not conducted, as they were judged unlikely to provide additional insights. Furthermore, due to time and computational constraints, each configuration was trained and evaluated only once, which means that potential variations in the results from repeated runs were not captured.

All policies were trained for 5,000 iterations, at which point no further improvement in policy performance was observed, indicating convergence. Four agents were selected as the training fleet size based on computational constraints and because this configuration proved sufficient to successfully suppress the selected wildfire scenario. The sparseness value of 20 was chosen to ensure adequate coverage of the active area with agent starting positions, as illustrated in Figure 5.7. Higher sparseness values would distribute agents further apart but reduce the number of reachable starting positions, causing many positions to never be sampled during training. The wildfire scenario was initialized at state 100 (Section 5.4), as this state most closely resembles the fire extent that agents encounter during deployment at a correction coefficient of 0.2.

For evaluation, six agents were deployed starting at a correction coefficient of 0.1, which increases by 0.02 after each successfully suppressed wildfire, until the agents are no longer able to contain the fire. Using fewer agents during training reduces computational effort and accelerates convergence, while increasing the number of agents during evaluation allows the learned strategy to be assessed under more demanding operating conditions. The evaluation fleet was limited to six agents to keep individual evaluation runs computationally tractable, as a larger fleet suppresses the fire more quickly and would require evaluation across a wider range of correction coefficients to identify the performance threshold. Each evaluation episode terminates under one of four conditions: (1) the

wildfire is fully suppressed (success); (2) the number of simultaneously burning cells exceeds 450, a threshold determined empirically by analyzing fire spread behavior, beyond which successful suppression was no longer achievable; (3) the fire reaches the boundary of the active area; or (4) the simulation reaches the maximum of 43,200 timesteps (equivalent to 12 hours of simulated time). Conditions (2) through (4) all constitute failure. In the experiment tables below, $\times$ indicates that a component is active in the given experiment. The *Mission Success* row reports whether the trained policy successfully suppressed the wildfire at the specified correction coefficient: " $\checkmark$ " denotes a successful outcome and "/" denotes failure. The y-axis is not fixed and scales dynamically to the maximum observed value, allowing the full burned area to be displayed and improving visual clarity.

6.3.1 Phase 1: Observation Space Baseline

Table 6.4: Phase 1: Baseline Identification

	Exp.1	Exp.2	Exp.3	Exp.4	Exp.5	Exp.6	Exp.7
Observation-Components							
Local-Grid	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$
N-Closest-Burning							$\times$
Last-Suppression					$\times$	$\times$	
Ignition-Center			$\times$	$\times$		$\times$	
Cardinal-Directions		$\times$		$\times$		$\times$	
Reward-Components							
Fire Proximity	$\times$	$\times$	$\times$	$\times$			$\times$
Suppression Proximity					$\times$	$\times$	
Suppression	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$
Connected-Suppression							
Timestep-Penalty							
Inactivity-Penalty							
Mission-Success	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$
Mission Successful	/	/	/	/	/	/	$\checkmark$
Training cc	0.20	0.20	0.20	0.20	0.20	0.20	0.20

Figure 6.1 shows the burned area over simulation timesteps for Exp.1 through Exp.7 (Table 6.4), all evaluated at a correction coefficient of 0.1. Exp.1 through Exp.6 run until the maximum simulation iteration of 43,200 timesteps is reached, constituting a failure under termination condition (4). The burned area curves of these experiments follow a nearly identical trajectory throughout the full episode duration, reflecting free fire spread in the absence of effective suppression. Exp.7, the only configuration that includes the N-Closest-Burning observation component, is the only configuration recorded as a successful mission (Table 6.4). All subsequent experiments were therefore conducted with the N-Closest-Burning component included. Figure 6.1 is the only evaluation plot that includes experiments in which the fire was not successfully extinguished at the lowest complexity level. All subsequent figures show only episodes with successful suppression. Experiments failing at a given cc value are therefore absent from the corresponding plot.

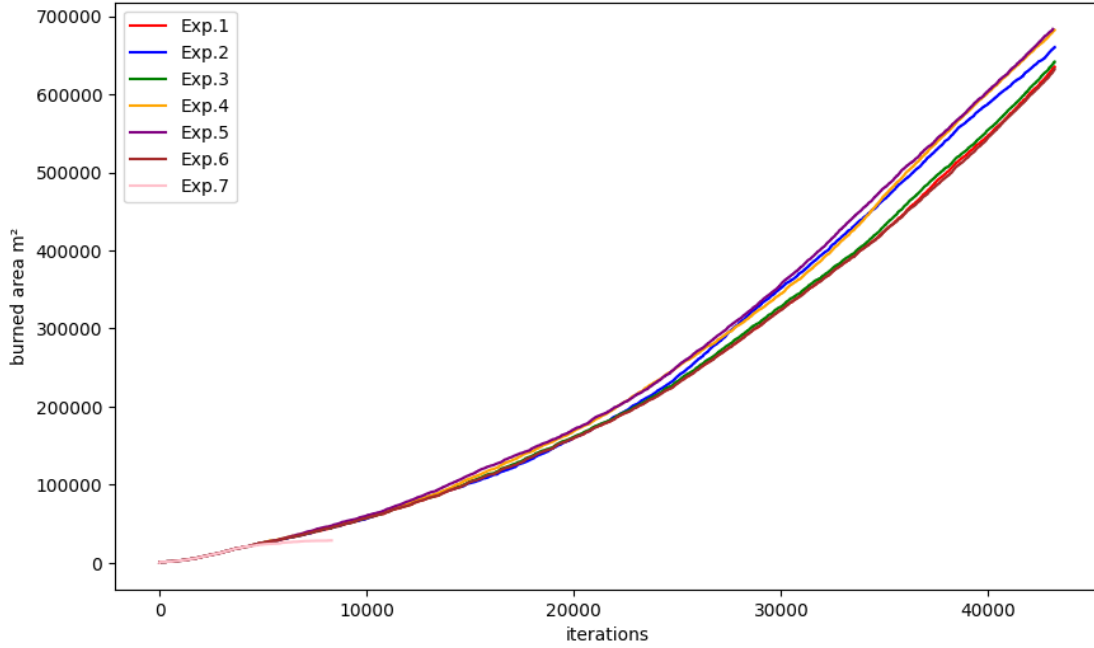


Figure 6.1: **Observation Space Baseline Evaluation.** Performance evaluation of experiments Exp.1 (red), Exp.2 (blue), Exp.3 (green), Exp.4 (yellow), Exp.5 (purple), Exp.6 (brown) and Exp.7 (pink) using a cc value of 0.10.

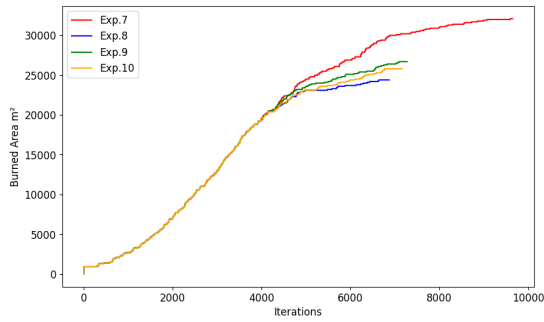
6.3.2 Phase 2: Reward Function Ablation

Table 6.5: Phase 2: Reward Function Ablation

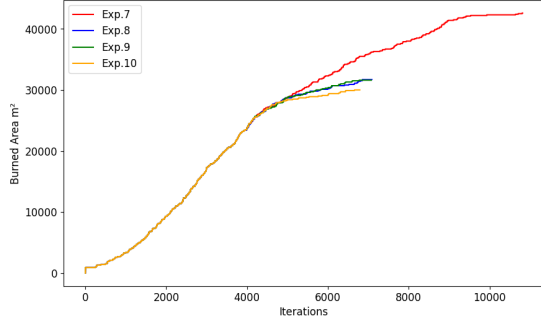
	Exp.8	Exp.9	Exp.10	Exp.11	Exp.12
Observation-Components					
Local-Grid	×	×	×	×	×
N-Closest-Burning	×	×	×	×	×
Last-Suppression					
Ignition-Center					
Cardinal-Directions					
Reward-Components					
Fire-Proximity	×	×	×	×	×
Suppression-Proximity					
Suppression	×	×	×		
Connected-Suppression				×	×
Timestep-Penalty	×		×		×
Inactivity-Penalty		×	×		×
Mission-Success		×		×	
Mission Successful	✓	✓	✓	✓	✓
Training cc	0.20	0.20	0.20	0.20	0.20

Figure 6.2 compares the performance of Exp.8, Exp.9, and Exp.10 (Table 6.5), evaluating the effect of the Timestep Penalty and Inactivity Penalty components. Exp.7 (Table 6.4) serves as the baseline, using neither penalty. At $cc = 0.10$, all four experiments successfully suppress the fire with burned areas ranging from approximately 24,000 m² (Exp.8) to 32,000 m² (Exp.7), indicating comparable performance at low fire intensity. The advantage of the penalty configurations becomes increasingly apparent at higher cc values. At $cc = 0.18$, all four experiments still suppress the fire, but Exp.7 accumulates approximately 153,000 m², compared to 63,000 m² for Exp.8, 68,000 m² for Exp.10, and 80,000 m² for Exp.9 — a gap of approximately 90,000 m² between the baseline and the best-performing configuration. Exp.7 fails at $cc = 0.20$ and is absent from all subsequent plots. At $cc = 0.20$, Exp.8 achieves approximately 85,000 m², Exp.10 approximately 105,000 m², and Exp.9 approximately 170,000 m². Exp.9, which uses the *Inactivity Penalty* together with the *Mission Success* reward, performs worst among the three penalty configurations, successfully suppressing the fire only up to $cc = 0.22$, where it achieves approximately 265,000 m² — compared to 135,000 m² for Exp.8 and 175,000 m²

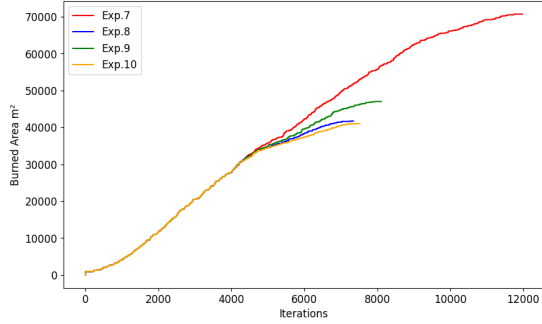
for Exp.10 at the same level. Exp.8, which includes only the *Timestep Penalty*, successfully suppresses wildfires up to a cc of 0.24, where it achieves approximately 175,000 m², compared to Exp.10 at approximately 140,000 m². Exp.10, which includes both penalty components, reaches the highest correction coefficient before failure, successfully suppressing wildfires up to a cc value of 0.26. At $cc = 0.26$, only Exp.10 remains, achieving its final successful suppression at approximately 210,000 m². The penalty configuration of Exp.10 is carried forward as the baseline for all subsequent experiments.



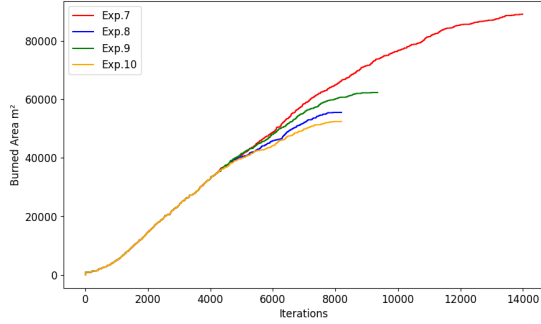
(a) $cc = 0.10$



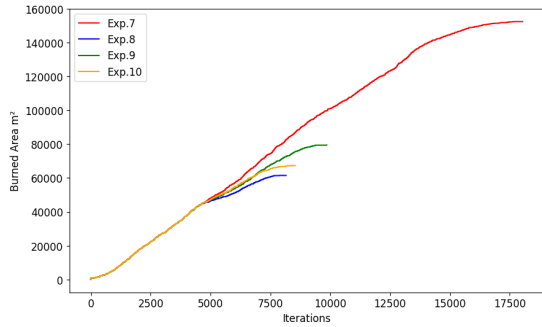
(b) $cc = 0.12$



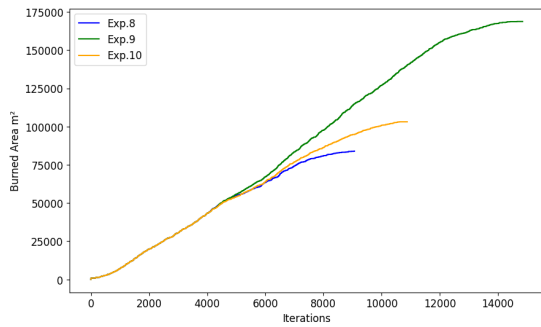
(c) $cc = 0.14$



(d) $cc = 0.16$



(e) $cc = 0.18$



(f) $cc = 0.20$

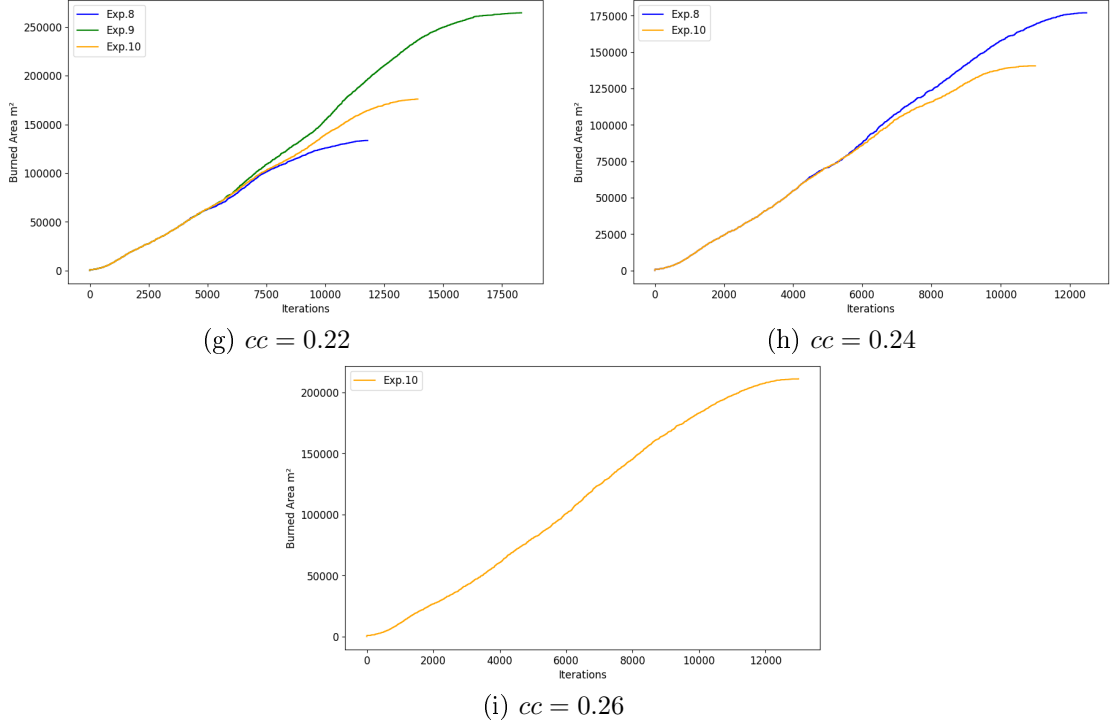
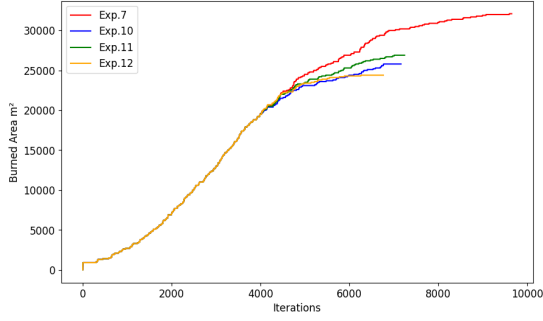


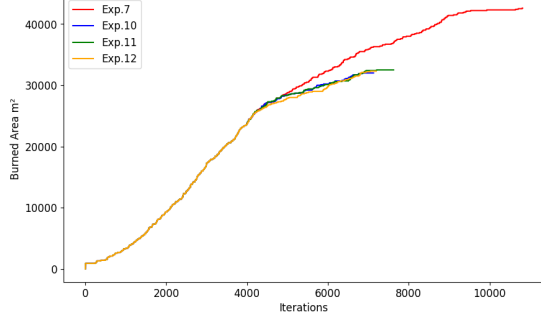
Figure 6.2: **Evaluation of Timestep- and Inactivity-Penalty.** Performance evaluation of Exp. 7 (red) used as a baseline without any penalty components, Exp. 8 (blue) using the Timestep Penalty, Exp. 9 (green) using the Inactivity Penalty, and Exp. 10 (yellow) including both penalty components. The evaluation was performed across increasing fire-spread complexity ($cc = 0.10$ to $cc = 0.26$).

Figure 6.3 compares four configurations: Exp. 7 (*Suppression Reward*, no penalties), Exp. 10 (*Suppression Reward*, with penalties), Exp. 11 (*Connected Suppression Reward*, no penalties), and Exp. 12 (*Connected Suppression Reward*, with penalties) (Tables 6.4 and 6.5). All configurations share the same observation space and differ only in the suppression reward and penalty components used. In Exp. 11 and Exp. 12, the *Connected Suppression Reward* is calculated using the *Region Size Method*; alternative calculation methods are evaluated in Phase 3. At $cc = 0.18$, the last successful suppression for Exp. 7, the no-penalty configurations differ by approximately 78,000 m²: Exp. 7 accumulates approximately 153,000 m² while Exp. 11 achieves approximately 75,000 m². With penalties, Exp. 10 and Exp. 12 both achieve approximately 67,000 m² at this level. At $cc = 0.20$, where Exp. 7 is no longer present, Exp. 11 achieves approximately 116,000 m², Exp. 10 approximately 103,000 m², and Exp. 12 approximately 87,000 m². At $cc = 0.24$, the last successful suppression for Exp. 11, it reaches approximately 180,000 m², com-

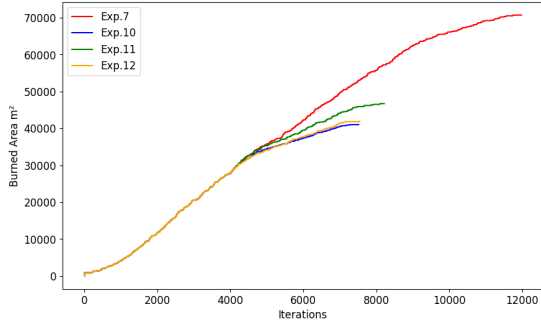
pared to $140,000 \text{ m}^2$ for Exp.10 and $120,000 \text{ m}^2$ for Exp.12. At $cc = 0.26$, only Exp.10 and Exp.12 remain, with Exp.12 achieving approximately $35,000 \text{ m}^2$ less burned area than Exp.10. Exp.10 fails at $cc = 0.28$, while Exp.12 continues to suppress the fire with a total burned area of approximately $325,000 \text{ m}^2$, one cc level beyond Exp.10. Given these results, all subsequent experiments include both penalty components and the *Connected Suppression Reward* component.



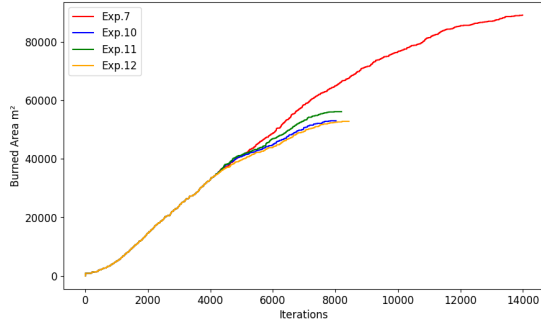
(a) $cc = 0.10$



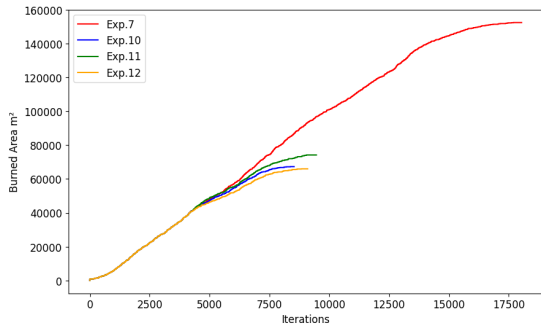
(b) $cc = 0.12$



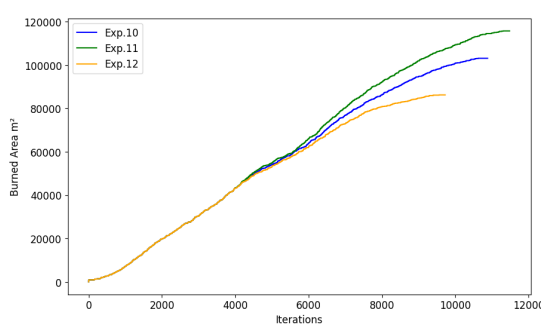
(c) $cc = 0.14$



(d) $cc = 0.16$



(e) $cc = 0.18$



(f) $cc = 0.20$

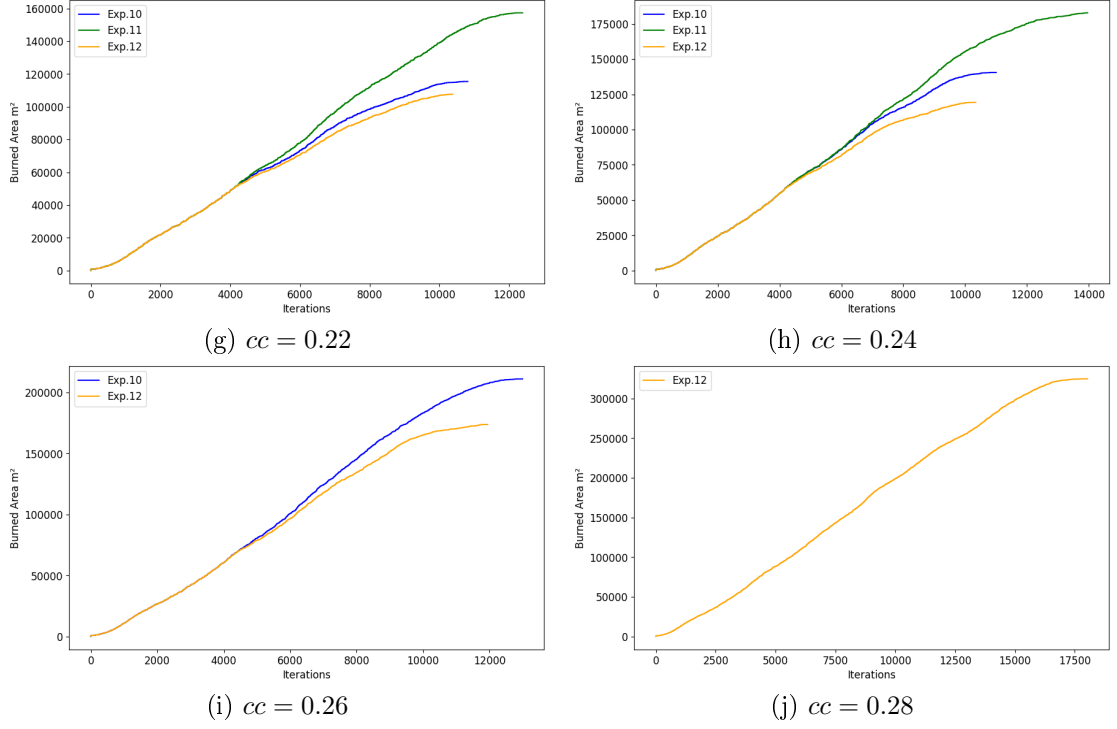
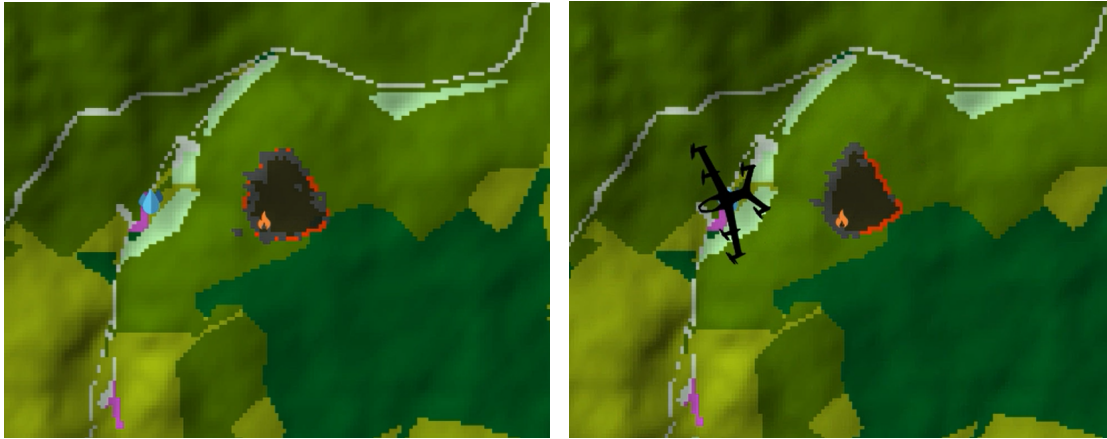


Figure 6.3: **Suppression Reward vs. Connected Suppression Reward.** Performance of Exp.7 (Suppression Reward, no penalties), Exp.10 (Suppression Reward, with penalties), Exp.11 (Connected Suppression Reward, no penalties), and Exp.12 (Connected Suppression Reward, with penalties). The evaluation was performed across increasing fire-spread complexity ($cc = 0.10$ to $cc = 0.28$)

Figure 6.4 shows a spatial snapshot of the suppression state during the same scenario for Exp.10 and Exp.12. In Exp.10 (Figure 6.4a), multiple isolated burning cells remain scattered across the map with no connected suppression region. In Exp.12 (Figure 6.4b), a continuous suppressed region has formed along one fire front, with the remaining burning cells concentrated on the opposite side. The reward configuration of Exp.12 is carried forward as the standard suppression component in all subsequent experiments.



(a) Individual burning cells (Exp.10: Suppression Reward) (b) Continuous suppression line (Exp.12: Connected Suppression Reward)

Figure 6.4: **Behavioral Difference Between Suppression Reward Variants.** Left: Exp.10 (Suppression Reward) leaves isolated burning cells unextinguished. Right: Exp.12 (Connected Suppression Reward) establishes a continuous suppression line.

6.3.3 Phase 3: Observation Space Extension and Optimization

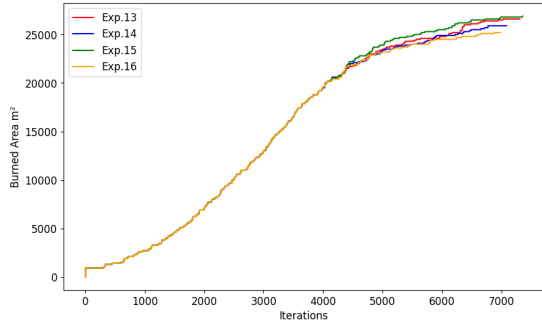
Table 6.6: Phase 3: Observation Space Extension and Optimization

	Exp.13	Exp.14	Exp.15	Exp.16	Exp.17	Exp.18	Exp.19	Exp.20
Observation-Components								
Local-Grid	×	×	×	×	×	×	×	×
N-Closest-Burning	×	×	×	×	×	×	×	×
Last-Suppression	×	×	×	×	×	×	×	×
Ignition-Center			×	×				
Cardinal-Directions		×		×	×	×	×	×
Reward-Components								
Fire-Proximity	×	×	×	×	×	×	×	×
Suppression-Proximity	×	×	×	×	×	×	×	×
Suppression								
Connected-Suppression	×	×	×	×	×	×	×	×
Timestep-Penalty	×	×	×	×	×	×	×	×
Inactivity-Penalty	×	×	×	×	×	×	×	×
Mission-Success								
Mission Successful	✓	✓	✓	✓	✓	✓	✓	✓
Training cc	0.20	0.20	0.20	0.20	0.20	0.20	0.30	0.20-0.30

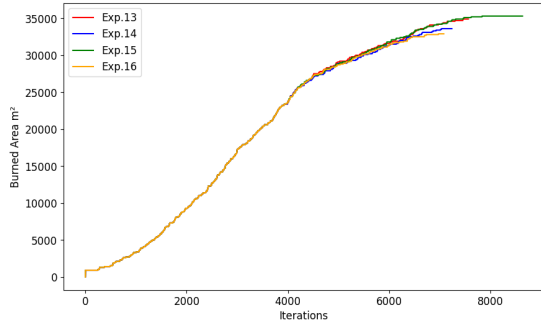
× = Region Size Method, × = Wedge Point Distance Method, × = Graph Diameter Method

Figure 6.5 presents the observation space ablation, evaluating Exp.13, Exp.14, Exp.15, and Exp.16 (Table 6.6). The reward function is fixed at the best-performing configuration from Phase 2. The experiments differ in which additional observation components are included alongside the mandatory *Local Grid* and *N-Closest-Burning* components. The components *Last-Suppression*, *Cardinal-Directions*, and *Ignition-Center* are evaluated individually and in selected combinations. From $cc = 0.10$ to $cc = 0.16$, all four experiments perform comparably with differences below $5,000 \text{ m}^2$. From $cc = 0.18$ onward, Exp.14 separates from the remaining configurations, consistently recording the lowest burned area across all subsequent difficulty levels. At $cc = 0.22$, the performance gap widens, with Exp.14 reaching approximately $100,000 \text{ m}^2$, roughly $35,000 \text{ m}^2$ less than Exp.13 at the same level. At $cc = 0.24$, Exp.16, which incorporates all three additional observation components, yields the highest burned area among the four experiments at approximately $157,000 \text{ m}^2$, followed by Exp.15 ($150,000 \text{ m}^2$) and Exp.13 ($149,000 \text{ m}^2$). Exp.14 achieves the lowest burned area at approximately $133,000 \text{ m}^2$. Exp.15 fails at

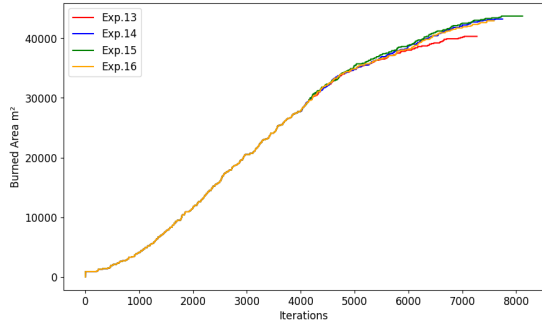
$cc = 0.26$; at this level the gap between Exp.13 (258,000 m²) and Exp.14 (170,000 m²) widens to approximately 88,000 m², with Exp.16 (180,000 m²) falling between the two. At $cc = 0.28$, Exp.13 accumulates approximately 350,000 m² — nearly double Exp.14’s 190,000 m² — while Exp.16 reaches approximately 270,000 m². Exp.16 fails at $cc = 0.30$, where Exp.13 reaches approximately 1,020,000 m² and Exp.14 approximately 410,000 m². Despite reaching the same maximum cc of 0.30, Exp.13 consistently accumulates substantially higher burned area than Exp.14 across all evaluated levels. Exp.14 is therefore carried forward as the observation space configuration for all subsequent experiments.



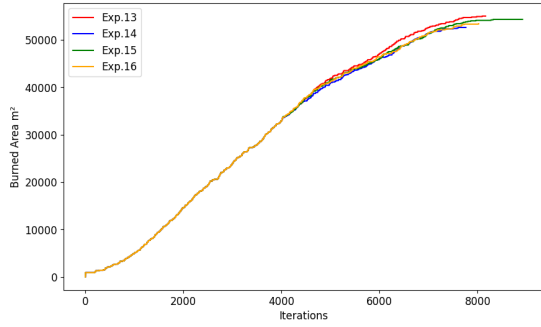
(a) $cc = 0.10$



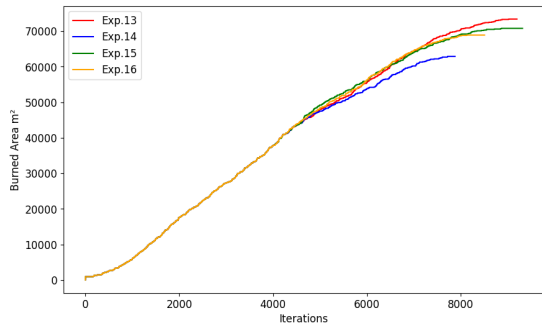
(b) $cc = 0.12$



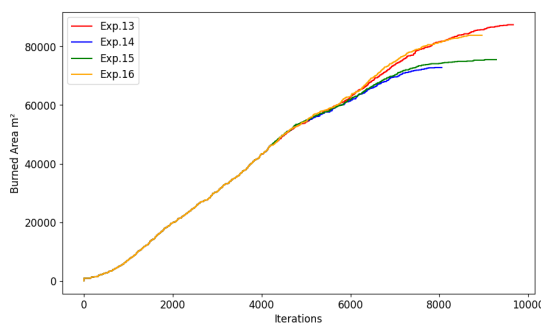
(c) $cc = 0.14$



(d) $cc = 0.16$



(e) $cc = 0.18$



(f) $cc = 0.20$

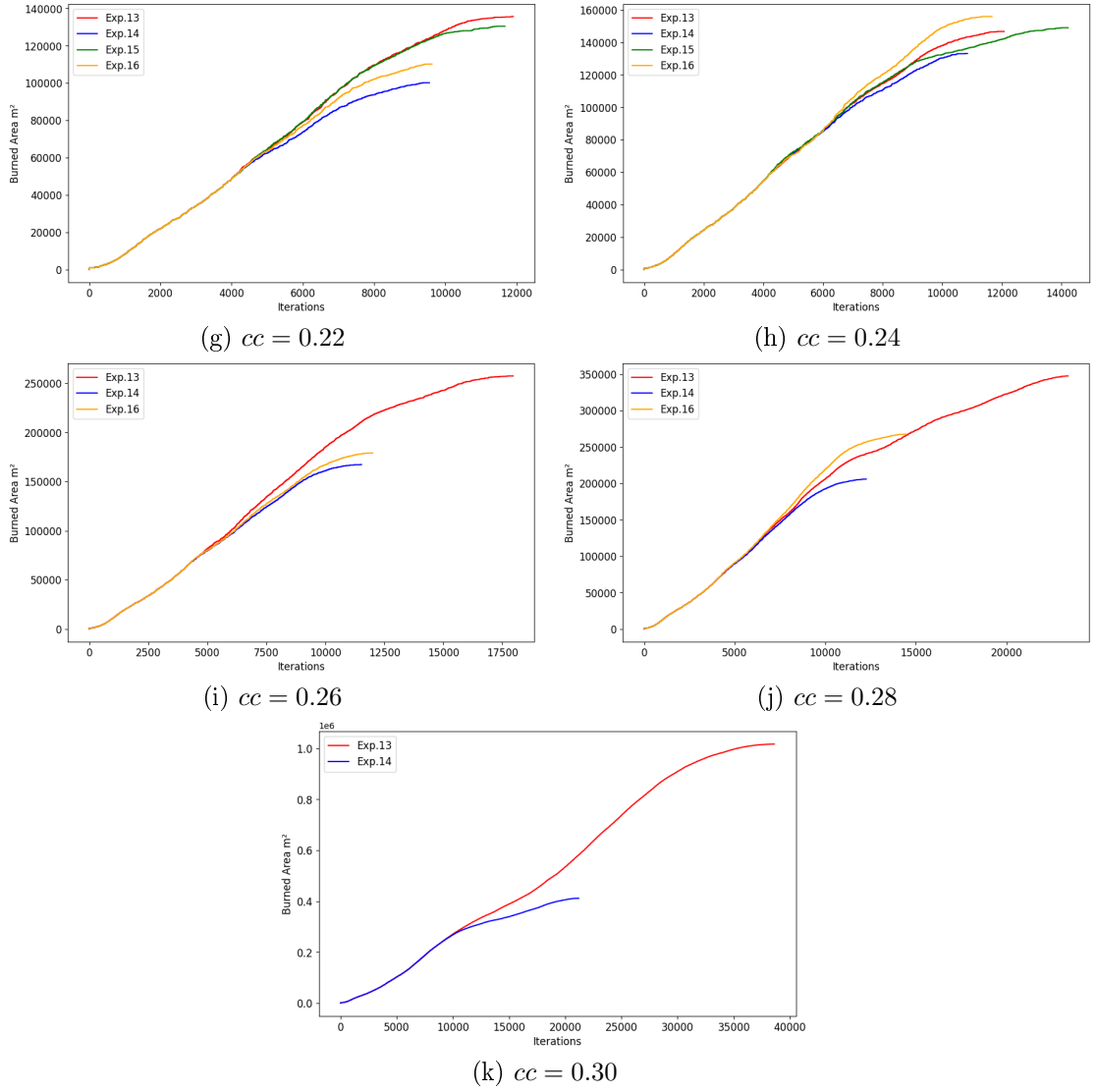
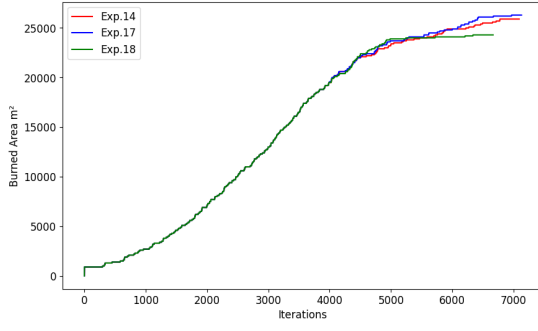
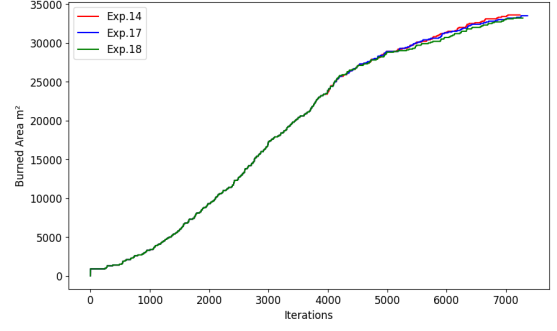


Figure 6.5: **Evaluation of Observation Components.** Performance of Exp.13 (Last-Suppression), Exp.14 (Last-Suppression, Cardinal-Directions), Exp.15 (Last-Suppression, Ignition-Center), and Exp.16 (all three components), evaluated across increasing fire-spread complexity ($cc = 0.10$ to $cc = 0.30$).

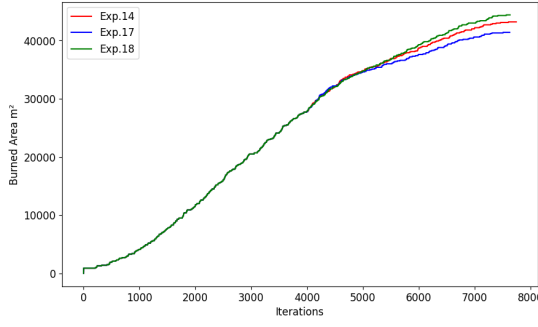
Figure 6.6 evaluates the three reward calculation methods for the *Connected Suppression Reward* introduced in Section 5.1.3 (Table 6.6): the *Region Size Method* (Exp.14), the *Wedge Point Distance Method* (Exp.17), and the *Graph Diameter Method* (Exp.18). All three experiments share the optimal observation space configuration identified in the previous evaluation, combining the mandatory Local-Grid and N-Closest-Burning observations with Last-Suppression and Cardinal-Directions, differing only in the reward calculation method of the *Connected Suppression Reward* component. Exp.14, which achieved the best performance in the previous evaluation, serves as the baseline for this comparison. From $cc = 0.10$ to $cc = 0.22$, all three configurations produce comparable results with differences below $10,000 \text{ m}^2$. From $cc = 0.24$ onward, the gap between the methods grows with each increase of the correction coefficient, with Exp.18 consistently producing the highest burned area while Exp.14 and Exp.17 achieve comparable results. The gap between Exp.18 and both Exp.14 and Exp.17 reaches up to $46,000 \text{ m}^2$ at $cc = 0.28$. At $cc = 0.30$, Exp.14 produces the highest burned area at approximately $410,000 \text{ m}^2$, followed by Exp.18 at approximately $300,000 \text{ m}^2$, while Exp.17 remains lowest at approximately $290,000 \text{ m}^2$. Only Exp.17 and Exp.18 sustain successful suppression at $cc = 0.32$, with Exp.17 achieving approximately $350,000 \text{ m}^2$ and Exp.18 approximately $390,000 \text{ m}^2$. Although Exp.14 achieves a lower burned area than Exp.17 at isolated correction coefficients such as $cc = 0.18$ and $cc = 0.20$, its performance degrades significantly at higher fire intensities, producing a burned area of approximately $410,000 \text{ m}^2$ at $cc = 0.30$ compared to $290,000 \text{ m}^2$ for Exp.17, and failing to suppress the fire entirely at $cc = 0.32$. As Exp.17 produces the lowest burned area across almost all correction coefficients and reaches the same maximum correction coefficient of $cc = 0.32$ as Exp.18, the *Wedge Point Distance Method* is carried forward as the reward calculation method for all subsequent experiments.



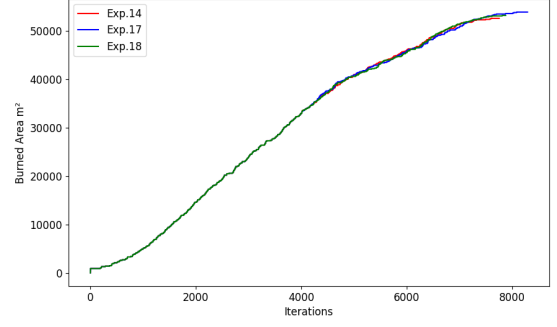
(a) $cc = 0.10$



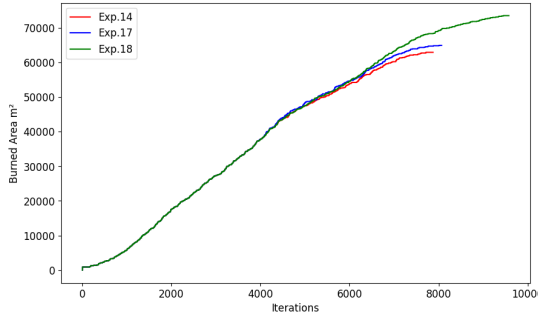
(b) $cc = 0.12$



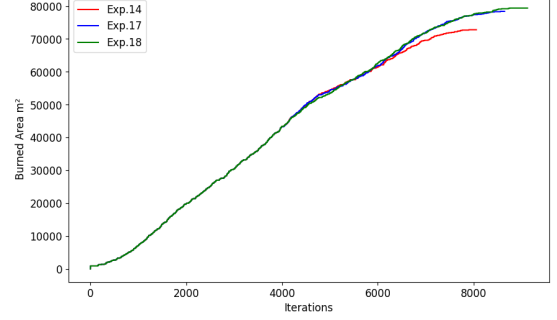
(c) $cc = 0.14$



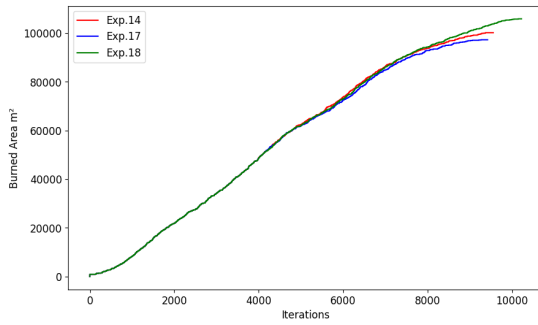
(d) $cc = 0.16$



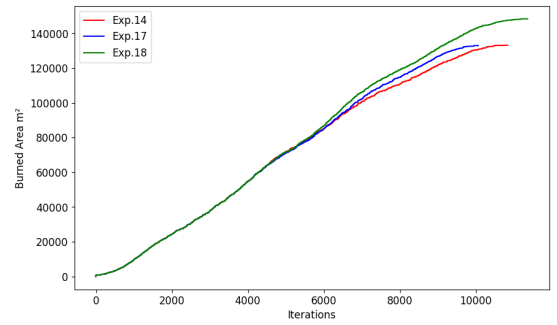
(e) $cc = 0.18$



(f) $cc = 0.20$



(g) $cc = 0.22$



(h) $cc = 0.24$

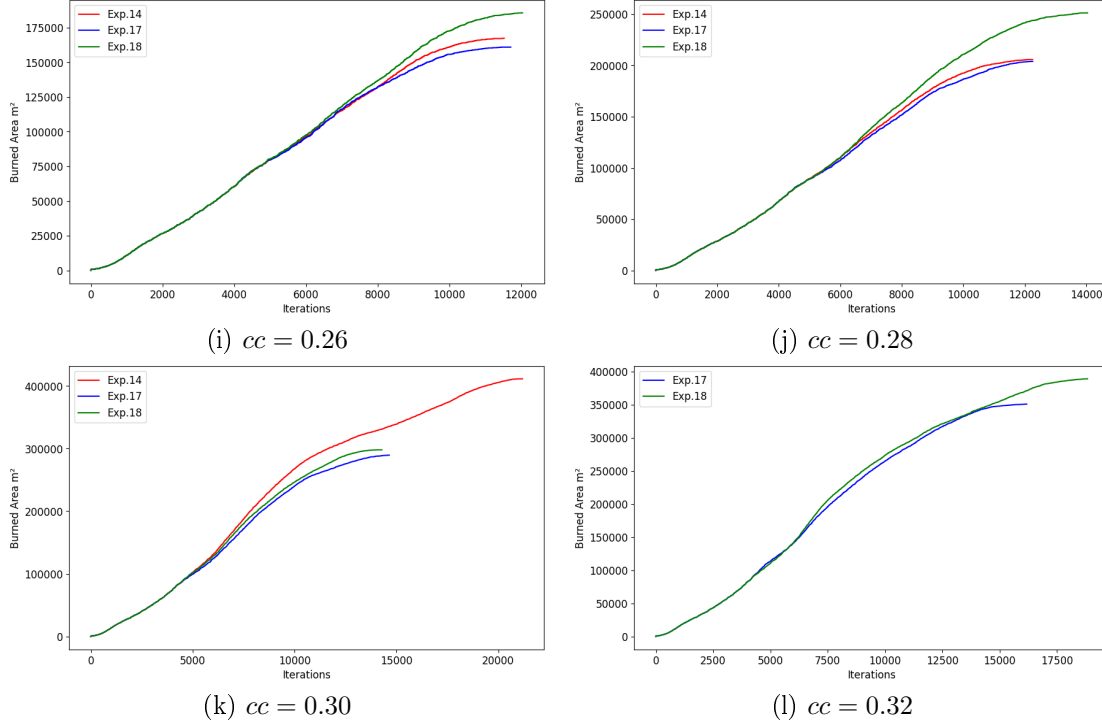
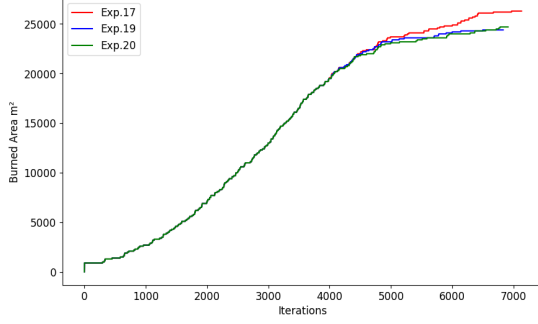
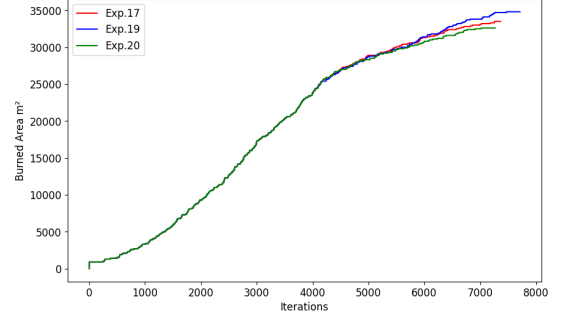
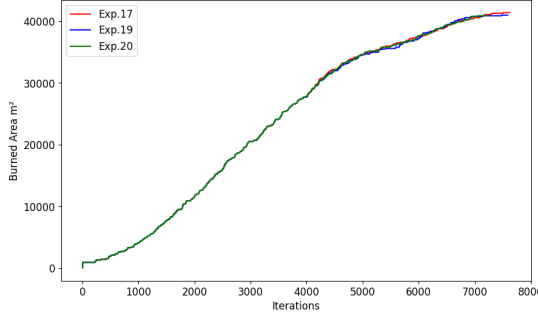
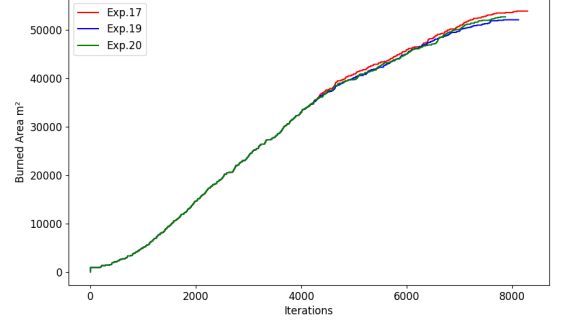
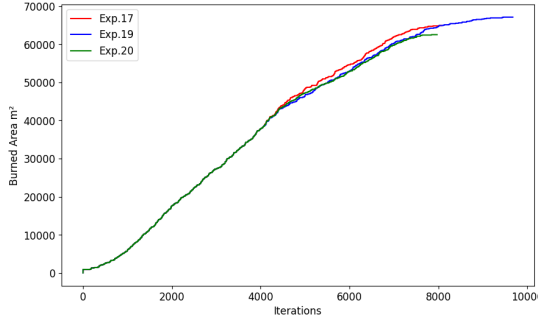
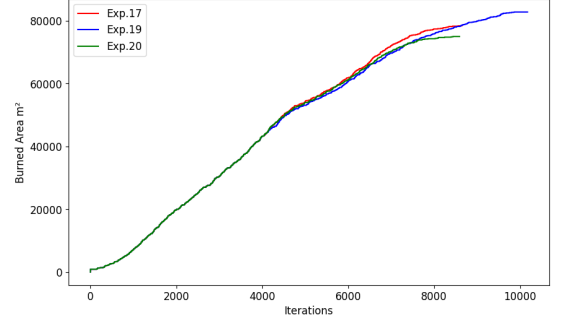


Figure 6.6: **Evaluation of Connected-Suppression Reward Variants.** Performance comparison of the three Connected-Suppression reward calculation methods described in Section 5.1.3: the **Region Size Method**, **Graph Diameter Method** and the **Wedge Point Distance Method**. The evaluation was performed across increasing fire-spread complexity ($cc = 0.10$ to $cc = 0.32$).

Exp.17 emerged as the best-performing policy from the preceding experiments. To further improve performance, two approaches were evaluated (Table 6.6). The first increases the training cc from 0.20 to 0.30 (Exp.19). The second approach applies curriculum learning as described in Section 2.1.7, starting at $cc = 0.20$ and incrementally increasing complexity by 0.02 up to 0.30 (Exp.20). As shown in Figure 6.7, from a correction coefficient of 0.10 to 0.20, all three experiments produce comparable results, with a difference in burned area below 10,000 m². At $cc = 0.22$, Exp.19 begins to diverge, with a gap of approximately 15,000 m² between Exp.19 and Exp.20, a difference that continues to grow at higher correction coefficients. At $cc = 0.24$, Exp.20 achieves the lowest burned area at approximately 120,000 m², while Exp.17 reaches approximately 134,000 m² and Exp.19 approximately 160,000 m². Exp.19 fails at $cc = 0.26$, where Exp.17 and Exp.20 both reach approximately 160,000 m². From $cc = 0.28$ onward, only Exp.17 and Exp.20 remain. The gap between them grows with fire intensity: approximately 18,000 m² at

$cc = 0.28$, $60,000 \text{ m}^2$ at $cc = 0.30$, and $95,000 \text{ m}^2$ at $cc = 0.32$, where Exp.17 reaches approximately $350,000 \text{ m}^2$ and Exp.20 approximately $255,000 \text{ m}^2$. Exp.17 fails at $cc = 0.34$, where Exp.20 remains the only configuration to successfully suppress the fire, reaching approximately $610,000 \text{ m}^2$.

(a) $cc = 0.10$ (b) $cc = 0.12$ (c) $cc = 0.14$ (d) $cc = 0.16$ (e) $cc = 0.18$ (f) $cc = 0.20$

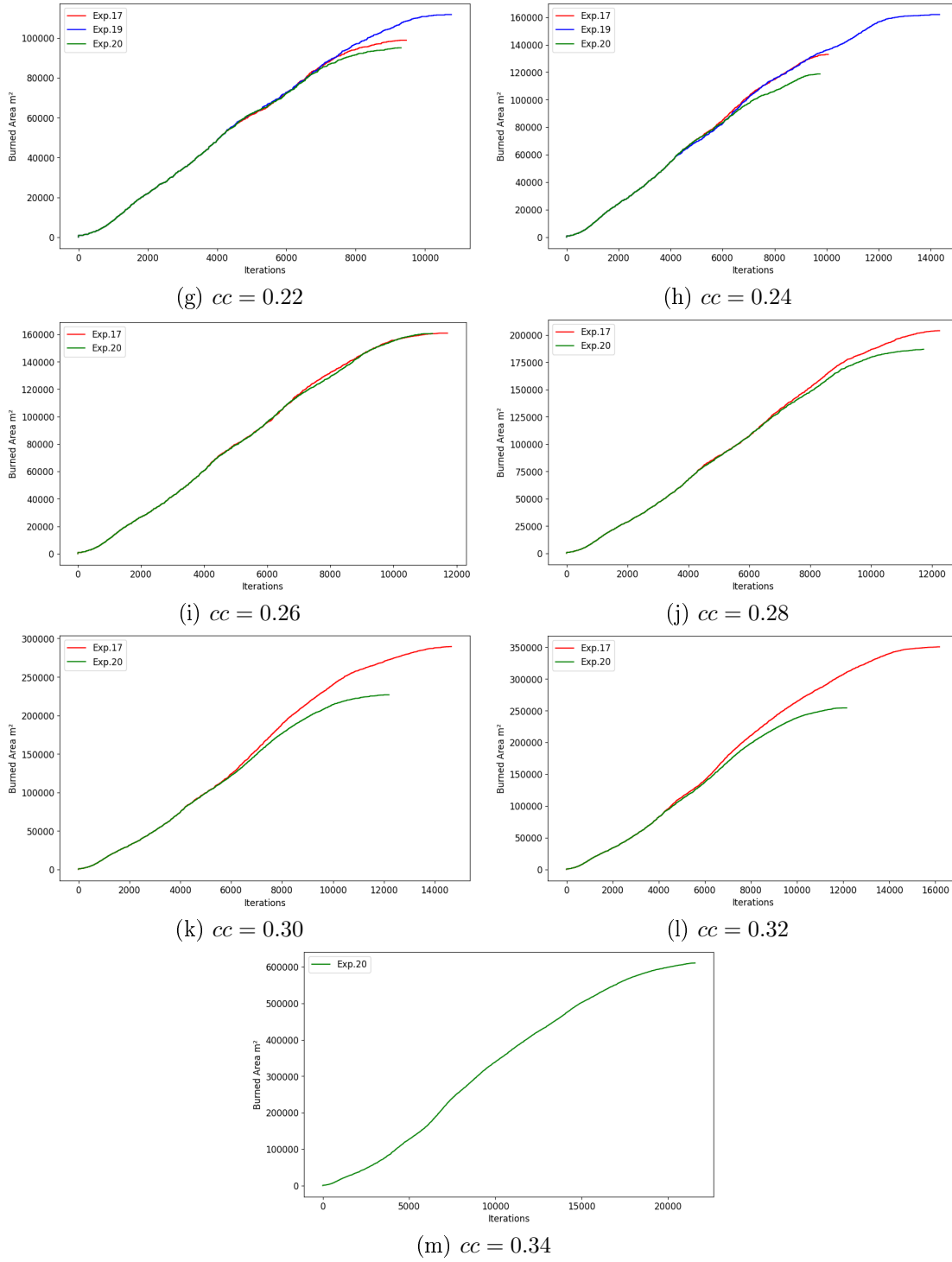
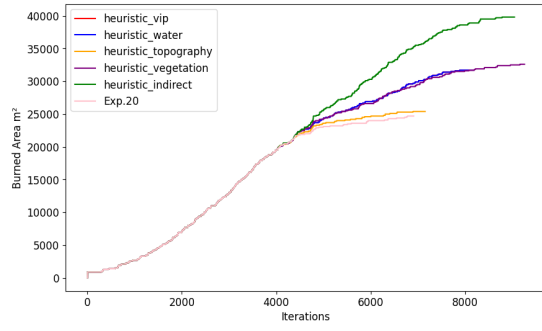


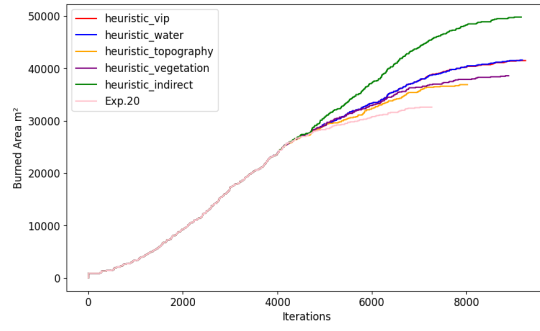
Figure 6.7: **Evaluation of the Optimization Experiments.** Performance comparison of the two optimization experiments Exp.19 and Exp.20 compared with the best performing policy without any optimization (Exp.17). The evaluation was performed across increasing fire-spread complexity ($cc = 0.10$ to $cc = 0.34$).

6.4 RL–Heuristic Comparison

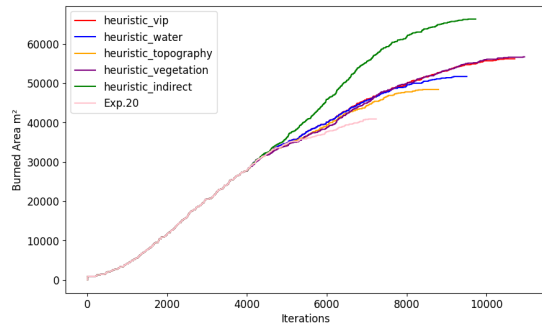
To evaluate the effectiveness of the *RL policy* itself, the heuristic tactics described in Section 2.2.5 are used as a baseline, specifically, the four direct suppression tactics (water, vip, topography, and vegetation) and the indirect attack. Figure 6.8 compares the burned area over simulation timesteps for Exp.20, the best-performing *RL policy* from the ablation study, against the heuristic tactics across correction coefficients from $cc = 0.10$ to $cc = 0.36$. At all evaluated fire intensity levels, the *RL policy* produces lower final burned area than all direct heuristic tactics. The *topography heuristic tactic* successfully suppresses the wildfire up to a cc of 0.22, the *vegetation heuristic tactic* up to a cc of 0.24, and the water and *vip heuristic tactics* up to a cc of 0.34, matching the upper complexity limit of the RL policy. However, at all shared complexity levels the *RL policy* consistently produces lower burned area than both the water and vip tactics. At $cc = 0.34$, for example, the *RL policy* achieves approximately 610,000 m², while the water and vip tactics each reach approximately 1,450,000 m². At low fire intensity levels, the *indirect tactic* produces the highest burned area of all evaluated tactics. As fire intensity increases, its relative performance improves and it outperforms the remaining direct heuristics at higher complexity levels. The *indirect tactic* achieves successful suppression up to $cc = 0.36$. In the comparison between the *RL policy* and the *indirect tactic*, the *RL policy* consistently produces lower burned area at all complexity levels up to and including $cc = 0.34$: at $cc = 0.26$, Exp.20 achieves approximately 160,000 m² compared to 280,000 m² for the indirect tactic, and at $cc = 0.34$, approximately 610,000 m² compared to 900,000 m². At $cc = 0.36$, the *RL policy* fails to suppress the wildfire, whereas the *indirect tactic* succeeds, recording approximately 1,200,000 m².



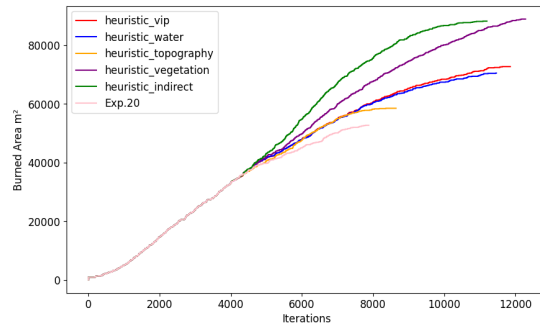
(a) $cc = 0.10$



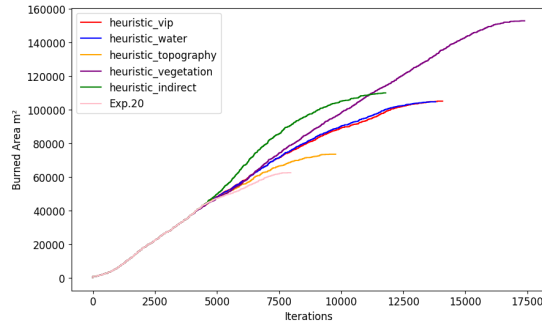
(b) $cc = 0.12$



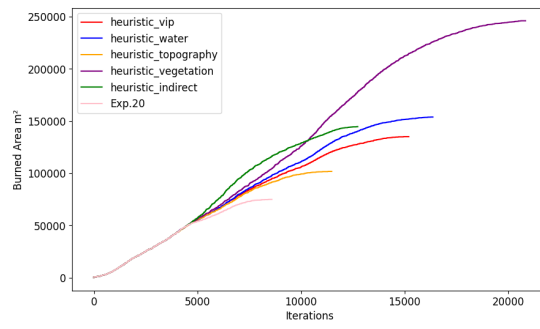
(c) $cc = 0.14$



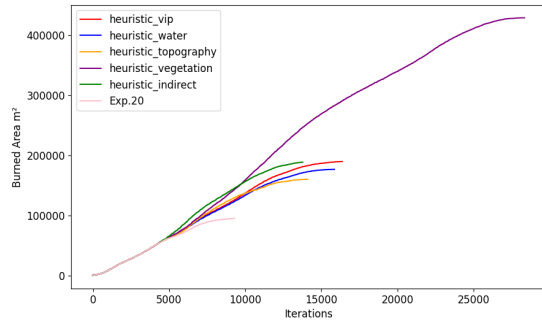
(d) $cc = 0.16$



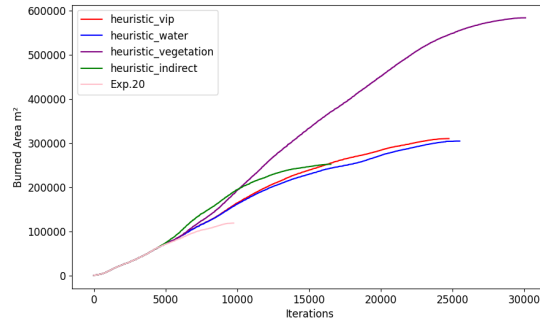
(e) $cc = 0.18$



(f) $cc = 0.20$



(g) $cc = 0.22$



(h) $cc = 0.24$

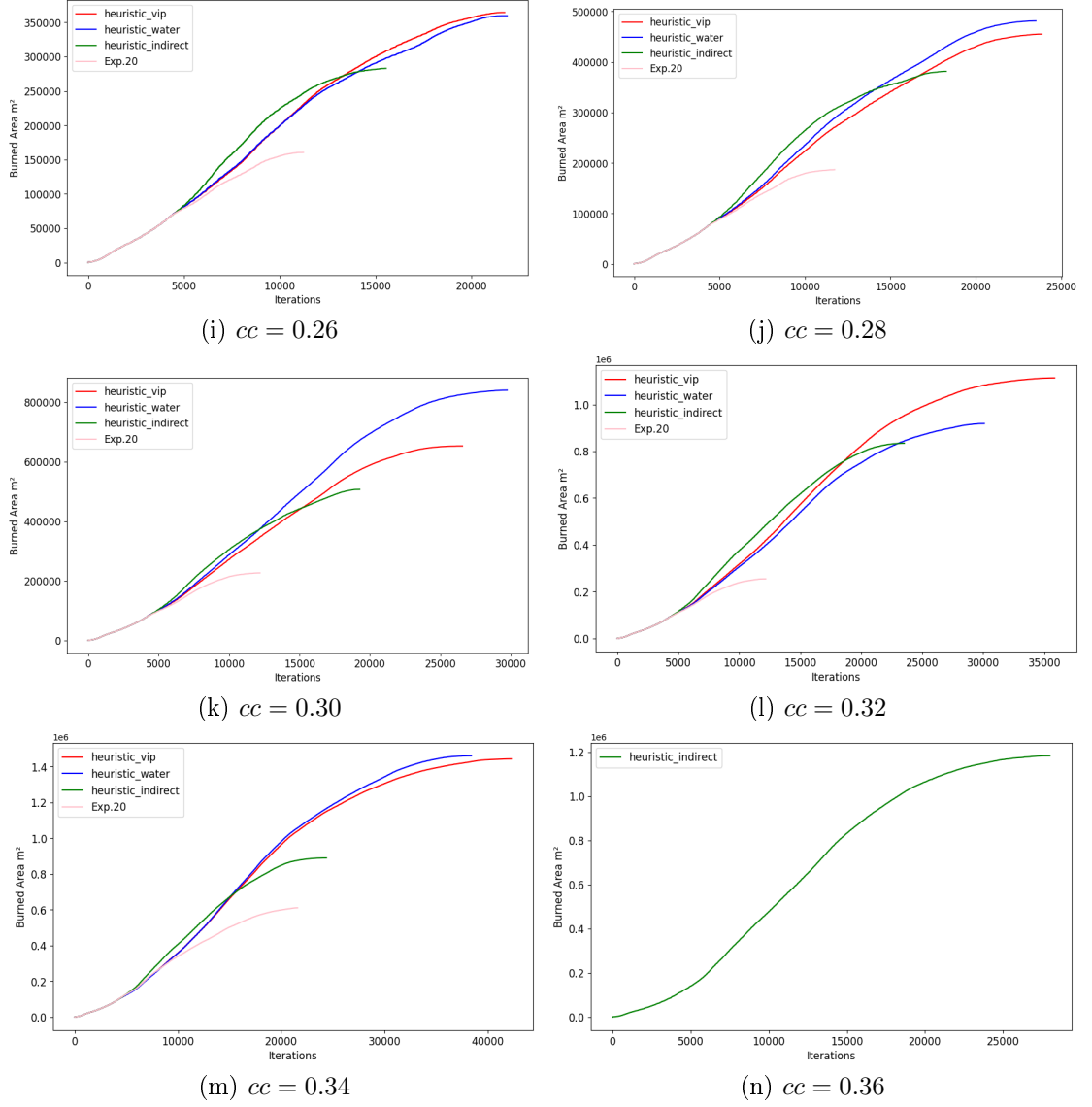


Figure 6.8: **RL-Heuristic Comparison.** Performance comparison of the best performing rl policy (Exp.20) with the heuristic tactics. The evaluation was performed across increasing fire-spread complexity ($cc = 0.10$ to $cc = 0.36$).

7 Discussion

This chapter interprets the experimental results presented in Chapter 6, examining the contribution of individual design choices to agent performance and discussing the behavioral characteristics of the learned policy. The discussion is structured around the three research questions of this thesis.

7.1 RQ1 – Systematic Complexity Structuring

How can the complexity of Reinforcement Learning integration into an agent-based wildfire simulation be systematically structured to enable effective learning of suppression strategies?

The integration of RL into the wildfire simulation was structured through a staged complexity framework spanning four levels. Starting from a minimal single-agent configuration with the sole objective of locating the fire, complexity was increased by extending the setup to multi-agent, progressively refining the observation space and reward function, specializing the mission objective from fire location to active suppression, and deferring operational constraints such as refueling and resupply to a later stage. Deferring these constraints allowed the training pipeline to be established and validated incrementally, making it possible to verify basic suppression behavior before introducing further complexity.

The experiments further demonstrate that systematic complexity structuring is also essential during training. Two optimization strategies are evaluated to improve upon the baseline policy Exp.17: training at a higher fixed correction coefficient and curriculum learning. Fixing the training difficulty at $cc = 0.30$ (Exp.19) degrades performance relative to Exp.17, which was trained at $cc = 0.20$. At $cc = 0.24$, Exp.19 already produces approximately 26,000 m² more burned area than Exp.17, and fails at $cc = 0.26$ — while

Exp.17 successfully suppresses the wildfire up to $cc = 0.32$. Simply exposing agents to more complex fires does not improve performance; on the contrary, fire spread at $cc = 0.30$ appears too complex for agents to develop an effective suppression strategy from scratch, leaving them unable to perform well even at less demanding fire intensities. Curriculum learning (Exp.20) addresses this by starting training at $cc = 0.20$ and advancing to the next complexity level only once agents demonstrate stable suppression behavior. Complexity is increased iteratively in steps of 0.02 up to a target of $cc = 0.30$, allowing agents to consolidate suppression skills at each stage before facing more dynamic fire spread. This approach outperforms Exp.17 from $cc = 0.28$ onward, with the performance advantage growing from approximately 18,000 m² at $cc = 0.28$ to approximately 95,000 m² at $cc = 0.32$, and extends the maximum achievable correction coefficient to $cc = 0.34$, one level beyond Exp.17.

7.1.1 RL Integration as a Validation Tool

Beyond the training results, the RL integration proved valuable as a simulation validation tool, uncovering an issue that had not been apparent through rule-based tactics alone. During training, a state transition inconsistency was identified in the simulation: suppression actions applied to already burnt cells caused them to transition from *Burnt* to *Suppressed*, artificially reducing the burned area metric. This issue was exposed through the broader state space coverage achieved by RL training and was subsequently corrected. The finding illustrates that integrating RL into a complex simulation can reveal latent implementation issues that rule-based approaches may never encounter, providing an additional benefit of the systematic integration process.

7.2 RQ2 – Modular Decomposition

How does modular decomposition of reward and observation space components affect the behavior and interpretability of learned suppression policies?

7.2.1 Phase 1: Observation Space Baseline

The results of Phase 1 demonstrate that the *N-Closest-Burning* observation component is a necessary condition for successful wildfire suppression in the evaluated configuration. None of the configurations that rely solely on the *Local-Grid*, together with *Cardinal-Directions*, *Ignition-Center* or last suppression information, produced a policy capable of suppressing the wildfire. The local grid provides only immediate spatial context, which is insufficient for navigating toward the fire front in a large environment. The ignition center direction and cardinal direction vectors convey the spatial extent and origin of the fire but are not enough to learn a successful suppression tactic. Also the *Last-Suppression* component, either alone or in combination with the other observation components, did not lead to a successful policy. The *N-Closest-Burning* component, by contrast, gives agents explicit directional information toward actively burning targets, enabling reliable navigation across a variety of fire configurations. This finding indicates that a direct, proximity-based navigational signal toward burning cells is essential for the agents to locate and approach the fire front and learn a successful suppression strategy. It should be noted that only $N = 1$ was evaluated throughout this work. The effect of providing agents with directions toward more than one burning cell simultaneously, and whether it improves navigation or coordination, remains an open question.

7.2.2 Phase 2: Reward Function Ablation

Penalty Components

The comparison of penalty configurations in Phase 2 shows the advantage of the penalty components against Exp.7 using no penalty components. First, the effect of *Inactivity-Penalty*, used in Exp.9, is shown by outperforming Exp.7 in every fire intensity and by achieving a higher cc value at which the fire is successfully extinguished. Thus the *Inactivity-Penalty* encourages the agent to suppress the fire more efficiently by penalizing

inactive behavior around the fire. Exp.8 shows that the *Timestep-Penalty* is even more effective and achieves an even lower burned area than Exp.9, leading to the conclusion that the *Timestep-Penalty* promotes more efficient policies than the *Inactivity-Penalty* together with the *Mission-Success* reward. Unlike the *Timestep-Penalty*, the *Mission-Success* reward returns the same fixed value regardless of how quickly the fire is suppressed. A policy that extinguishes the fire in 200 timesteps receives the same success reward as one that takes 300, providing no incentive for speed. The *Timestep-Penalty* resolves this by accumulating over the episode: the faster the suppression, the lower the cumulative penalty and the higher the total reward, directly encouraging efficient behavior. The combination of the *Timestep-Penalty* and the *Inactivity-Penalty* yields the highest performance overall, as demonstrated by Exp.10. At lower fire intensities, Exp.8 and Exp.10 produce comparable burned areas, with Exp.8 occasionally achieving lower values. The advantage of Exp.10 lies in its ability to suppress the wildfire at higher correction coefficients, reaching $cc = 0.26$ compared to $cc = 0.24$ for Exp.8. The *Timestep-Penalty* creates a continuous incentive to minimize episode length, rewarding policies that suppress the fire efficiently. The *Inactivity-Penalty* specifically discourages agents from remaining near the fire without acting, addressing a common failure mode in which agents approach the fire front but do not commit to suppression. In theory, the *Timestep-Penalty* component should be enough as this component indirectly penalizes inactive behavior, making the *Inactivity-Penalty* obsolete. The advantage of the combination of these components could result from the training time of 5,000 iterations. Longer training might bring a similar result when not using the *Inactivity-Penalty* component. In the early training episodes this component might have helped avoiding the inactivity behavior, thereby accelerating convergence.

Suppression Reward Variants

The *Connected Suppression Reward* consistently outperforms the standard *Suppression Reward*, both with and without penalty components. The spatial connectivity constraint embedded in the *Connected Suppression Reward* directs agents toward positions adjacent to existing suppressed regions, naturally encouraging the formation of a continuous containment line. As shown in Figure 6.4, the *Suppression Reward* leads to isolated cell-by-cell extinguishment with no spatial coherence, whereas the *Connected Suppression Reward* produces a structured suppression front that advances systematically across the fire perimeter. At higher fire intensities, a continuous firebreak is more effective at

reducing total burned area, as individual unsuppressed cells develop into new fire fronts more quickly. For this reason, Exp.12 is capable of extinguishing the fire up to a cc of 0.28, while Exp.10 can only do so up to 0.26.

7.2.3 Phase 3: Observation Space Extension

Additional Observation Components

The results from the observation space extension show that both configurations using the *Ignition-Center* observation component (Exp.15 and Exp.16) underperform relative to the configurations without it. *Exp.15*, which combines *Last-Suppression* and *Ignition-Center*, fails earliest with its last successful suppression at $cc = 0.24$. Exp.13, which uses only *Last-Suppression*, produces comparable burned areas at shared fire complexity levels but successfully suppresses the wildfire up to $cc = 0.30$ — three levels past the last successful suppression of Exp.15. *Exp.16*, which incorporates all three additional observation components, surpasses Exp.15's maximum cc and outperforms Exp.13 in burned area at $cc = 0.26$ and $cc = 0.28$, yet still fails to suppress the wildfire at $cc = 0.30$ — one level before Exp.13 and Exp.14. Despite having access to the most observation information of all four configurations, adding the *Ignition-Center* does not improve performance and reduces the maximum achievable correction coefficient relative to configurations without it. The *Ignition-Center* direction offers a fixed spatial reference that does not update as the fire evolves, making it less informative as the fire front moves away from the ignition point, which likely explains its counterproductive effect in both configurations.

The *Last-Suppression* and *Cardinal-Directions* components, used together in Exp.14, provide the most effective combination. The *Last-Suppression* direction guides agents toward the most recently active suppression point, helping to maintain continuity along the suppression front and preventing agents from dispersing to isolated burning cells. Including direction vectors toward the burning cells that have spread farthest in each of the four cardinal directions allows agents to perceive the overall spatial distribution of the fire, supporting more globally aware suppression decisions. This combination enables Exp.14 to suppress the wildfire with approximately 610,000 m² less burned area than Exp.13 at $cc = 0.30$, while both reach the same maximum correction coefficient.

Connected Suppression Reward Variants

Among the three reward calculation methods for the *Connected Suppression Reward*, the *Wedge Point Distance Method* (Exp.17) achieves the best performance, while the *Region Size Method* (Exp.14) fails at a lower correction coefficient than Exp.17 and Exp.18. The *Region Size Method* rewards agents for increasing the total suppressed area in any direction, providing no incentive to extend the suppression line toward the advancing fire front. The *Wedge Point Distance Method*, by contrast, produces a more directionally sensitive reward signal, encouraging agents to extend the suppression line in a specific direction rather than simply growing the suppressed region. The *Graph Diameter Method* (Exp.18) achieves comparable performance to Exp.17 at most fire intensities, since maximizing the longest path through the suppressed region also implicitly rewards directional extension, producing similar behavioral effects.

However, the graph diameter does not distinguish between extension toward and away from the fire. An agent can maximize the diameter with a long line that does not intercept the fire front. The *Wedge Point Distance Method* avoids this by measuring the angular gap in the containment perimeter relative to the ignition center, which more directly rewards closing the perimeter around the fire. One limitation of the Wedge Point Distance Method is that as suppression nears completion, the angular gap approaches zero and the reward signal weakens. However, this occurs primarily in the final phase of an episode when the outcome is largely determined, so the impact on overall policy learning is limited. The superior alignment of the *Wedge Point Distance Method* with the containment objective throughout the majority of the episode likely explains why it outperforms the *Graph Diameter Method* despite this late-episode limitation. Notably, the *Region Size Method* (Exp.14) achieves the lowest burned area at several individual complexity levels, including $cc = 0.18$ and $cc = 0.20$. The relatively comparable performance across all three methods can be partly attributed to their shared underlying condition: a reward is only issued when a burning cell is actually extinguished. The choice of calculation method therefore shapes the incentive structure but does not fundamentally alter the suppression mechanism, which limits the magnitude of the differences while keeping them measurable, as visible in the evaluation figures.

7.3 RQ3 – RL vs. Heuristic Comparison

Can multi-agent RL learn suppression strategies that match or exceed the performance of established heuristics in an operationally grounded wildfire simulation?

The RL policy consistently outperforms the direct heuristic tactics across all evaluated fire intensities. At lower correction coefficients the performance difference remains moderate, but the gap widens as fire complexity increases — at $cc = 0.34$, the direct heuristics reach approximately 1,450,000 m² of burned area while Exp.20 achieves approximately 610,000 m². Unlike the rule-based direct tactics, which apply a fixed agent-to-target assignment without adapting to the current fire state, the RL policy responds to the evolving fire configuration and distributes suppression effort more effectively across the fire perimeter.

The indirect tactic presents a different performance profile: it produces the highest burned area among all evaluated tactics at low fire intensities, but its relative performance improves as fire complexity grows. By establishing a containment perimeter rather than directly attacking the fire front, it avoids the growing fire core and concentrates suppression resources along the perimeter boundary, which remains accessible even at high fire intensities. This advantage only becomes decisive under more complex fire conditions. The RL policy fails to suppress the wildfire at $cc = 0.36$, whereas the indirect tactic achieves successful containment at this level — making it the only evaluated approach to outperform the RL policy. Since the reward function used in Exp.20 does not promote indirect suppression behavior, the RL policy cannot develop this strategy through training; perimeter containment is structural to the indirect tactic’s design. The advantage of the indirect tactic at high fire intensities therefore reflects a difference in suppression approach rather than a fundamental limitation of RL — a reward function that explicitly promotes perimeter containment could potentially extend the effective range of the learned policy to higher fire intensities.

It should also be noted that burned area is the only evaluation metric used in this comparison. Under different criteria, such as the reduction of civilian casualties, CO₂ emissions, or energy consumption, the ranking of tactics could look different. Heuristic approaches, for example, might achieve similar suppression outcomes with lower resource expenditure, depending on how flight efficiency and agent utilization are weighted.

7.3.1 Emergent Tactic Analysis

A qualitative comparison between the learned RL policy and the heuristic suppression tactics reveals systematic differences in agent behavior. Figure 7.1 shows the frequency distribution of cells suppressed per action for both approaches. The heuristic performs considerably more suppression actions overall, with 497 compared to 150 for the RL policy. Of the heuristic’s 497 suppression actions, 121 ($\approx 24\%$) yield zero suppressed cells, whereas the RL agent never suppresses zero cells in a single action. This is because RL agents receive a positive reward only when at least one burning cell is successfully suppressed, while an action that yields no suppression results in a penalty. These zero-cell suppressions contribute to a lower mean of approximately 1.4 cells suppressed per action for the heuristic, compared to approximately 2.2 for the RL policy. While zero-cell suppressions can theoretically serve as a preventive firebreak when placed ahead of the advancing fire front, the considerably higher total number of suppression actions performed by the heuristic suggests that most of these actions are ineffective. This targeting inefficiency was identified through this analysis and has since been corrected in the heuristic implementation.

The avoidance of zero-cell suppressions is a structural property of the learned policy, shaped directly by the reward signal rather than by an awareness of the heuristic bug. The RL policy’s per-action efficiency advantage (2.2 vs. 1.4 cells per action) therefore reflects a genuine behavioral difference rather than an artifact of the heuristic’s targeting error. Nevertheless, a direct comparison between the RL policy and the corrected heuristic implementation was not conducted within the scope of this work. The burned area metric captures the actual suppression outcome and therefore remains a valid basis for the comparison presented here; the heuristic’s zero-cell actions represent wasted effort that does not directly reduce the burned area.

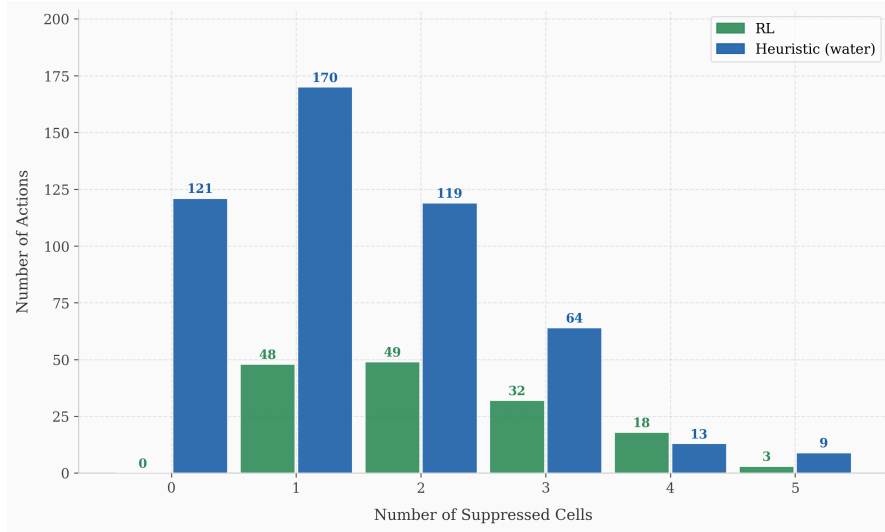


Figure 7.1: Frequency distribution of suppressed cells per action for the RL agent and the Heuristic approach.

Another difference is the selection of fire fronts. Agents controlled by the learned policy predominantly prioritize the suppression of the nearest actively burning cells, rather than adhering to a fixed fire front-selection strategy. In the evaluated scenario, the water source closest to the wildfire is located to the west of the ignition point. When approaching the wildfire from the east, agents following the learned policy initially suppress the eastern fire front before proceeding toward the water source for resupply. Afterwards, the agents again target the nearest burning cells, which are then located along the western fire front. As a result, the eastern fire front is addressed primarily during initial deployment from the airport or after refueling.

The terrain east of the ignition center contains a slope that accelerates fire propagation in that direction. Although suppression efforts on the eastern fire front are limited, they appear sufficient to slow the eastward spread of the wildfire. This behavior is illustrated in Figure 7.2.



Figure 7.2: Snapshot of the emergent suppression tactic. The active fire front (red) is concentrated on the eastern perimeter, where limited initial suppression has slowed the eastward spread. The suppression line (white) is being built along the northern side while agents return to the water source (left) for resupply.

These observations indicate a potential direction for improving heuristic suppression tactics. The emergent behavior of the RL policy bears resemblance to a combination of the topography and water heuristic tactics: terrain and vegetation features that accelerate fire spread — such as slope or combustibility — could be analyzed prior to deployment to identify critical fire fronts. A small number of targeted suppressions at these key locations during the initial phase could slow fire propagation in high-risk directions, after which the strategy could transition to a proximity-based approach such as the water tactic to contain the remaining fire front. Such a two-phase hybrid strategy would combine the terrain awareness of the topography heuristic with the operational efficiency of the water heuristic, and could potentially close the performance gap between rule-based tactics and the learned RL policy.

7.4 Limitations

Several methodological limitations of this work should be acknowledged. These limitations do not invalidate the reported findings but should be considered when interpreting the results.

Single-run evaluation. All reported results are based on single training runs per configuration. Due to the stochastic nature of PPO training results may differ across runs. Replication across multiple seeds was not feasible within the available time and computational budget, given that a single training run required approximately 24–30 hours.

Selective configuration evaluation. When selecting the experimental configurations, not every combination was examined; instead, the selection was based on the preliminary results obtained during the ongoing experiments. While this approach was necessary to manage the combinatorial space within the available resources, it introduces a risk of confirmation bias, as configurations that appeared less promising based on early training dynamics were not pursued.

Absence of hyperparameter tuning. Systematic hyperparameter tuning was not performed. The hyperparameters used across all configurations were held constant and selected based on the RLlib documentation and preliminary studies. This includes the weights assigned to individual reward components, which were selected empirically rather than through systematic optimization. It cannot be excluded that configurations classified as underperforming may achieve comparable or superior results under different hyperparameter settings. The component comparisons presented in Chapter 6 should therefore be interpreted as indicative rather than conclusive.

Single-scenario evaluation. All experiments were conducted exclusively on the Sintra wildfire scenario. The extent to which the learned policies generalize to other geographic regions, fire configurations, or fleet compositions remains an open question and constitutes a direction for future work.

8 Conclusion

This thesis investigated the application of multi-agent reinforcement learning to aerial wildfire suppression within an operationally grounded agent-based simulation of the SoSID Toolkit. The results demonstrate that a systematically structured complexity framework, applied both during implementation and training, is essential for learning effective suppression policies. The staged complexity level design simplified implementation validation, making it possible to verify basic suppression behavior before introducing further complexity. The same principle was applied during training through curriculum learning. While direct training under high fire intensity impaired performance, curriculum-based training, in which the complexity of the fire was gradually increased, improved the effectiveness of the firefighting strategy.

The Dual Decomposition Framework structures the reward function and observation space as modular, independently configurable components, enabling a systematic ablation study of their individual contributions to agent behavior. By isolating each component, the framework makes the effect of individual design decisions directly measurable and interpretable, a key advantage when developing suppression policies in a complex multi-agent environment. The *N-Closest-Burning* observation component was found to be a necessary condition for successful suppression, providing agents with the directional signal required to locate the fire front. Among the reward components, the *Timestep-Penalty* and the *Connected Suppression Reward* were identified as the most impactful, consistently improving suppression performance across all evaluated configurations by encouraging efficient episode completion and the formation of a structured containment line.

Beyond the ablation results, the RL integration yielded an additional finding: a state transition inconsistency in the simulation was uncovered during training that had remained undetected under rule-based tactics alone. Suppression actions applied to already burnt cells caused them to transition to a suppressed state, artificially reducing

the measured burned area. This issue was exposed through the broader state space coverage achieved by RL exploration and was subsequently corrected, illustrating the value of RL as a simulation validation tool independently of its suppression performance.

The learned RL policy consistently outperforms direct heuristic tactics across all evaluated fire intensities and exhibits more targeted suppression behavior, averaging approximately 2.2 suppressed cells per action compared to 1.4 for the heuristic approach. Analysis of the learned policy further reveals an emergent suppression strategy in which agents prioritize the nearest fire front and systematically extend a containment line, implicitly adapting to terrain features without explicit programming. The indirect attack tactic outperforms RL only at very high fire spread rates, reflecting a structural difference in suppression strategy rather than a fundamental limitation of the RL approach. Overall, the results demonstrate the effectiveness of multi-agent RL for aerial wildfire suppression and establish a systematic methodology for reward and observation space design in this domain.

8.1 Future Work

The observation space and reward function can be extended and further analyzed using the Dual Decomposition Framework, which allows individual components to be isolated and evaluated independently. Additional observation components could be integrated to provide agents with more comprehensive state information, and continued reward shaping could further refine the incentive structure. It also remains an open question whether the *Timestep-Penalty* alone would make the *Connected Suppression Reward* and the *Inactivity-Penalty* obsolete given sufficient training time; due to limited computational resources, these components were found to accelerate convergence, and longer training runs could clarify their individual necessity. Additionally, a systematic hyperparameter search could further improve policy performance, as the configurations used in this work were not exhaustively optimized. Exploring Centralized Training with Decentralized Execution (CTDE) could also improve the training process, as it leverages global state information during training to improve agent coordination while maintaining decentralized execution during deployment. Future evaluations should also incorporate broader performance metrics to enable a more comprehensive assessment of suppression tactics beyond burned area alone. These metrics could include the reduction of civilian casualties, CO₂ emissions, or energy consumption.

Replacing the shared policy with individual agent policies would allow different policy networks per aircraft type, enabling effective training with a heterogeneous fleet and exploration of the fleet-composition space to identify optimal fleet sizes and combinations for different wildfire scenarios. A centralized commander architecture, in which a single agent directs the behavior of all firefighting agents, represents a further alternative to the current decentralized approach. Extending the scenario beyond the Sintra region to one that includes seaplane-accessible water sources would further enable investigation of mixed fleet compositions. The integration of ground-based suppression units alongside aerial agents could additionally broaden the operational scope and enable more realistic multi-domain suppression strategies.

The effect of direct inter-agent communication on coordination effectiveness could also be investigated, including which type of communication yields the greatest performance benefit and what information should be transmitted between agents. A more realistic scenario could furthermore incorporate dedicated fire monitoring agents that relay fire state information to the firefighting agents. Introducing a communication delay would require agents to operate on outdated fire state data, testing the robustness of the learned policy under partial observability.

Reconstruction of historical wildfire events could serve as a validation benchmark, allowing the learned policy to be evaluated against documented fire behavior and real suppression outcomes.

Bibliography

- Christopher Amato. An Introduction to Centralized Training with Decentralized Execution in Cooperative Multi-Agent Reinforcement Learning. In *Autonomous Agents and Multi-Agent Systems*. Springer, 2024.
- Anyscale, Inc. Rllib: Industry-grade reinforcement learning, 2024. URL <https://docs.ray.io/en/latest/rllib/index.html>. Accessed: 30 September 2025.
- Leonard Bardtke, Nabih Naeem, Nikolaos Kalliatakis, Prajwal Shiva Prakasha, and Thomas Clemen. Multi-agent deep reinforcement learning framework for efficient aerial wildfire fighting. *Engineering Proceedings*, 133(1), 2026. ISSN 2673-4591. doi: 10.3390/engproc2026133188. URL <https://www.mdpi.com/2673-4591/133/1/188>.
- Marc-André Blais and Moulay A. Akhloufi. Drone Swarm Coordination Using Reinforcement Learning for Efficient Wildfires Fighting. *SN Computer Science*, 5(3): 314, March 2024. ISSN 2661-8907. doi: 10.1007/s42979-024-02650-6. URL <https://doi.org/10.1007/s42979-024-02650-6>.
- Sayed Pedram Haeri Boroujeni, Abolfazl Razi, Sahand Khoshdel, Fatemeh Afghah, Janice L. Coen, Leo O'Neill, Peter Z. Fule, Adam Watts, Nick-Marios T. Kokolakis, and Kyriakos G. Vamvoudakis. A comprehensive survey of research towards AI-enabled unmanned aerial systems in pre-, active-, and post-wildfire management, January 2024. URL <http://arxiv.org/abs/2401.02456>. arXiv:2401.02456 [cs.LG].
- G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba. Openai gym. Technical Report arXiv:1606.01540, arXiv, 2016.
- Hao Chen, Yang Liu, Wei Zhang, and others. SOScheduler: Proactive Multi-UAV Scheduling for Wildfire Suppression. *IEEE Transactions on Intelligent Transportation Systems*, 2024a.

- Zhiwei Chen, Ruituo Feng, and Jian Li. Spatial-Temporal Return Decomposition for Multi-Agent Credit Assignment. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024b.
- N. Cigal, N. Naeem, P. Ratei, S. Kilgis, P. S. Prakasha, and B. Nagel. Sensitivity analysis of aerial wildfire fighting tactics with heterogeneous fleets using an agent-based simulation framework. *CEAS Aeronautical Journal*, 17(1):165–195, January 2026. ISSN 1869-5582, 1869-5590. doi: 10.1007/s13272-025-00840-3. URL <https://link.springer.com/10.1007/s13272-025-00840-3>.
- EFFIS. European forest fire information system, 2025. URL <https://forest-fire.emergency.copernicus.eu/>. Accessed: 30 September 2025.
- Jeffrey L. Elman. Learning and development in neural networks: the importance of starting small. *Cognition*, 48(1):71–99, July 1993. ISSN 0010-0277. doi: 10.1016/0010-0277(93)90058-4. URL <https://www.sciencedirect.com/science/article/pii/0010027793900584>.
- Ravi N. Haksar and Mac Schwager. Distributed Deep Reinforcement Learning for Fighting Forest Fires with a Network of Aerial Robots. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1067–1074, October 2018. doi: 10.1109/IROS.2018.8593539. URL <https://ieeexplore.ieee.org/document/8593539>. ISSN: 2153-0866.
- Kyle D. Julian and Mykel J. Kochenderfer. Distributed Wildfire Surveillance with Autonomous Aircraft Using Deep Reinforcement Learning. *Journal of Guidance, Control, and Dynamics*, 42(8):1768–1778, August 2019. ISSN 1533-3884. doi: 10.2514/1.G004106. URL <https://arc.aiaa.org/doi/10.2514/1.G004106>.
- Z. Juozapaitis, A. Koul, A. Fern, M. Erwig, and F. Doshi-Velez. Explainable reinforcement learning via reward decomposition. In *IJCAI/ECAI Workshop on Explainable Artificial Intelligence*, pages 47–53, 2019.
- S. Kilgis, P. S. Prakasha, N. Naeem, and B. Nagel. A python modelling and simulation toolkit for rapid development of system of systems inverse design (sosid) case studies. In *AIAA AVIATION 2021 Forum*, 2021.
- Chao Li, Yanfei Liu, Jiuling Wang, Zhong Wang, Chengjin Wang, and Qi Tian. Efficient evolutionary curriculum learning for scalable multi-agent reinforcement learning. *Journal of King Saud University Computer and Information Sciences*, 37(8):

- 243, September 2025. ISSN 2213-1248. doi: 10.1007/s44443-025-00215-y. URL <https://doi.org/10.1007/s44443-025-00215-y>.
- E. Liang, R. Liaw, R. Nishihara, P. Moritz, R. Fox, K. Goldberg, J. E. Gonzalez, M. I. Jordan, and I. Stoica. Rllib: Abstractions for distributed reinforcement learning. In *Proceedings of the International Conference on Machine Learning (ICML)*, Stockholm, Sweden, 2018. URL <https://arxiv.org/pdf/1712.09381>.
- Jiaxin Liang, Haotian Miao, Kai Li, Jianheng Tan, Xi Wang, Rui Luo, and Yueqiu Jiang. A Review of Multi-Agent Reinforcement Learning Algorithms. *Electronics*, 14 (4):820, February 2025. ISSN 2079-9292. doi: 10.3390/electronics14040820. URL <https://www.mdpi.com/2079-9292/14/4/820>.
- You-Dong Liang and B. A. Barsky. A New Concept and Method for Line Clipping. *ACM Transactions on Graphics*, 3(1):1–22, January 1984. ISSN 0730-0301, 1557-7368. doi: 10.1145/357332.357333. URL <https://dl.acm.org/doi/10.1145/357332.357333>.
- P. Moritz, R. Nishihara, S. Wang, A. Tumanov, E. Liang, I. Elibol, Z. Yang, W. Paul, M. I. Jordan, and I. Stoica. Ray: A distributed framework for emerging ai applications. In *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, Carlsbad, CA, USA, 2018.
- A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32. 2019.
- P. S. Prakasha et al. Establishing a collaborative open source agent based simulation environment for system of systems aviation problems. *CEAS Aeronautical Journal*, 2025. In press.
- Prajwal S. Prakasha, Björn Nagel, San Kilkis, Nabih Naeem, and Patrick Ratei. System of Systems Simulation Driven Wildfire Fighting Aircraft Design. In *AIAA AVIATION 2021 FORUM*, VIRTUAL EVENT, August 2021. American Institute of Aeronautics and Astronautics. ISBN 978-1-62410-610-1. doi: 10.2514/6.2021-2455. URL <https://arc.aiaa.org/doi/10.2514/6.2021-2455>.

- Yunpeng Qing, Shunyu Liu, Jie Song, Yang Zhou, Kaixuan Chen, Huiqiong Wang, and Mingli Song. A Survey on Explainable Reinforcement Learning: Concepts, Algorithms, Challenges, August 2025. URL <http://arxiv.org/abs/2211.06665>. arXiv:2211.06665 [cs].
- A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.
- Tabish Rashid, Mikayel Samvelyan, Christian Schroeder de Witt, Gregory Farquhar, Jakob Foerster, and Shimon Whiteson. QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning. *Journal of Machine Learning Research*, 21(178):1–51, 2020.
- Xiaoping Rui, Shan Hui, Xuetao Yu, Guangyuan Zhang, and Bin Wu. Forest fire spread simulation algorithm based on cellular automata. *Natural Hazards*, 91(1):309–319, March 2018. ISSN 0921-030X, 1573-0840. doi: 10.1007/s11069-017-3127-5. URL <http://link.springer.com/10.1007/s11069-017-3127-5>.
- J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. Technical Report arXiv:1707.06347, arXiv, 2017.
- John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-Dimensional Continuous Control Using Generalized Advantage Estimation, June 2015. URL <https://arxiv.org/abs/1506.02438v6>.
- Yael Sepon, Tobias Huber, Elisabeth André, and Ofra Amir. Integrating Policy Summaries with Reward Decomposition for Explaining Reinforcement Learning Agents. In Philippe Mathieu, Frank Dignum, Paulo Novais, and Fernando De La Prieta, editors, *Advances in Practical Applications of Agents, Multi-Agent Systems, and Cognitive Mimetics. The PAAMS Collection*, volume 13955, pages 320–332. Springer Nature Switzerland, Cham, 2023. ISBN 978-3-031-37615-3 978-3-031-37616-0. doi: 10.1007/978-3-031-37616-0_27. URL https://link.springer.com/10.1007/978-3-031-37616-0_27. Series Title: Lecture Notes in Computer Science.
- Ashish Kumar Shakya, Gopinatha Pillai, and Sohom Chakrabarty. Reinforcement learning algorithms: A brief survey. *Expert Systems with Applications*, 231:120495, November 2023. ISSN 09574174. doi: 10.1016/j.eswa.2023.120495. URL <https://linkinghub.elsevier.com/retrieve/pii/S0957417423009971>.

- Peter Sunehag, Guy Lever, Audrunas Gruslys, Wojciech M. Czarnecki, Vinicius Zambaldi, Max Jaderberg, Marc Lanctot, Nicolas Sonnerat, Joel Z. Leibo, Karl Tuyls, and Thore Graepel. Value-Decomposition Networks For Cooperative Multi-Agent Learning Based On Team Reward. In *Proceedings of the International Conference on Autonomous Agents and Multi-Agent Systems (AAMAS)*, pages 2085–2087, Stockholm, Sweden, 2018.
- Richard S. Sutton and Andrew G. Barto. *Reinforcement learning: an introduction*. Adaptive computation and machine learning series. The MIT Press, Cambridge, Massachusetts, second edition edition, 2018. ISBN 978-0-262-03924-6.
- Richard S Sutton, David McAllester, Satinder Singh, and Yishay Mansour. Policy Gradient Methods for Reinforcement Learning with Function Approximation. In S. Solla, T. Leen, and K. Müller, editors, *Advances in Neural Information Processing Systems*, volume 12. MIT Press, 1999. URL https://proceedings.neurips.cc/paper_files/paper/1999/file/464d828b85b0bed98e80ade0a5c43b0f-Paper.pdf.
- Reza Shami Tanha, Mohsen Hooshmand, and Mohsen Afsharchi. UAV-based Firefighting by Multi-agent Reinforcement Learning. In *2023 13th International Conference on Computer and Knowledge Engineering (ICCKE)*, pages 117–123, November 2023. doi: 10.1109/ICCKE60553.2023.10326250. URL <https://ieeexplore.ieee.org/document/10326250>. ISSN: 2643-279X.
- Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3-4): 279–292, May 1992. ISSN 0885-6125, 1573-0565. doi: 10.1007/BF00992698. URL <http://link.springer.com/10.1007/BF00992698>.
- Z. Wu, E. Liang, M. Luo, S. Mika, J. E. Gonzalez, and I. Stoica. Rllib flow: Distributed reinforcement learning is a dataflow problem. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. URL <https://proceedings.neurips.cc/paper/2021/file/2bce32ed409f5ebcee2a7b417ad9beed-Paper.pdf>.
- Chao Yu, Akash Velu, Eugene Vinitzky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The Surprising Effectiveness of PPO in Cooperative, Multi-Agent Games, November 2022. URL <http://arxiv.org/abs/2103.01955>. arXiv:2103.01955.
- Jiaming Zhao, Fei Wang, and Hao Sun. Real-Time Wildfire Detection from UAV Imagery Using YOLOv8. *Remote Sensing*, 16, 2024.

Yihe Zhou, Shunyu Liu, Yunpeng Qing, Kaixuan Chen, Tongya Zheng, Yanhao Huang, Jie Song, and Mingli Song. Is Centralized Training with Decentralized Execution Framework Centralized Enough for MARL?, May 2023. URL <http://arxiv.org/abs/2305.17352>. arXiv:2305.17352 [cs].

A Appendix

A.1 Training input file

```
1 {
2   "experiments": [
3     {
4       "name": "Experiment_Sintra_1",
5       "type": "decentralized",
6       "iterations": 5000,
7       "max_sim_iterations": 1000,
8       "max_burning_cells": 1000,
9       "num_agents": 4,
10      "resume_training": true,
11      "norm_rewards": true,
12      "norm_target_range": [-1, 1],
13      "rand_agent_pos": true,
14      "random_sparseness": 20,
15      "use_16_directions": true,
16      "load_wildfire": true,
17      "wildfire_filename": "wildfire_100_Default_Sintra.pkl",
18      "tactic": "ray_tactic_recharge",
19      "shared_policy": true,
20      "curriculum_learning": {
21        "enable": false,
22        "correction_coefficient_start": 0.2,
23        "correction_coefficient_step": 0.02,
24        "correction_coefficient_max": 0.3,
25        "max_mission_success_streak": 20
26      },
27      "hyperparameters": {
28        "lr": 5e-4,
29        "gamma": 0.99,
```



```

30         "use_gae": true,
31         "lambda_": 0.95,
32         "clip_param": 0.2,
33         "vf_loss_coeff": 0.5,
34         "entropy_coeff": 0.01,
35         "kl_coeff": 0.2,
36         "kl_target": 0.01,
37         "vf_share_layers": true,
38         "fcnet_hiddens": [
39             512,
40             256,
41             128
42         ]
43     },
44     "observations": {
45         "closest_n_fire_direction_observation": {
46             "num_fire_pos": 1
47         },
48         "ignition_center_direction_observation": {},
49         "fire_edges_observation": {},
50         "local_grid_observation": {
51             "grid_size": 15
52         }
53     },
54     "reward_function": {
55         "proximity_reward": {
56             "progress_threshold": 10,
57             "min_reward": -1,
58             "max_reward": 1,
59             "min_reward_distance": 50,
60             "weight": 0.15
61         },
62         "connected_suppression_reward": {
63             "min_reward": -10,
64             "max_reward": 10,
65             "min_reward_distance": 50,
66             "weight": 0.55
67         },
68         "timestep_penalty": {
69             "min_reward": -1,

```

```

70         "max_reward": -1,
71         "weight": 0.15
72     },
73     "inactivity_penalty": {
74         "min_reward_distance": 50,
75         "max_no_suppression_streak": 30,
76         "min_reward": -1,
77         "max_reward": 1,
78         "weight": 0.15
79     }
80 }
81 }
82 ]
83 }

```

A.2 Tools Used

Table A.1 lists the tools and resources used in the preparation of this Master thesis.

Table A.1: Tools and Resources Used

Tool	Usage
L ^A T _E X	Text composition and layout tool used to create this document
Claude	AI language model used for idea generation, literature exploration, and text refinement
DeepL	Neural machine translation service used for multilingual text conversion
PyCharm	Integrated development environment used for Python software development
VSCoDe	Source code editor used for code editing and development

Erklärung zur selbständigen Bearbeitung

Hiermit versichere ich, dass ich die vorliegende Arbeit ohne fremde Hilfe selbständig verfasst und nur die angegebenen Hilfsmittel benutzt habe. Wörtlich oder dem Sinn nach aus anderen Werken entnommene Stellen sind unter Angabe der Quellen kenntlich gemacht.

Ort

Datum

Unterschrift im Original